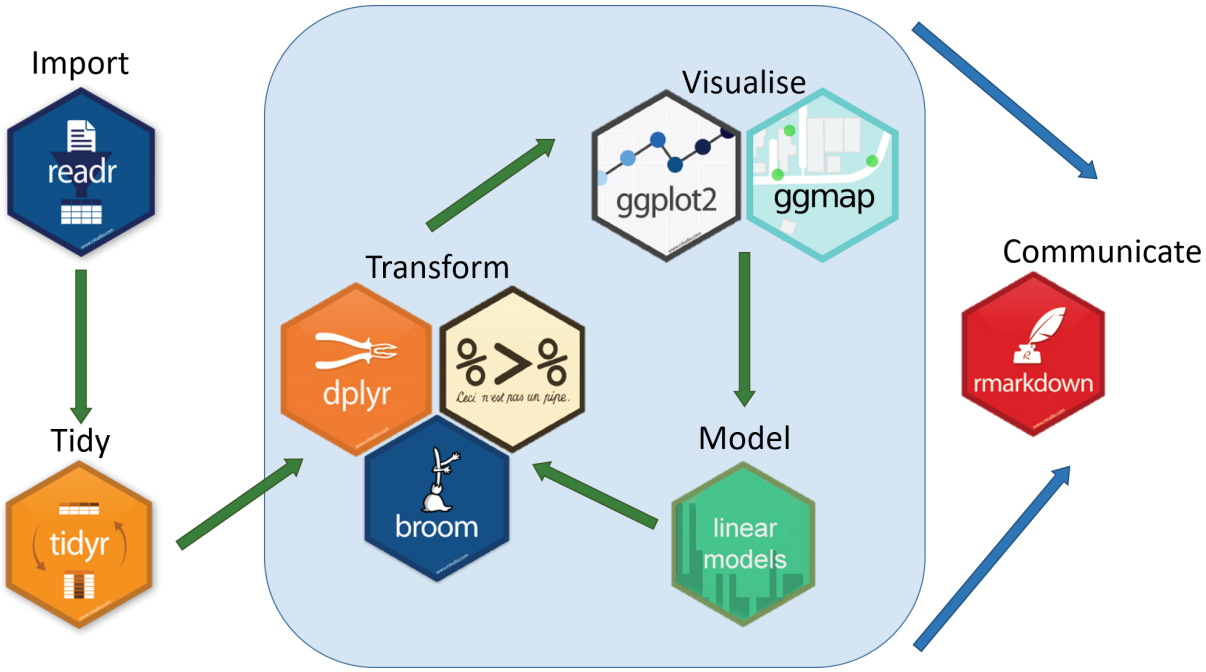
1 Introduction to the Tidyverse

In this class will get you started on the path to exploring and visualizing your own data with the R programming language.

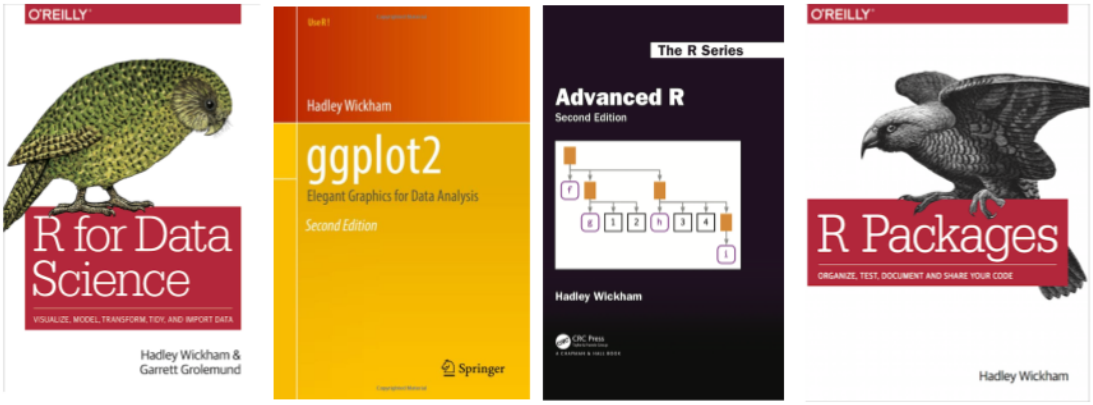
tidyverse is a collection of data science tools within R for transforming and visualizing data. This is not the only set of tools in R, but it's a powerful and popular approach for exploring data.



A useful R package for data science is **dplyr** created by Hadley Wickham (hadley.nz), which provides step-by-step tools for transforming this data, such as filtering, sorting, and summarizing it.







1.1 Gapminder

For this, we will use the dataset Gapminder. Gapminder tracks economic and social indicators like life expectancy and the GDP per capita of countries over time.

https://github.com/jennybc/gapminder (https://github.com/jennybc/gapminder)



what kinds of questions emerge from these data and how to answer them?

Let's starts

In [1]:
```
#install.packages("gapminder")
```

```
package 'gapminder' successfully unpacked and MD5 sums checked

The downloaded binary packages are in
        C:\Users\danie\AppData\Local\Temp\RtmpeKEfUp\downloaded_packages
```

In [1]:
```
library(gapminder)
```

```
Warning message:
"package 'gapminder' was built under R version 3.6.3"
```

In [2]:
```
gapminder
```

country	continent	year	lifeExp	pop	gdpPercap
Afghanistan	Asia	1952	28.801	8425333	779.4453
Afghanistan	Asia	1957	30.332	9240934	820.8530
Afghanistan	Asia	1962	31.997	10267083	853.1007
Afghanistan	Asia	1967	34.020	11537966	836.1971
Afghanistan	Asia	1972	36.088	13079460	739.9811
Afghanistan	Asia	1977	38.438	14880372	786.1134
Afghanistan	Asia	1982	39.854	12881816	978.0114
Afghanistan	Asia	1987	40.822	13867957	852.3959
Afghanistan	Asia	1992	41.674	16317921	649.3414
Afghanistan	Asia	1997	41.763	22227415	635.3414
Afghanistan	Asia	2002	42.129	25268405	726.7341
Afghanistan	Asia	2007	43.828	31889923	974.5803
Albania	Europe	1952	55.230	1282697	1601.0561
Albania	Europe	1957	59.280	1476505	1942.2842
Albania	Europe	1962	64.820	1728137	2312.8890
Albania	Europe	1967	66.220	1984060	2760.1969
Albania	Europe	1972	67.690	2263554	3313.4222
Albania	Europe	1977	68.930	2509048	3533.0039
Albania	Europe	1982	70.420	2780097	3630.8807
Albania	Europe	1987	72.000	3075321	3738.9327
Albania	Europe	1992	71.581	3326498	2497.4379
Albania	Europe	1997	72.950	3428038	3193.0546
Albania	Europe	2002	75.651	3508512	4604.2117
Albania	Europe	2007	76.423	3600523	5937.0295
Algeria	Africa	1952	43.077	9279525	2449.0082
Algeria	Africa	1957	45.685	10270856	3013.9760
Algeria	Africa	1962	48.303	11000948	2550.8169
Algeria	Africa	1967	51.407	12760499	3246.9918
Algeria	Africa	1972	54.518	14760787	4182.6638
Algeria	Africa	1977	58.014	17152804	4910.4168
...	...	...	...	...	...
Yemen, Rep.	Asia	1982	49.113	9657618	1977.5570
Yemen, Rep.	Asia	1987	52.922	11219340	1971.7415
Yemen, Rep.	Asia	1992	55.599	13367997	1879.4967

country	continent	year	lifeExp	pop	gdpPercap
Yemen, Rep.	Asia	1997	58.020	15826497	2117.4845
Yemen, Rep.	Asia	2002	60.308	18701257	2234.8208
Yemen, Rep.	Asia	2007	62.698	22211743	2280.7699
Zambia	Africa	1952	42.038	2672000	1147.3888
Zambia	Africa	1957	44.077	3016000	1311.9568
Zambia	Africa	1962	46.023	3421000	1452.7258
Zambia	Africa	1967	47.768	3900000	1777.0773
Zambia	Africa	1972	50.107	4506497	1773.4983
Zambia	Africa	1977	51.386	5216550	1588.6883
Zambia	Africa	1982	51.821	6100407	1408.6786
Zambia	Africa	1987	50.821	7272406	1213.3151
Zambia	Africa	1992	46.100	8381163	1210.8846
Zambia	Africa	1997	40.238	9417789	1071.3538
Zambia	Africa	2002	39.193	10595811	1071.6139
Zambia	Africa	2007	42.384	11746035	1271.2116
Zimbabwe	Africa	1952	48.451	3080907	406.8841
Zimbabwe	Africa	1957	50.469	3646340	518.7643
Zimbabwe	Africa	1962	52.358	4277736	527.2722
Zimbabwe	Africa	1967	53.995	4995432	569.7951
Zimbabwe	Africa	1972	55.635	5861135	799.3622
Zimbabwe	Africa	1977	57.674	6642107	685.5877
Zimbabwe	Africa	1982	60.363	7636524	788.8550
Zimbabwe	Africa	1987	62.351	9216418	706.1573
Zimbabwe	Africa	1992	60.377	10704340	693.4208
Zimbabwe	Africa	1997	46.809	11404948	792.4500
Zimbabwe	Africa	2002	39.989	11926563	672.0386
Zimbabwe	Africa	2007	43.487	12311143	469.7093

In [3]:
```
str(gapminder)
```

```
tibble [1,704 × 6] (S3: tbl_df/tbl/data.frame)
 $ country  : Factor w/ 142 levels "Afghanistan",..: 1 1 1 1 1 1 1 1 1 1 ...
 $ continent: Factor w/ 5 levels "Africa","Americas",..: 3 3 3 3 3 3 3 3 3 3
...
 $ year     : int [1:1704] 1952 1957 1962 1967 1972 1977 1982 1987 1992 1997
...
 $ lifeExp  : num [1:1704] 28.8 30.3 32 34 36.1 ...
 $ pop      : int [1:1704] 8425333 9240934 10267083 11537966 13079460 14880372
12881816 13867957 16317921 22227415 ...
 $ gdpPercap: num [1:1704] 779 821 853 836 740 ...
```

Gapminder is structured as a data frame. A data frame keeps rectangular data in rows and columns, similar to a spreadsheet, or a table in a SQL database. Most data analyses in R are centered around data frames.

```
gapminder
```

```
A tibble: 1704 × 6
```

This tells you the **tibble** has one 1704 rows, each of which we call an **observation**. It has six columns, each of which we call a **variable**. It's important in an analysis to understand what each observation, or row, represents.

Here, each represents a unique pair of a country and a year. For example, the first observation represents country statistics for Afghanistan in 1952, the second for Afghanistan in 1957, and so on. For each combination of a country and year, the dataset contains several variables, or columns, describing the country's demographics.

We see the continent - in this case, Asia - the life expectancy in years, the population, and the GDP per capita. The GDP per capita is the country's total economic output (Gross Domestic Product) divided by its population, and it's a common measure of how wealthy a country is. Each variable is of one consistent data type: some are numbers, like life expectancy and population and some are categorical, like country and continent.

1.2 Filter()

You use **filter** when you want to look only at a subset of your observations, based on a particular condition. Filtering data is a common first step in an analysis.

A pipe: %>% says "take whatever is before it, and feed it into the next step.

In [6]:
```
#install.packages('dplyr')
library(gapminder)
library(dplyr)

gapminder %>%
filter(continent == "Asia") %>%
filter(year == 2007)
```

country	continent	year	lifeExp	pop	gdpPercap
Afghanistan	Asia	2007	43.828	31889923	974.5803
Bahrain	Asia	2007	75.635	708573	29796.0483
Bangladesh	Asia	2007	64.062	150448339	1391.2538
Cambodia	Asia	2007	59.723	14131858	1713.7787
China	Asia	2007	72.961	1318683096	4959.1149
Hong Kong, China	Asia	2007	82.208	6980412	39724.9787
India	Asia	2007	64.698	1110396331	2452.2104
Indonesia	Asia	2007	70.650	223547000	3540.6516
Iran	Asia	2007	70.964	69453570	11605.7145
Iraq	Asia	2007	59.545	27499638	4471.0619
Israel	Asia	2007	80.745	6426679	25523.2771
Japan	Asia	2007	82.603	127467972	31656.0681
Jordan	Asia	2007	72.535	6053193	4519.4612
Korea, Dem. Rep.	Asia	2007	67.297	23301725	1593.0655
Korea, Rep.	Asia	2007	78.623	49044790	23348.1397
Kuwait	Asia	2007	77.588	2505559	47306.9898
Lebanon	Asia	2007	71.993	3921278	10461.0587
Malaysia	Asia	2007	74.241	24821286	12451.6558
Mongolia	Asia	2007	66.803	2874127	3095.7723
Myanmar	Asia	2007	62.069	47761980	944.0000
Nepal	Asia	2007	63.785	28901790	1091.3598
Oman	Asia	2007	75.640	3204897	22316.1929
Pakistan	Asia	2007	65.483	169270617	2605.9476
Philippines	Asia	2007	71.688	91077287	3190.4810
Saudi Arabia	Asia	2007	72.777	27601038	21654.8319
Singapore	Asia	2007	79.972	4553009	47143.1796
Sri Lanka	Asia	2007	72.396	20378239	3970.0954
Syria	Asia	2007	74.143	19314747	4184.5481
Taiwan	Asia	2007	78.400	23174294	28718.2768

country	continent	year	lifeExp	pop	gdpPercap
Thailand	Asia	2007	70.616	65068149	7458.3963
Vietnam	Asia	2007	74.249	85262356	2441.5764
West Bank and Gaza	Asia	2007	73.422	4018332	3025.3498
Yemen, Rep.	Asia	2007	62.698	22211743	2280.7699

Notice that now, we have only 142 rows: that's how many countries are in the dataset. It's important to note that you're not removing any rows from the original gapminder data. You can still use the gapminder object for other analyses.

In [9]:
```
gapminder %>%
filter(country == "United States")
```

country	continent	year	lifeExp	pop	gdpPercap
United States	Americas	1952	68.440	157553000	13990.48
United States	Americas	1957	69.490	171984000	14847.13
United States	Americas	1962	70.210	186538000	16173.15
United States	Americas	1967	70.760	198712000	19530.37
United States	Americas	1972	71.340	209896000	21806.04
United States	Americas	1977	73.380	220239000	24072.63
United States	Americas	1982	74.650	232187835	25009.56
United States	Americas	1987	75.020	242803533	29884.35
United States	Americas	1992	76.090	256894189	32003.93
United States	Americas	1997	76.810	272911760	35767.43
United States	Americas	2002	77.310	287675526	39097.10
United States	Americas	2007	78.242	301139947	42951.65

1.2.1 Multiple conditions

In [11]:
```
gapminder %>%
filter(year == 2007, country == "United States")
```

A tibble: 1 × 6

country	continent	year	lifeExp	pop	gdpPercap
<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
United States	Americas	2007	78.242	301139947	42951.65

1.3 Arrange()

arrange sorts the observations in a dataset, in ascending or descending order based on one of its variables. This is useful, for example, when you want to know the most extreme values in a dataset.

In [10]:
```
gapminder %>%
arrange(gdpPercap)
```

country	continent	year	lifeExp	pop	gdpPercap
Congo, Dem. Rep.	Africa	2002	44.966	55379852	241.1659
Congo, Dem. Rep.	Africa	2007	46.462	64606759	277.5519
Lesotho	Africa	1952	42.138	748747	298.8462
Guinea-Bissau	Africa	1952	32.500	580653	299.8503
Congo, Dem. Rep.	Africa	1997	42.587	47798986	312.1884
Eritrea	Africa	1952	35.928	1438760	328.9406
Myanmar	Asia	1952	36.319	20092996	331.0000
Lesotho	Africa	1957	45.047	813338	335.9971
Burundi	Africa	1952	39.031	2445618	339.2965
Eritrea	Africa	1957	38.047	1542611	344.1619
Myanmar	Asia	1992	59.320	40546538	347.0000
Myanmar	Asia	1967	49.379	25870271	349.0000
Myanmar	Asia	1957	41.905	21731844	350.0000
Burundi	Africa	1962	42.045	2961915	355.2032
Myanmar	Asia	1972	53.070	28466390	357.0000
Ethiopia	Africa	1952	34.078	20860941	362.1463
Cambodia	Asia	1952	39.417	4693836	368.4693
Malawi	Africa	1952	36.256	2917802	369.1651
Myanmar	Asia	1977	56.059	31528087	371.0000
Equatorial Guinea	Africa	1952	34.482	216964	375.6431
Ethiopia	Africa	1957	36.667	22815614	378.9042
Burundi	Africa	1957	40.533	2667518	379.5646
Eritrea	Africa	1962	40.158	1666618	380.9958
Myanmar	Asia	1987	58.339	38028578	385.0000
Myanmar	Asia	1962	45.108	23634436	388.0000
Mozambique	Africa	1987	42.861	12891952	389.8762
China	Asia	1952	44.000	556263527	400.4486
Zimbabwe	Africa	1952	48.451	3080907	406.8841
Mozambique	Africa	1992	44.284	13160731	410.8968
Lesotho	Africa	1962	47.747	893143	411.8006
...	...	...	...	...	...
Ireland	Europe	2002	77.783	3879155	34077.05
Saudi Arabia	Asia	1977	58.690	8128505	34167.76
Australia	Oceania	2007	81.235	20434176	34435.37

country	continent	year	lifeExp	pop	gdpPercap
Switzerland	Europe	2002	80.620	7361757	34480.96
Kuwait	Asia	1992	75.190	1418095	34932.92
Kuwait	Asia	2002	76.904	2111561	35110.11
Denmark	Europe	2007	78.332	5468120	35278.42
United States	Americas	1997	76.810	272911760	35767.43
Singapore	Asia	2002	78.770	4197776	36023.11
Austria	Europe	2007	79.829	8199783	36126.49
Iceland	Europe	2007	81.757	301931	36180.79
Canada	Americas	2007	80.653	33390141	36319.24
Netherlands	Europe	2007	79.762	16570613	36797.93
Switzerland	Europe	2007	81.701	7554661	37506.42
United States	Americas	2002	77.310	287675526	39097.10
Hong Kong, China	Asia	2007	82.208	6980412	39724.98
Kuwait	Asia	1997	76.156	1765345	40300.62
Ireland	Europe	2007	78.885	4109086	40676.00
Norway	Europe	1997	78.320	4405672	41283.16
United States	Americas	2007	78.242	301139947	42951.65
Norway	Europe	2002	79.050	4535591	44683.98
Singapore	Asia	2007	79.972	4553009	47143.18
Kuwait	Asia	2007	77.588	2505559	47306.99
Norway	Europe	2007	80.196	4627926	49357.19
Kuwait	Asia	1977	69.343	1140357	59265.48
Kuwait	Asia	1967	64.624	575003	80894.88
Kuwait	Asia	1962	60.470	358266	95458.11
Kuwait	Asia	1952	55.565	160000	108382.35
Kuwait	Asia	1972	67.712	841934	109347.87
Kuwait	Asia	1957	58.033	212846	113523.13

The observations are now sorted in ascending order, with the lowest GDP per capita appearing first. Look at the rightmost column: notice that it starts with 241, the smallest value in the dataset, then keeps increasing.

You can see that the country-year pair with the lowest GDP per capita was the Democratic Republic of the Congo in 2002.

Just like with filter, the gapminder object itself is unchanged: arrange is just giving you a new, sorted dataset.

Arrange also lets you sort in **descending order**.

In [13]:
```
gapminder %>%
arrange(desc(gdpPercap))
```

country	continent	year	lifeExp	pop	gdpPercap
Kuwait	Asia	1957	58.033	212846	113523.13
Kuwait	Asia	1972	67.712	841934	109347.87
Kuwait	Asia	1952	55.565	160000	108382.35
Kuwait	Asia	1962	60.470	358266	95458.11
Kuwait	Asia	1967	64.624	575003	80894.88
Kuwait	Asia	1977	69.343	1140357	59265.48
Norway	Europe	2007	80.196	4627926	49357.19
Kuwait	Asia	2007	77.588	2505559	47306.99
Singapore	Asia	2007	79.972	4553009	47143.18
Norway	Europe	2002	79.050	4535591	44683.98
United States	Americas	2007	78.242	301139947	42951.65
Norway	Europe	1997	78.320	4405672	41283.16
Ireland	Europe	2007	78.885	4109086	40676.00
Kuwait	Asia	1997	76.156	1765345	40300.62
Hong Kong, China	Asia	2007	82.208	6980412	39724.98
United States	Americas	2002	77.310	287675526	39097.10
Switzerland	Europe	2007	81.701	7554661	37506.42
Netherlands	Europe	2007	79.762	16570613	36797.93
Canada	Americas	2007	80.653	33390141	36319.24
Iceland	Europe	2007	81.757	301931	36180.79
Austria	Europe	2007	79.829	8199783	36126.49
Singapore	Asia	2002	78.770	4197776	36023.11
United States	Americas	1997	76.810	272911760	35767.43
Denmark	Europe	2007	78.332	5468120	35278.42
Kuwait	Asia	2002	76.904	2111561	35110.11
Kuwait	Asia	1992	75.190	1418095	34932.92
Switzerland	Europe	2002	80.620	7361757	34480.96
Australia	Oceania	2007	81.235	20434176	34435.37
Saudi Arabia	Asia	1977	58.690	8128505	34167.76
Ireland	Europe	2002	77.783	3879155	34077.05
...	...	...	...	...	...
Lesotho	Africa	1962	47.747	893143	411.8006
Mozambique	Africa	1992	44.284	13160731	410.8968
Zimbabwe	Africa	1952	48.451	3080907	406.8841

country	continent	year	lifeExp	pop	gdpPercap
China	Asia	1952	44.000	556263527	400.4486
Mozambique	Africa	1987	42.861	12891952	389.8762
Myanmar	Asia	1962	45.108	23634436	388.0000
Myanmar	Asia	1987	58.339	38028578	385.0000
Eritrea	Africa	1962	40.158	1666618	380.9958
Burundi	Africa	1957	40.533	2667518	379.5646
Ethiopia	Africa	1957	36.667	22815614	378.9042
Equatorial Guinea	Africa	1952	34.482	216964	375.6431
Myanmar	Asia	1977	56.059	31528087	371.0000
Malawi	Africa	1952	36.256	2917802	369.1651
Cambodia	Asia	1952	39.417	4693836	368.4693
Ethiopia	Africa	1952	34.078	20860941	362.1463
Myanmar	Asia	1972	53.070	28466390	357.0000
Burundi	Africa	1962	42.045	2961915	355.2032
Myanmar	Asia	1957	41.905	21731844	350.0000
Myanmar	Asia	1967	49.379	25870271	349.0000
Myanmar	Asia	1992	59.320	40546538	347.0000
Eritrea	Africa	1957	38.047	1542611	344.1619
Burundi	Africa	1952	39.031	2445618	339.2965
Lesotho	Africa	1957	45.047	813338	335.9971
Myanmar	Asia	1952	36.319	20092996	331.0000
Eritrea	Africa	1952	35.928	1438760	328.9406
Congo, Dem. Rep.	Africa	1997	42.587	47798986	312.1884
Guinea-Bissau	Africa	1952	32.500	580653	299.8503
Lesotho	Africa	1952	42.138	748747	298.8462
Congo, Dem. Rep.	Africa	2007	46.462	64606759	277.5519
Congo, Dem. Rep.	Africa	2002	44.966	55379852	241.1659

Looking across all countries and all years might not be that useful. Suppose you wanted to find the highest GDP-per-capita countries in just one year.

1.3.1 Combining filter and arrange

In [6]:
```
gapminder %>%
filter(year == 1952) %>%
arrange(desc(lifeExp))
```

country	continent	year	lifeExp	pop	gdpPercap
Norway	Europe	1952	72.670	3327728	10095.422
Iceland	Europe	1952	72.490	147962	7267.688
Netherlands	Europe	1952	72.130	10381988	8941.572
Sweden	Europe	1952	71.860	7124673	8527.845
Denmark	Europe	1952	70.780	4334000	9692.385
Switzerland	Europe	1952	69.620	4815000	14734.233
New Zealand	Oceania	1952	69.390	1994794	10556.576
United Kingdom	Europe	1952	69.180	50430000	9979.508
Australia	Oceania	1952	69.120	8691212	10039.596
Canada	Americas	1952	68.750	14785584	11367.161
United States	Americas	1952	68.440	157553000	13990.482
Belgium	Europe	1952	68.000	8730405	8343.105
Germany	Europe	1952	67.500	69145952	7144.114
France	Europe	1952	67.410	42459667	7029.809
Ireland	Europe	1952	66.910	2952156	5210.280
Czech Republic	Europe	1952	66.870	9125183	6876.140
Austria	Europe	1952	66.800	6927772	6137.076
Finland	Europe	1952	66.550	4090500	6424.519
Uruguay	Americas	1952	66.071	2252965	5716.767
Italy	Europe	1952	65.940	47666000	4931.404
Greece	Europe	1952	65.860	7733250	3530.690
Slovenia	Europe	1952	65.570	1489518	4215.042
Israel	Asia	1952	65.390	1620914	4086.522
Spain	Europe	1952	64.940	28549870	3834.035
Slovak Republic	Europe	1952	64.360	3558137	5074.659
Puerto Rico	Americas	1952	64.280	2227000	3081.960
Hungary	Europe	1952	64.030	9504000	5263.674
Japan	Asia	1952	63.030	86459025	3216.956
Paraguay	Americas	1952	62.649	1555876	1952.309
Argentina	Americas	1952	62.485	17876956	5911.315
...	...	...	...	...	...
Chad	Africa	1952	38.092	2682462	1178.6659
Haiti	Americas	1952	37.579	3201488	1840.3669

country	continent	year	lifeExp	pop	gdpPercap
Oman	Asia	1952	37.578	507833	1828.2303
Bangladesh	Asia	1952	37.484	46886859	684.2442
Indonesia	Asia	1952	37.468	82052000	749.6817
Niger	Africa	1952	37.444	3379468	761.8794
India	Asia	1952	37.373	372000000	546.5657
Senegal	Africa	1952	37.278	2755589	1450.3570
Gabon	Africa	1952	37.003	420702	4293.4765
Madagascar	Africa	1952	36.681	4762912	1443.0117
Nigeria	Africa	1952	36.324	33119096	1077.2819
Myanmar	Asia	1952	36.319	20092996	331.0000
Malawi	Africa	1952	36.256	2917802	369.1651
Nepal	Asia	1952	36.157	9182536	545.8657
Eritrea	Africa	1952	35.928	1438760	328.9406
Central African Republic	Africa	1952	35.463	1291695	1071.3107
Djibouti	Africa	1952	34.812	63149	2669.5295
Equatorial Guinea	Africa	1952	34.482	216964	375.6431
Ethiopia	Africa	1952	34.078	20860941	362.1463
Mali	Africa	1952	33.685	3838168	452.3370
Guinea	Africa	1952	33.609	2664249	510.1965
Somalia	Africa	1952	32.978	2526994	1135.7498
Yemen, Rep.	Asia	1952	32.548	4963829	781.7176
Guinea-Bissau	Africa	1952	32.500	580653	299.8503
Burkina Faso	Africa	1952	31.975	4469979	543.2552
Mozambique	Africa	1952	31.286	6446316	468.5260
Sierra Leone	Africa	1952	30.331	2143249	879.7877
Angola	Africa	1952	30.015	4232095	3520.6103
Gambia	Africa	1952	30.000	284320	485.2307
Afghanistan	Asia	1952	28.801	8425333	779.4453

1.4 Mutate()

Suppose you want to change one of the variables in your dataset, based on the other ones. Or suppose you want to add a new variable. For that, you would use the **mutate** function.

1.4.1 Modifying a variable with mutate

In [7]:
```
gapminder
gapminder %>%
mutate(pop = pop/1000000)
```

country	continent	year	lifeExp	pop	gdpPercap
Afghanistan	Asia	1952	28.801	8425333	779.4453
Afghanistan	Asia	1957	30.332	9240934	820.8530
Afghanistan	Asia	1962	31.997	10267083	853.1007
Afghanistan	Asia	1967	34.020	11537966	836.1971
Afghanistan	Asia	1972	36.088	13079460	739.9811
Afghanistan	Asia	1977	38.438	14880372	786.1134
Afghanistan	Asia	1982	39.854	12881816	978.0114
Afghanistan	Asia	1987	40.822	13867957	852.3959
Afghanistan	Asia	1992	41.674	16317921	649.3414
Afghanistan	Asia	1997	41.763	22227415	635.3414
Afghanistan	Asia	2002	42.129	25268405	726.7341
Afghanistan	Asia	2007	43.828	31889923	974.5803

This is how you manipulate existing variables in the table. That's often necessary during data processing and cleaning. Just like filter and arrange, you're not altering the original gapminder data, you're just changing the value in this new data frame.

1.4.2 Adding a new variable with mutate

you may want to add a new variable. For instance, you currently have the GDP per capita: that's the Gross Domestic Product of the country divided by the current population. But in your analysis, you may want to know the total GDP. You'd have to multiply the population by the GDP per capita to find this.

In [18]:
```
gapminder %>%
mutate(gpd = gdpPercap * pop)
```

country	continent	year	lifeExp	pop	gdpPercap	gpd
Afghanistan	Asia	1952	28.801	8425333	779.4453	6567086330
Afghanistan	Asia	1957	30.332	9240934	820.8530	7585448670
Afghanistan	Asia	1962	31.997	10267083	853.1007	8758855797
Afghanistan	Asia	1967	34.020	11537966	836.1971	9648014150
Afghanistan	Asia	1972	36.088	13079460	739.9811	9678553274
Afghanistan	Asia	1977	38.438	14880372	786.1134	11697659231
Afghanistan	Asia	1982	39.854	12881816	978.0114	12598563401
Afghanistan	Asia	1987	40.822	13867957	852.3959	11820990309
Afghanistan	Asia	1992	41.674	16317921	649.3414	10595901589
Afghanistan	Asia	1997	41.763	22227415	635.3414	14121995875
Afghanistan	Asia	2002	42.129	25268405	726.7341	18363410424
Afghanistan	Asia	2007	43.828	31889923	974.5803	31079291949
Albania	Europe	1952	55.230	1282697	1601.0561	2053669902
Albania	Europe	1957	59.280	1476505	1942.2842	2867792398
Albania	Europe	1962	64.820	1728137	2312.8890	3996988985
Albania	Europe	1967	66.220	1984060	2760.1969	5476396323
Albania	Europe	1972	67.690	2263554	3313.4222	7500110047
Albania	Europe	1977	68.930	2509048	3533.0039	8864476394
Albania	Europe	1982	70.420	2780097	3630.8807	10094200603
Albania	Europe	1987	72.000	3075321	3738.9327	11498418358
Albania	Europe	1992	71.581	3326498	2497.4379	8307722183
Albania	Europe	1997	72.950	3428038	3193.0546	10945912519
Albania	Europe	2002	75.651	3508512	4604.2117	16153932130
Albania	Europe	2007	76.423	3600523	5937.0295	21376411360
Algeria	Africa	1952	43.077	9279525	2449.0082	22725632678
Algeria	Africa	1957	45.685	10270856	3013.9760	30956113720
Algeria	Africa	1962	48.303	11000948	2550.8169	28061403854
Algeria	Africa	1967	51.407	12760499	3246.9918	41433235247
Algeria	Africa	1972	54.518	14760787	4182.6638	61739408943
Algeria	Africa	1977	58.014	17152804	4910.4168	84227416174
...	...	...	...	...	...	...
Yemen, Rep.	Asia	1982	49.113	9657618	1977.5570	19098490176
Yemen, Rep.	Asia	1987	52.922	11219340	1971.7415	22121638707
Yemen, Rep.	Asia	1992	55.599	13367997	1879.4967	25125105886

country	continent	year	lifeExp	pop	gdpPercap	gpd
Yemen, Rep.	Asia	1997	58.020	15826497	2117.4845	33512362498
Yemen, Rep.	Asia	2002	60.308	18701257	2234.8208	41793958635
Yemen, Rep.	Asia	2007	62.698	22211743	2280.7699	50659874994
Zambia	Africa	1952	42.038	2672000	1147.3888	3065822956
Zambia	Africa	1957	44.077	3016000	1311.9568	3956861606
Zambia	Africa	1962	46.023	3421000	1452.7258	4969774845
Zambia	Africa	1967	47.768	3900000	1777.0773	6930601540
Zambia	Africa	1972	50.107	4506497	1773.4983	7992264611
Zambia	Africa	1977	51.386	5216550	1588.6883	8287471946
Zambia	Africa	1982	51.821	6100407	1408.6786	8593512579
Zambia	Africa	1987	50.821	7272406	1213.3151	8823720129
Zambia	Africa	1992	46.100	8381163	1210.8846	10148621483
Zambia	Africa	1997	40.238	9417789	1071.3538	10089784202
Zambia	Africa	2002	39.193	10595811	1071.6139	11354618752
Zambia	Africa	2007	42.384	11746035	1271.2116	14931695864
Zimbabwe	Africa	1952	48.451	3080907	406.8841	1253572117
Zimbabwe	Africa	1957	50.469	3646340	518.7643	1891590901
Zimbabwe	Africa	1962	52.358	4277736	527.2722	2255531194
Zimbabwe	Africa	1967	53.995	4995432	569.7951	2846372532
Zimbabwe	Africa	1972	55.635	5861135	799.3622	4685169626
Zimbabwe	Africa	1977	57.674	6642107	685.5877	4553746742
Zimbabwe	Africa	1982	60.363	7636524	788.8550	6024110454
Zimbabwe	Africa	1987	62.351	9216418	706.1573	6508240905
Zimbabwe	Africa	1992	60.377	10704340	693.4208	7422611852
Zimbabwe	Africa	1997	46.809	11404948	792.4500	9037850590
Zimbabwe	Africa	2002	39.989	11926563	672.0386	8015110972
Zimbabwe	Africa	2007	43.487	12311143	469.7093	5782658337

1.4.3 Combining filter, arrange and mutate

Suppose we wanted to know the countries with the highest total GDP in the year 2007. We would do this in three steps- create the column, filter for 2007, then sort. We use mutate to create the total GDP column. Then we'd use filter to restrict our analysis to only the year 2007. Then we'd use arrange to sort the data in descending order of our new GDP variable.

In [8]:
```
gapminder %>%
mutate(gdp = gdpPercap * pop) %>%
filter(year == 2007) %>%
arrange(desc(gdp))
```

country	continent	year	lifeExp	pop	gdpPercap	gdp
United States	Americas	2007	78.242	301139947	42951.653	1.293446e+13
China	Asia	2007	72.961	1318683096	4959.115	6.539501e+12
Japan	Asia	2007	82.603	127467972	31656.068	4.035135e+12
India	Asia	2007	64.698	1110396331	2452.210	2.722925e+12
Germany	Europe	2007	79.406	82400996	32170.374	2.650871e+12
United Kingdom	Europe	2007	79.425	60776238	33203.261	2.017969e+12
France	Europe	2007	80.657	61083916	30470.017	1.861228e+12
Brazil	Americas	2007	72.390	190010647	9065.801	1.722599e+12
Italy	Europe	2007	80.546	58147733	28569.720	1.661264e+12
Mexico	Americas	2007	76.195	108700891	11977.575	1.301973e+12
Canada	Americas	2007	80.653	33390141	36319.235	1.212704e+12
Spain	Europe	2007	80.941	40448191	28821.064	1.165760e+12
Korea, Rep.	Asia	2007	78.623	49044790	23348.140	1.145105e+12
Iran	Asia	2007	70.964	69453570	11605.714	8.060583e+11
Indonesia	Asia	2007	70.650	223547000	3540.652	7.915020e+11
Australia	Oceania	2007	81.235	20434176	34435.367	7.036584e+11
Taiwan	Asia	2007	78.400	23174294	28718.277	6.655258e+11
Netherlands	Europe	2007	79.762	16570613	36797.933	6.097643e+11
Turkey	Europe	2007	71.777	71158647	8458.276	6.018795e+11
Saudi Arabia	Asia	2007	72.777	27601038	21654.832	5.976958e+11
Poland	Europe	2007	75.563	38518241	15389.925	5.927928e+11
Argentina	Americas	2007	75.320	40301927	12779.380	5.150336e+11
Thailand	Asia	2007	70.616	65068149	7458.396	4.853040e+11
Egypt	Africa	2007	71.338	80264543	5581.181	4.479709e+11
Pakistan	Asia	2007	65.483	169270617	2605.948	4.411104e+11
South Africa	Africa	2007	49.339	43997828	9269.658	4.078448e+11
Belgium	Europe	2007	79.441	10392226	33692.605	3.501412e+11
Colombia	Americas	2007	72.889	44227550	7006.580	3.098839e+11
Malaysia	Asia	2007	74.241	24821286	12451.656	3.090661e+11
Sweden	Europe	2007	80.884	9031088	33859.748	3.057904e+11
...	...	...	...	...	...	...
Mali	Africa	2007	54.467	12031795	1042.5816	12544127565

country	continent	year	lifeExp	pop	gdpPercap	gdp
West Bank and Gaza	Asia	2007	73.422	4018332	3025.3498	12156859904
Benin	Africa	2007	56.728	8078314	1441.2849	11643151768
Iceland	Europe	2007	81.757	301931	36180.7892	10924101861
Haiti	Americas	2007	60.916	8502814	1201.6372	10217297216
Malawi	Africa	2007	48.303	13327079	759.3499	10119916241
Namibia	Africa	2007	52.906	2055080	4811.0604	9887114066
Guinea	Africa	2007	56.007	9947814	942.6542	9377348758
Mongolia	Asia	2007	66.803	2874127	3095.7723	8897642670
Somalia	Africa	2007	48.159	9118773	926.1411	8445270168
Niger	Africa	2007	56.867	12894865	619.6769	7990649871
Rwanda	Africa	2007	46.242	8860588	863.0885	7647471286
Equatorial Guinea	Africa	2007	51.579	551201	12154.0897	6699346424
Montenegro	Europe	2007	74.543	684736	9253.8961	6336475807
Reunion	Africa	2007	76.442	798094	7670.1226	6121478793
Mauritania	Africa	2007	64.164	3270065	1803.1515	5896422597
Zimbabwe	Africa	2007	43.487	12311143	469.7093	5782658337
Sierra Leone	Africa	2007	42.568	6144562	862.5408	5299935153
Swaziland	Africa	2007	39.613	1133066	4513.4806	5114071458
Togo	Africa	2007	58.420	5701579	882.9699	5034322889
Burundi	Africa	2007	49.580	8390505	430.0707	3608510288
Lesotho	Africa	2007	42.592	2012649	1569.3314	3158513357
Eritrea	Africa	2007	58.040	4906585	641.3695	3146934084
Central African Republic	Africa	2007	44.741	4369038	706.0165	3084613079
Liberia	Africa	2007	45.678	3193942	414.5073	1323912407
Gambia	Africa	2007	59.448	1688359	752.7497	1270911775
Djibouti	Africa	2007	54.791	496374	2082.4816	1033689705
Guinea-Bissau	Africa	2007	46.388	1472041	579.2317	852652874
Comoros	Africa	2007	65.152	710960	986.1479	701111696
Sao Tome and Principe	Africa	2007	65.528	199579	1598.4351	319014077

2 Visualizing with ggplot2

Previously, you used the dplyr package to answer some questions about the gapminder dataset. You've been able to filter for particular observations, arrange to find the highest or lowest values, and mutate to add new columns. However, so far you've engaged with the results only as a table

printed out from your code. Often a better way to understand and present this kind of data is as a graph.

Now, we will learn the essential skill of data visualization using the **ggplot2 package**. Here, we will see how the **dplyr** and **ggplot2** packages work closely together to create informative graphs.

When you're working with just a subset, it's useful to save the filtered data, as a new data frame.

In [7]:
```
gapminder_2007 <- gapminder %>%
filter(year ==2007)

gapminder_2007
```

country	continent	year	lifeExp	pop	gdpPercap
Afghanistan	Asia	2007	43.828	31889923	974.5803
Albania	Europe	2007	76.423	3600523	5937.0295
Algeria	Africa	2007	72.301	33333216	6223.3675
Angola	Africa	2007	42.731	12420476	4797.2313
Argentina	Americas	2007	75.320	40301927	12779.3796
Australia	Oceania	2007	81.235	20434176	34435.3674
Austria	Europe	2007	79.829	8199783	36126.4927
Bahrain	Asia	2007	75.635	708573	29796.0483
Bangladesh	Asia	2007	64.062	150448339	1391.2538
Belgium	Europe	2007	79.441	10392226	33692.6051
Benin	Africa	2007	56.728	8078314	1441.2849
Bolivia	Americas	2007	65.554	9119152	3822.1371
Bosnia and Herzegovina	Europe	2007	74.852	4552198	7446.2988
Botswana	Africa	2007	50.728	1639131	12569.8518
Brazil	Americas	2007	72.390	190010647	9065.8008
Bulgaria	Europe	2007	73.005	7322858	10680.7928
Burkina Faso	Africa	2007	52.295	14326203	1217.0330
Burundi	Africa	2007	49.580	8390505	430.0707
Cambodia	Asia	2007	59.723	14131858	1713.7787
Cameroon	Africa	2007	50.430	17696293	2042.0952
Canada	Americas	2007	80.653	33390141	36319.2350
Central African Republic	Africa	2007	44.741	4369038	706.0165
Chad	Africa	2007	50.651	10238807	1704.0637
Chile	Americas	2007	78.553	16284741	13171.6388
China	Asia	2007	72.961	1318683096	4959.1149
Colombia	Americas	2007	72.889	44227550	7006.5804
Comoros	Africa	2007	65.152	710960	986.1479
Congo, Dem. Rep.	Africa	2007	46.462	64606759	277.5519
Congo, Rep.	Africa	2007	55.322	3800610	3632.5578
Costa Rica	Americas	2007	78.782	4133884	9645.0614
...	...	...	...	...	...
Sierra Leone	Africa	2007	42.568	6144562	862.5408

country	continent	year	lifeExp	pop	gdpPercap
Singapore	Asia	2007	79.972	4553009	47143.1796
Slovak Republic	Europe	2007	74.663	5447502	18678.3144
Slovenia	Europe	2007	77.926	2009245	25768.2576
Somalia	Africa	2007	48.159	9118773	926.1411
South Africa	Africa	2007	49.339	43997828	9269.6578
Spain	Europe	2007	80.941	40448191	28821.0637
Sri Lanka	Asia	2007	72.396	20378239	3970.0954
Sudan	Africa	2007	58.556	42292929	2602.3950
Swaziland	Africa	2007	39.613	1133066	4513.4806
Sweden	Europe	2007	80.884	9031088	33859.7484
Switzerland	Europe	2007	81.701	7554661	37506.4191
Syria	Asia	2007	74.143	19314747	4184.5481
Taiwan	Asia	2007	78.400	23174294	28718.2768
Tanzania	Africa	2007	52.517	38139640	1107.4822
Thailand	Asia	2007	70.616	65068149	7458.3963
Togo	Africa	2007	58.420	5701579	882.9699
Trinidad and Tobago	Americas	2007	69.819	1056608	18008.5092
Tunisia	Africa	2007	73.923	10276158	7092.9230
Turkey	Europe	2007	71.777	71158647	8458.2764
Uganda	Africa	2007	51.542	29170398	1056.3801
United Kingdom	Europe	2007	79.425	60776238	33203.2613
United States	Americas	2007	78.242	301139947	42951.6531
Uruguay	Americas	2007	76.384	3447496	10611.4630
Venezuela	Americas	2007	73.747	26084662	11415.8057
Vietnam	Asia	2007	74.249	85262356	2441.5764
West Bank and Gaza	Asia	2007	73.422	4018332	3025.3498
Yemen, Rep.	Asia	2007	62.698	22211743	2280.7699
Zambia	Africa	2007	42.384	11746035	1271.2116
Zimbabwe	Africa	2007	43.487	12311143	469.7093

We can see that it's another table. But this one has only 142 rows, and they come only from the year 2007. Now that you've saved this variable, you can use it to create our visualization.

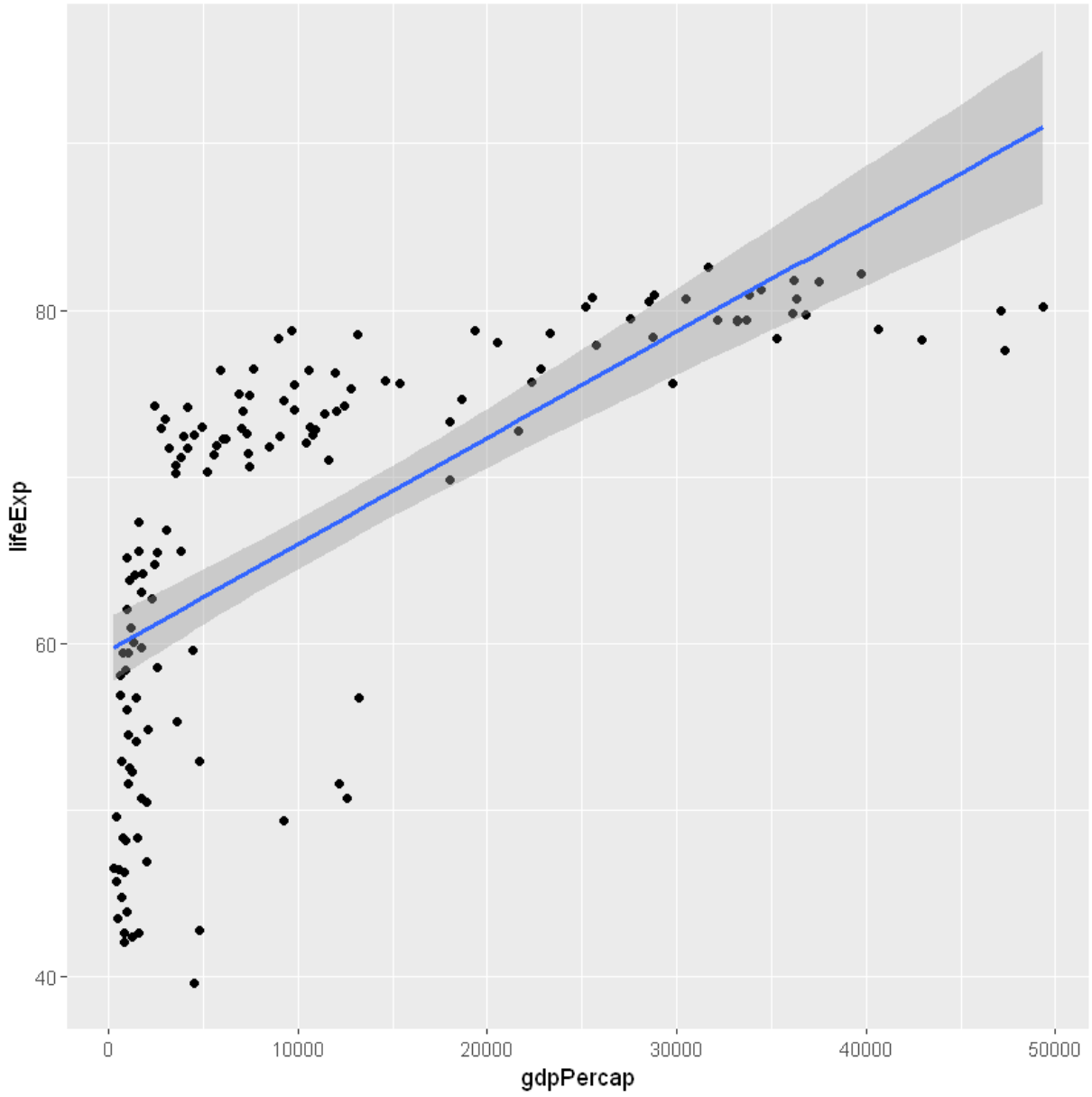
Suppose you want to examine the relationship between a country's wealth and its life expectancy. You could do this with a scatterplot comparing two variables in our gapminder dataset: GDP per capita on the X axis and life expectancy on the y-axis.

In [8]:
```
#install.packages("ggplot2")
library(ggplot2)
```

```
Registered S3 methods overwritten by 'ggplot2':
  method         from
  [.quosures     rlang
  c.quosures     rlang
  print.quosures rlang
```

In [9]:
```
ggplot(gapminder_2007, aes(x = gdpPercap, y = lifeExp)) +
      geom_point()+
    geom_smooth(method="lm")

#geom_smooth(method="lm")
#?geom_smooth
```



2.1 Scatter plots

2.1.1 What is a scatter plot?

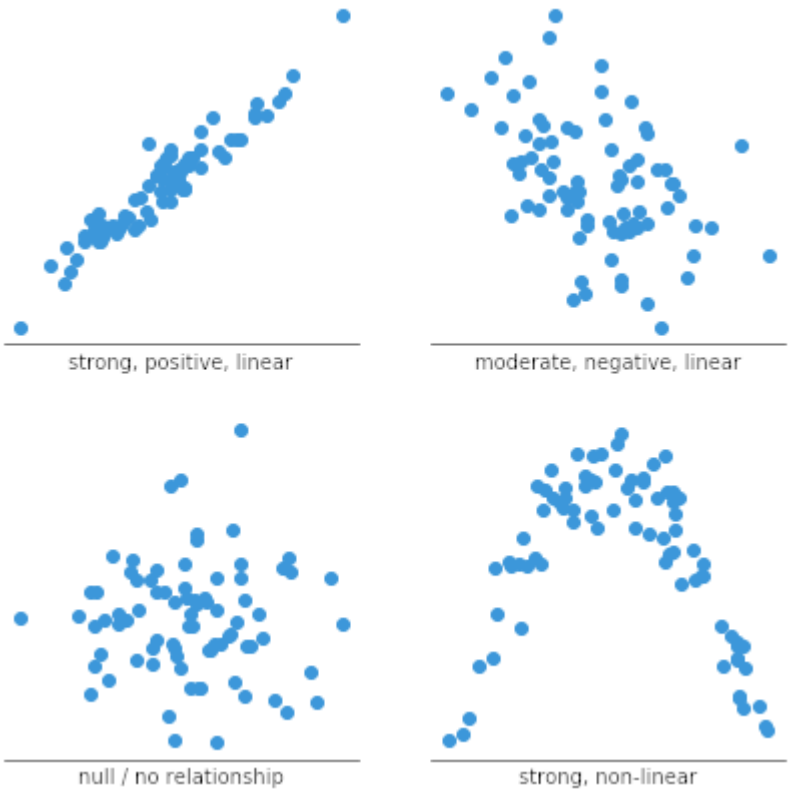
A scatter plot (aka scatter chart, scatter graph) uses dots to represent values for two different numeric variables.

The position of each dot on the horizontal and vertical axis indicates values for an individual data point.

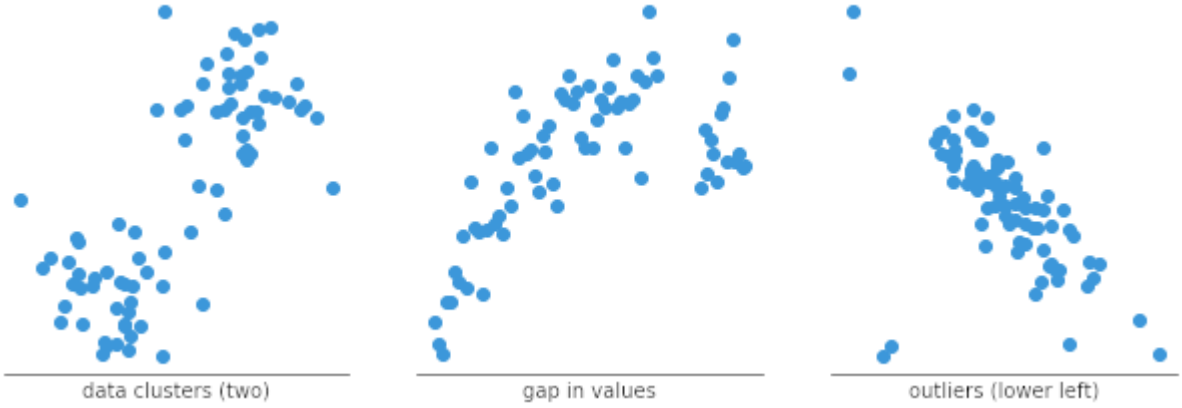
Scatter plots are used to observe relationships between variables such as GDP per capita and life expectancy in gapminder dataset.

2.1.2 When you should use a scatter plot

Scatter plots' primary uses are to observe and show relationships between two numeric variables. The dots in a scatter plot not only report the values of individual data points, but also patterns when the data are taken as a whole.



A scatter plot can also be useful for identifying other patterns in data. We can divide data points into groups based on how closely sets of points cluster together. Scatter plots can also show if there are any unexpected gaps in the data and if there are any outlier points. This can be useful if we want to segment the data into different parts.



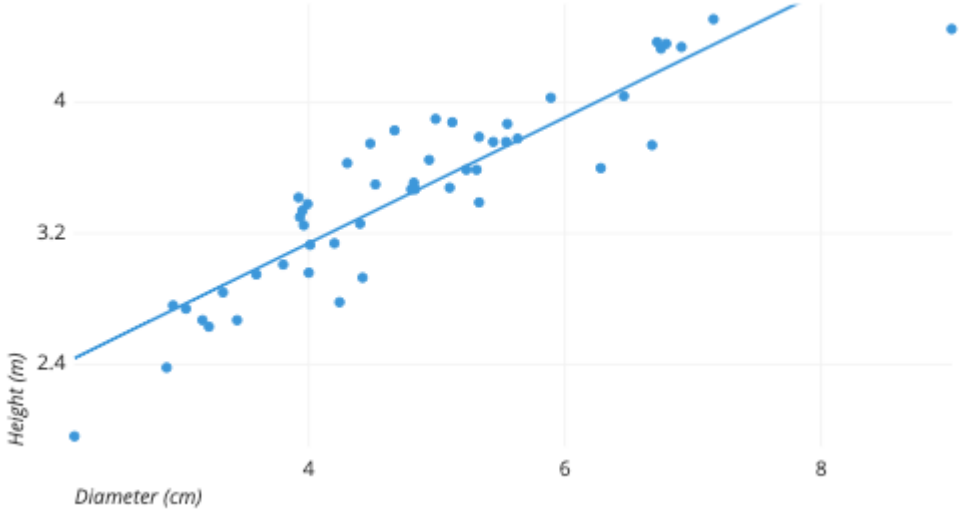
2.1.3 Common issues when using scatter plots

This is not so much an issue with creating a scatter plot as it is an issue with its interpretation. Simply because we observe a relationship between two variables in a scatter plot, it does not mean that changes in one variable are responsible for changes in the other. This gives rise to the common phrase in statistics that **correlation does not imply causation**. It is possible that the observed relationship is driven by some third variable that affects both of the plotted variables, that the causal link is reversed, or that the pattern is simply coincidental.

2.1.4 Common scatter plot options

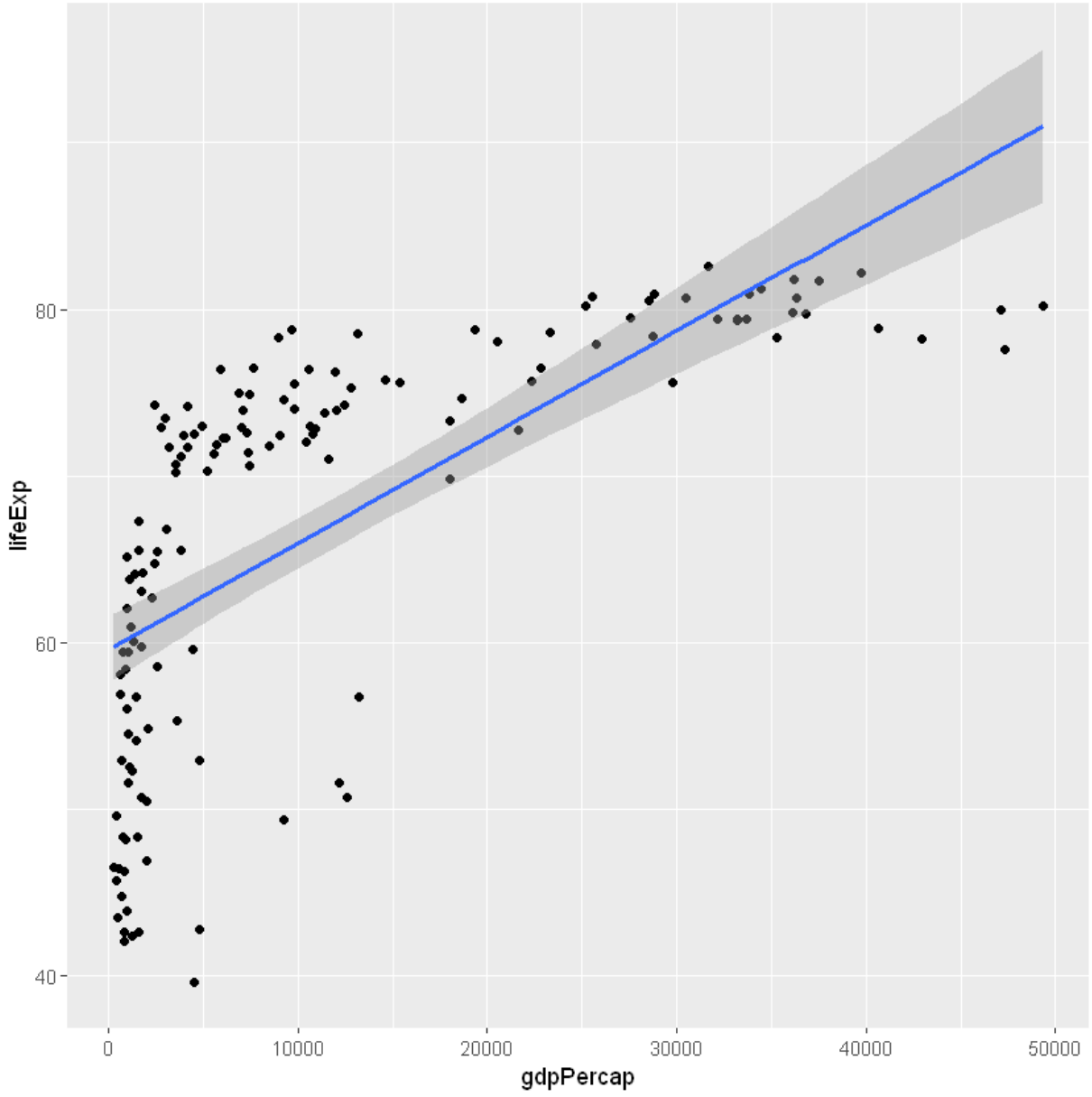
2.1.4.1 Add a trend line

When a scatter plot is used to look at a predictive or correlational relationship between variables, it is common to add a trend line to the plot showing the mathematically best fit to the data. This can provide an additional signal as to how strong the relationship between the two variables is, and if there are any unusual points that are affecting the computation of the trend line.



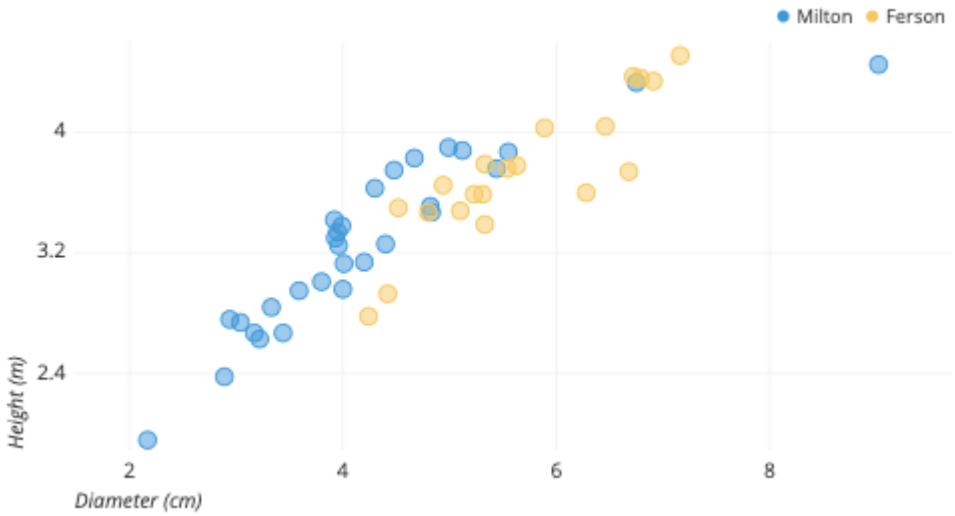
In [13]:
```
ggplot(gapminder_2007, aes(x = gdpPercap, y = lifeExp)) +
      geom_point()+
    geom_smooth(method="lm")

#geom_smooth(method="lm")
#?geom_smooth
```



2.1.4.2 Categorical third variable

A common modification of the basic scatter plot is the addition of a third variable. Values of the third variable can be encoded by modifying how the points are plotted. For a third variable that indicates categorical values (like geographical region or gender), the most common encoding is through point color. Giving each point a distinct hue makes it easy to show membership of each point to a respective group.

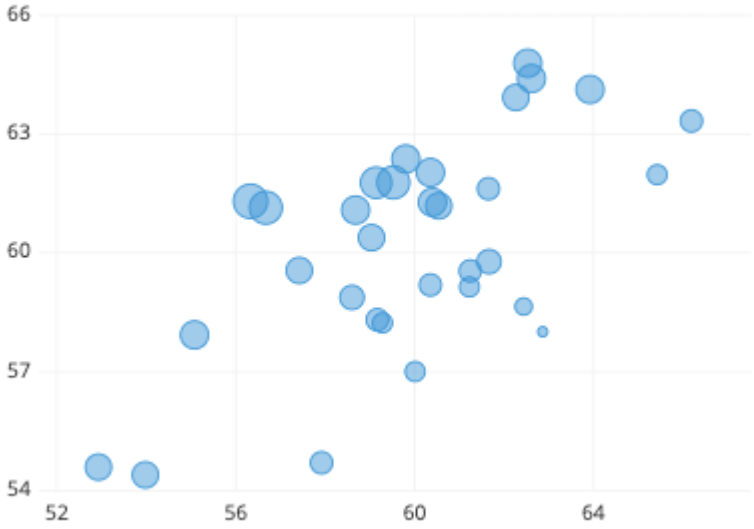


One other option that is sometimes seen for third-variable encoding is that of shape. One potential issue with shape is that different shapes can have different sizes and surface areas, which can have an effect on how groups are perceived. However, in certain cases where color cannot be used (like in print), shape may be the best option for distinguishing between groups.

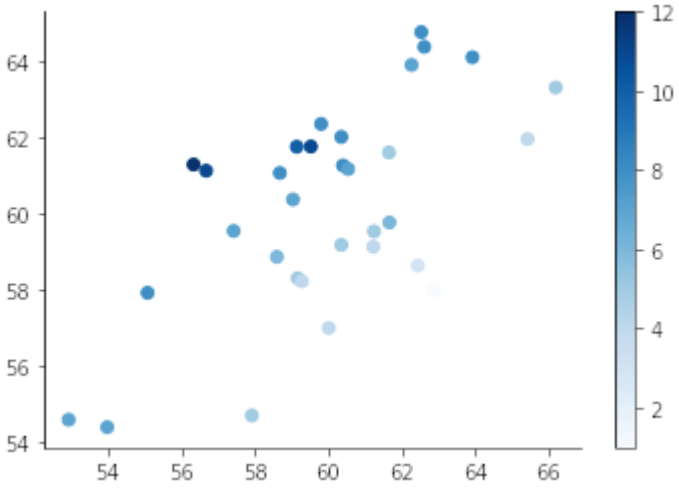


2.1.4.3 Numeric third variable

For third variables that have numeric values, a common encoding comes from changing the point size. A scatter plot with point size based on a third variable actually goes by a distinct name, the **bubble chart**. Larger points indicate higher values.



Another alternative. Rather than using distinct colors for points like in the categorical case, we want to use a continuous sequence of colors, so that, for example, darker colors indicate higher value. Note that, for both size and color, a legend is important for interpretation of the third variable, since our eyes are much less able to discern size and color as easily as position.

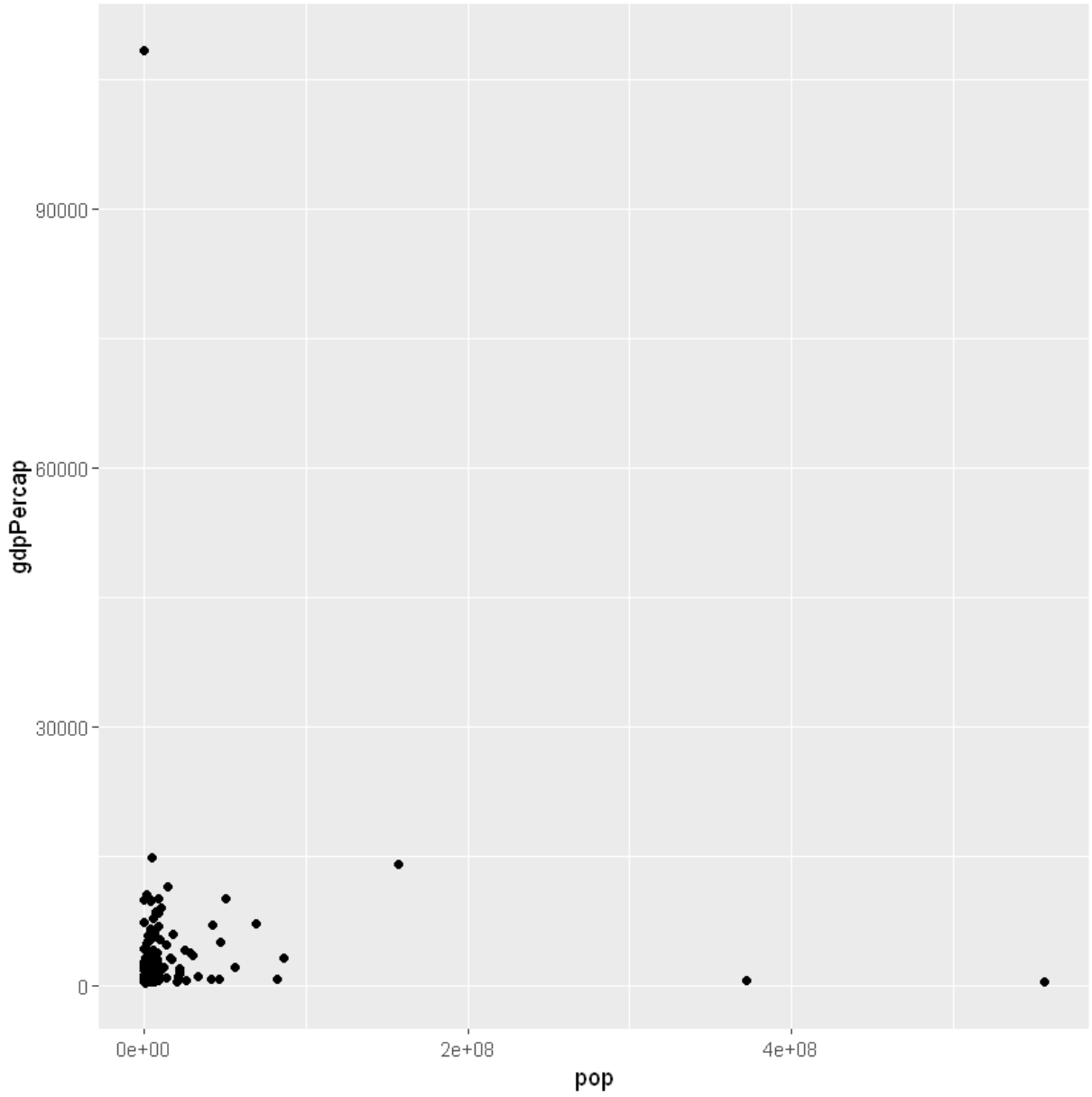


as homework, find out how to implement points 2.1.4.1 to 2.1.4.3 in ggplot

explore outliers

In [25]:
```
gapminder_1952 <- gapminder %>%
  filter(year == 1952)

# Change to put pop on the x-axis and gdpPercap on the y-axis
ggplot(gapminder_1952, aes(x = pop , y = gdpPercap)) +
  geom_point()
```

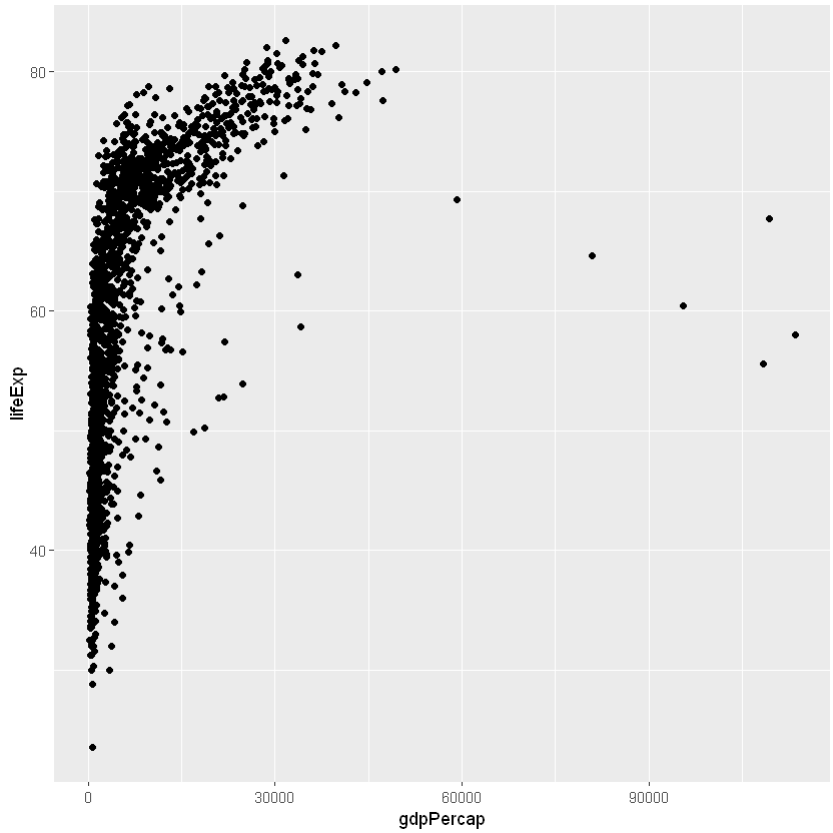


2.2 Logarithmic scales

Let plot comparing the GDP per capita of each country to its life expectancy.

In [20]:
```
library(ggplot2)
library(gapminder)
library(dplyr)


ggplot(gapminder, aes(x = gdpPercap , y = lifeExp)) +
  geom_point()
```



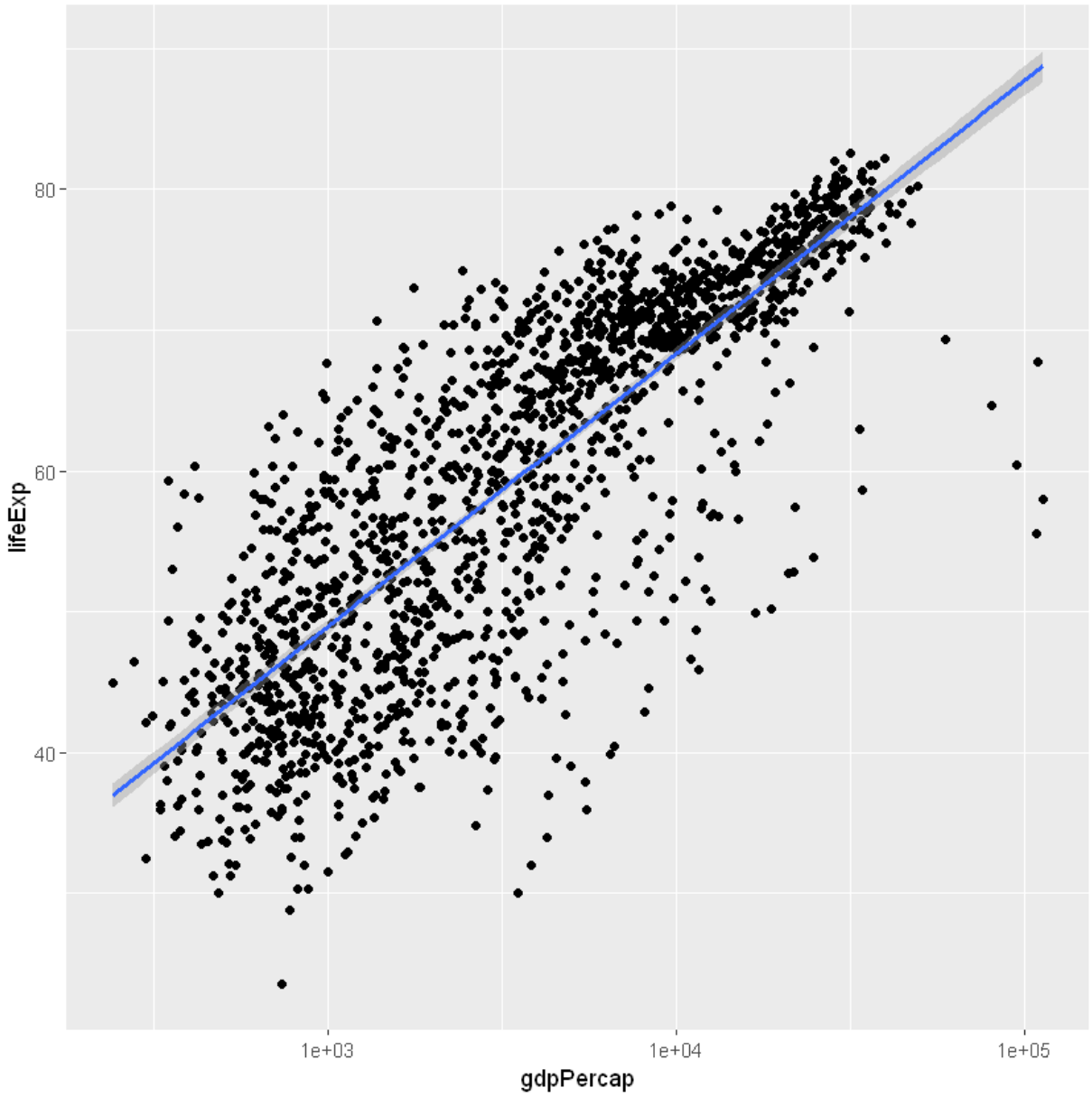
This plot communicates some interesting information: we can see that higher income countries tend to have higher life expectancy. One problem with this plot, however, is that a lot of countries get crammed into the leftmost part of the x-axis. This is because the distribution of GDP per capita spans several orders of magnitude, with some countries in the tens of thousands of dollars and others in the hundreds. When one of your axes has that kind of distribution, it's useful to work with a **logarithmic scale**

https://www.youtube.com/watch?v=W_BZb_va6jY&list=WL&index=2 (https://www.youtube.com/watch?v=W_BZb_va6jY&list=WL&index=2)

In [19]:
```
ggplot(gapminder, aes(x = gdpPercap , y = lifeExp)) +
  geom_point() +
scale_x_log10()+
geom_smooth(method="lm")

gdpPercap.lm = lm(gdpPercap ~ lifeExp, data=gapminder)
summary(gdpPercap.lm)$r.squared
```

0.340712951110232

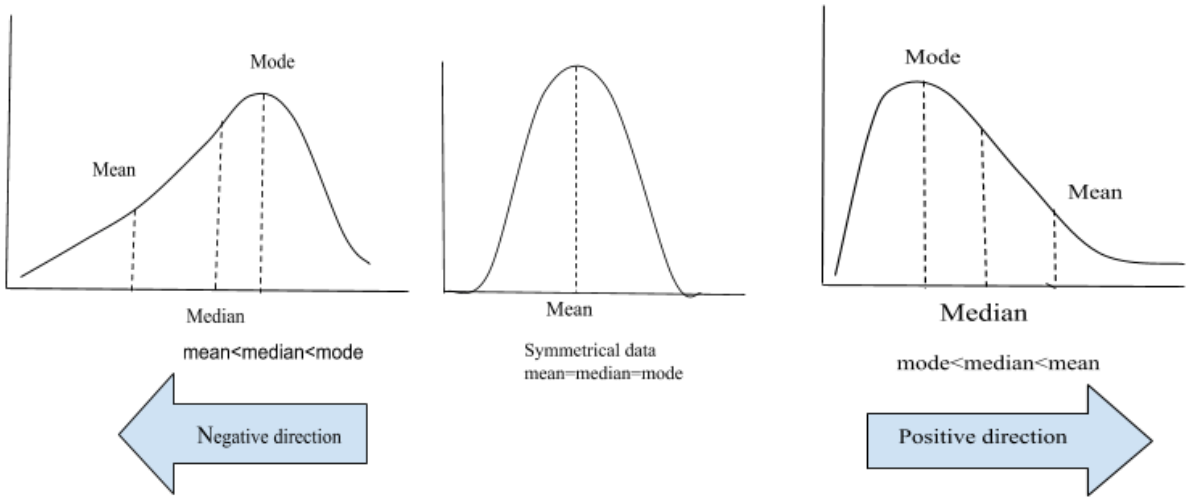


On this scale, the relationship between GDP per capita and life expectancy looks more linear: and you can more easily distinguish the countries at the lower end of the spectrum.

2.2.1 When should use Logarithmic scales?

There are one main reason to use logarithmic scales in charts and graphs:

- To respond to skewness towards large values; i.e., cases in which one or a few points are much larger than the bulk of the data.



It is extremely useful when graphing a large variance in data!.

	Logarithm	=	Exponent
	$\log_a N = x$	$\Longleftrightarrow$	$N = a^x$
(Common Log)	$\log N = x$	$\Longleftrightarrow$	$N = 10^x$
(Natural Log)	$\ln N = x$	$\Longleftrightarrow$	$N = e^x$

To refresh your memory of school math, logs are just another way of writing exponential equations, one that allows you to separate the exponent on one side of the equation.

The equation:

$$x = log_a(N)$$

means that **x** is the power or exponent that **a** is raised to in order to get **N**.

The common base for logarithmic scales is the base 10. However, other bases are also useful. While a base of ten is useful when the data range over several orders of magnitude.

2.3 Additional aesthetics in ggplot2

Previously, we plot to compare two variables within your data using two visual aesthetics: GDP per capita on the x-axis, and life expectancy on the y-axis.

But these aren't the only variables in the gapminder dataset: for example, you also have continent and population. You may want to examine relationships among all these variables in the same plot.

You already used the x-axis to represent GDP per capita and the y-axis to represent life expectancy. Now you'll learn to add two more aesthetics- color and size- to communicate even more information in your scatter plot.

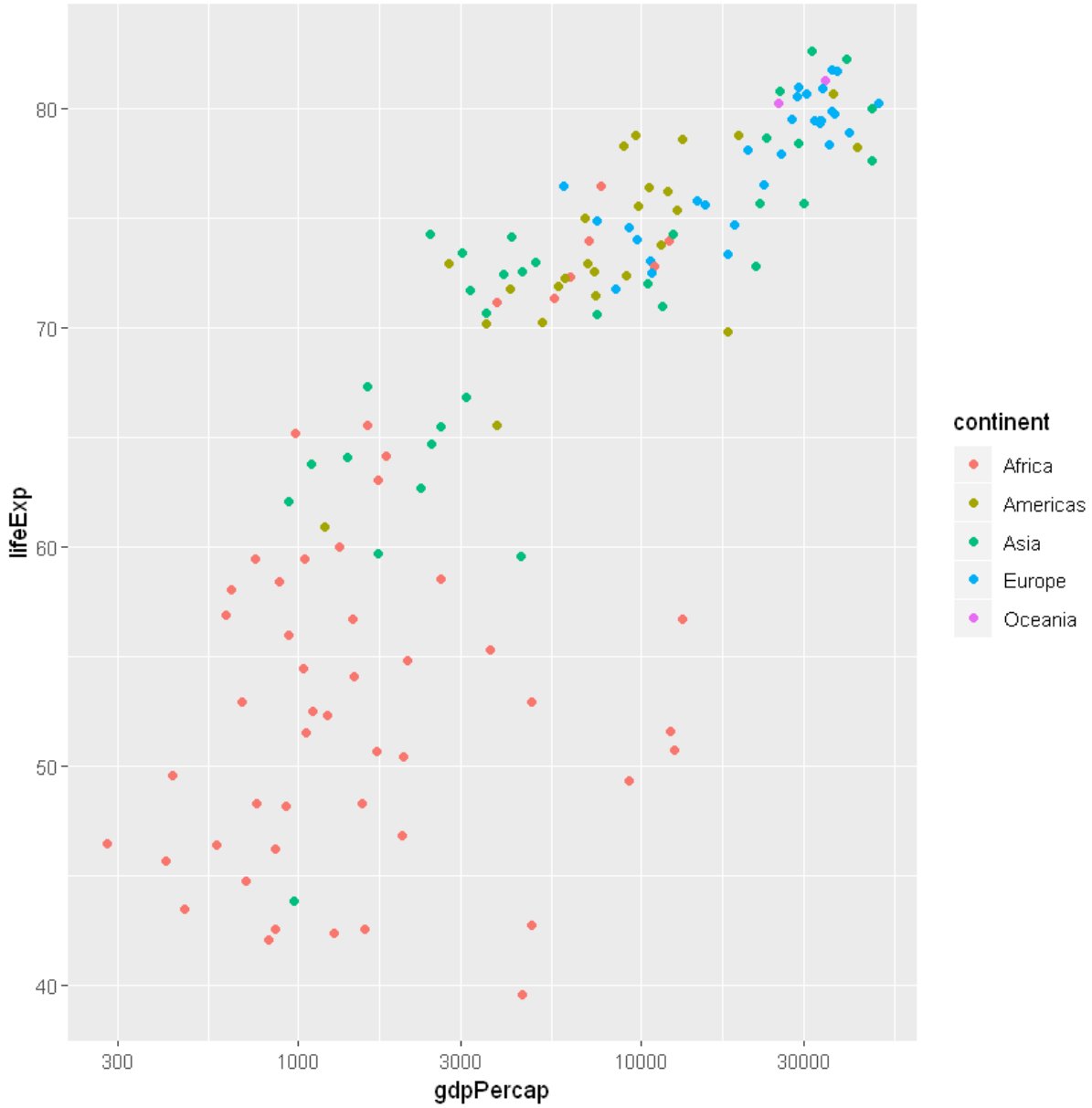
In [10]:
```
summary(gapminder)
```

```
       country        continent        year         lifeExp     
 Afghanistan:  12   Africa  :624   Min.   :1952   Min.   :23.60  
 Albania    :  12   Americas:300   1st Qu.:1966   1st Qu.:48.20  
 Algeria    :  12   Asia    :396   Median :1980   Median :60.71  
 Angola     :  12   Europe  :360   Mean   :1980   Mean   :59.47  
 Argentina  :  12   Oceania : 24   3rd Qu.:1993   3rd Qu.:70.85  
 Australia  :  12                  Max.   :2007   Max.   :82.60  
 (Other)    :1632                                               
      pop              gdpPercap       
 Min.   :6.001e+04   Min.   :   241.2  
 1st Qu.:2.794e+06   1st Qu.:  1202.1  
 Median :7.024e+06   Median :  3531.8  
 Mean   :2.960e+07   Mean   :  7215.3  
 3rd Qu.:1.959e+07   3rd Qu.:  9325.5  
 Max.   :1.319e+09   Max.   :113523.1  
```

Continent is a categorical variable: it has a few specific values, such as Asia and Europe. A good way to represent a categorical variable in a scatterplot is the color of your points.

In [20]:
```
gapminder_2007 <- gapminder %>%
  filter(year == 2007)

# Change to put pop on the x-axis and gdpPercap on the y-axis
ggplot(gapminder_2007, aes(x = gdpPercap , y = lifeExp, color = continent)) +
  geom_point() +
scale_x_log10()
```

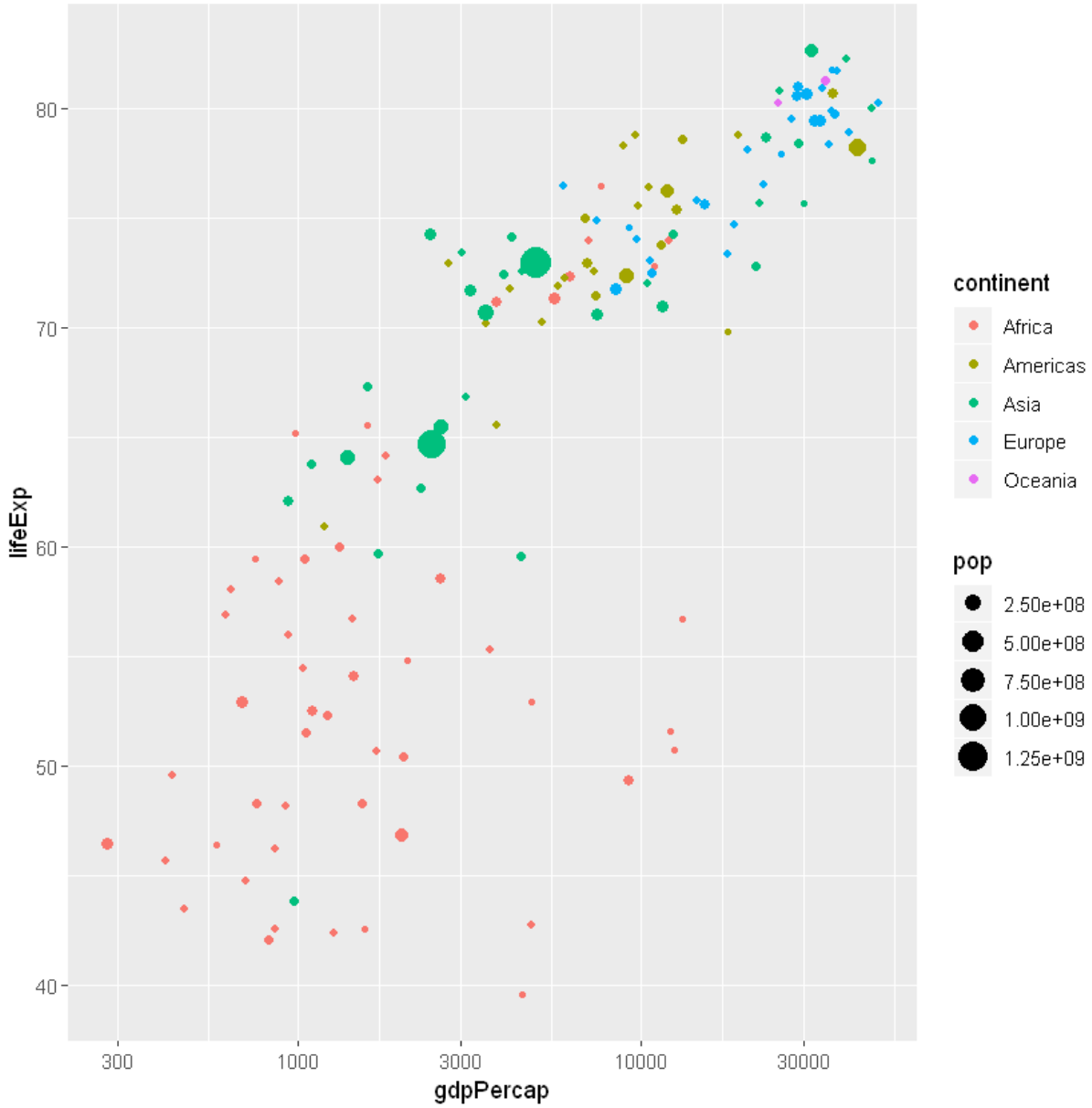


This communicates a lot about differences between continents. The average life expectancy and GDP per capita tends to be lowest for African countries, shown in red, and highest for European countries, shown in blue.

Another variable you may want to include in the graph is population, represented by the pop variable in the dataset. This is a numeric variable, so a good way to represent it is with the size of the points in the scatterplot.

In [21]:
```
gapminder_2007 <- gapminder %>%
  filter(year == 2007)

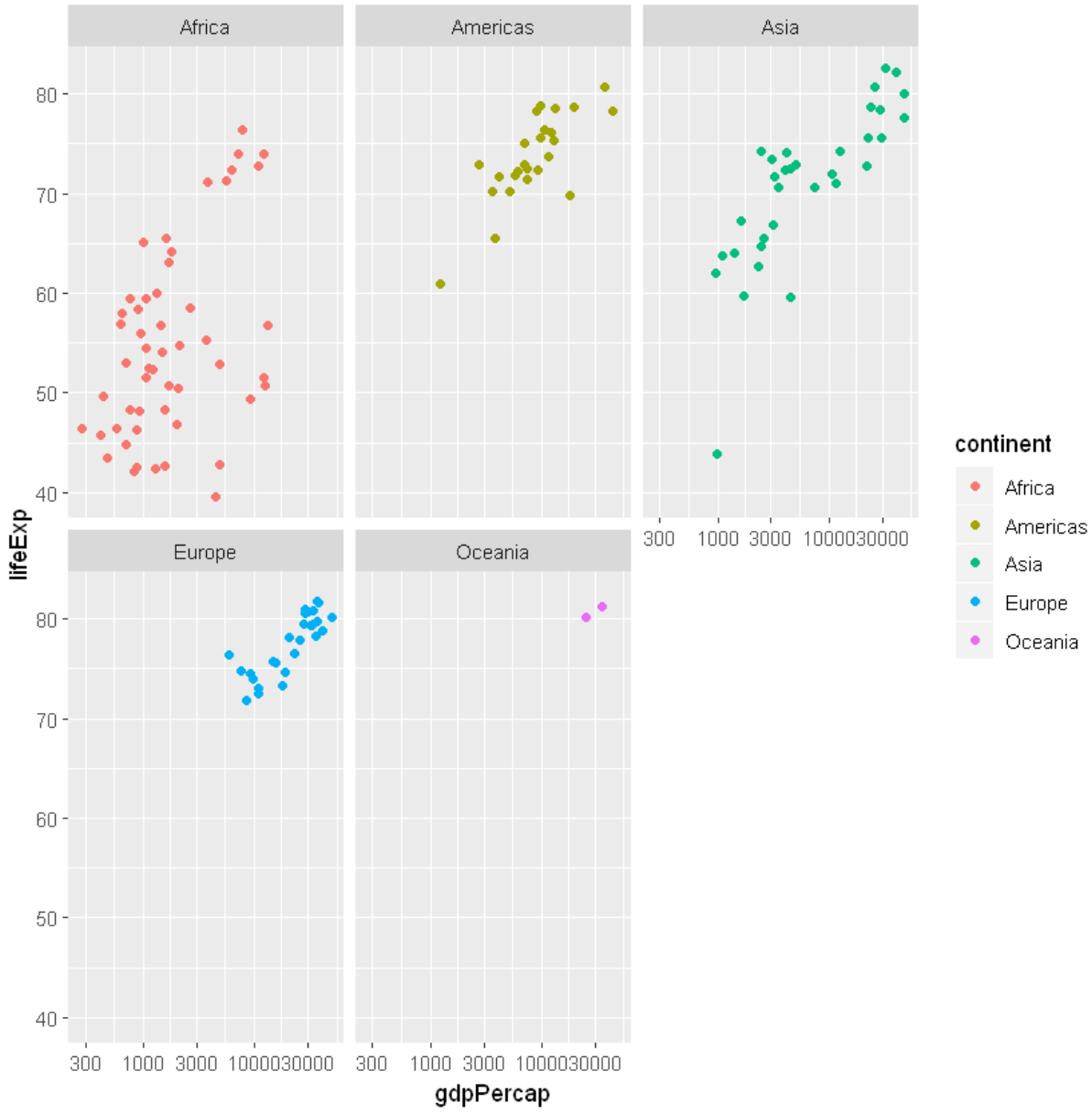
# Change to put pop on the x-axis and gdpPercap on the y-axis
ggplot(gapminder_2007, aes(x = gdpPercap , y = lifeExp, color = continent, size =
  geom_point() +
scale_x_log10()
```



ggplot2 lets you divide your plot into subplots to get one smaller graph for each continent: Africa, the Americas, and so on. This is called faceting, and it's another powerful way to communicate relationships within your data.

In [33]:
```
gapminder_2007 <- gapminder %>%
  filter(year == 2007)

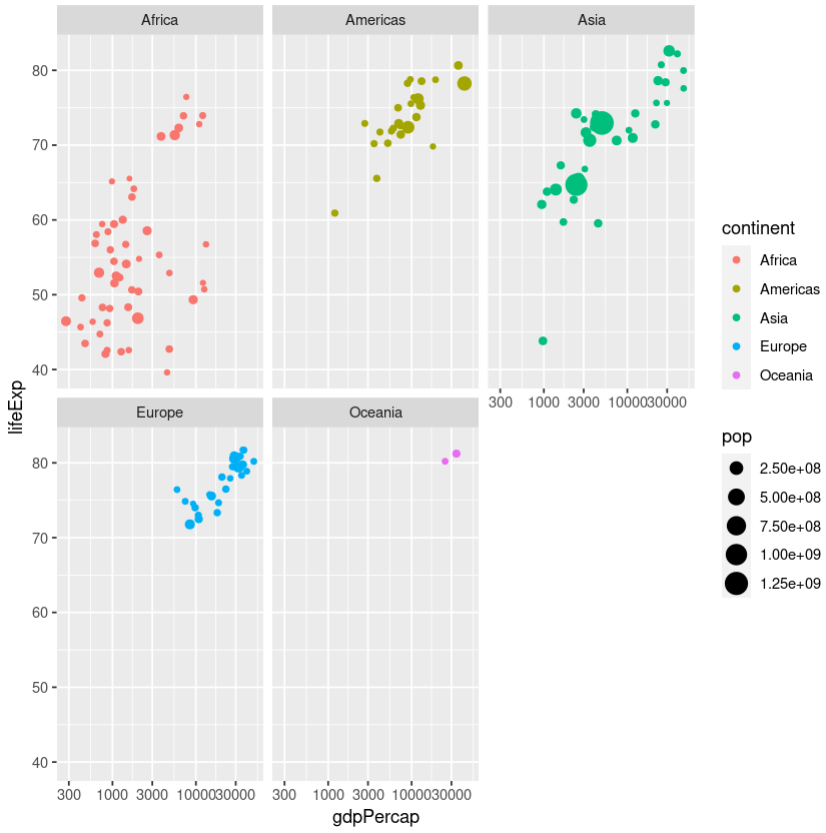
# Change to put pop on the x-axis and gdpPercap on the y-axis
ggplot(gapminder_2007, aes(x = gdpPercap , y = lifeExp, color=continent)) +
  geom_point() +
scale_x_log10() +
facet_wrap(~ continent)
```



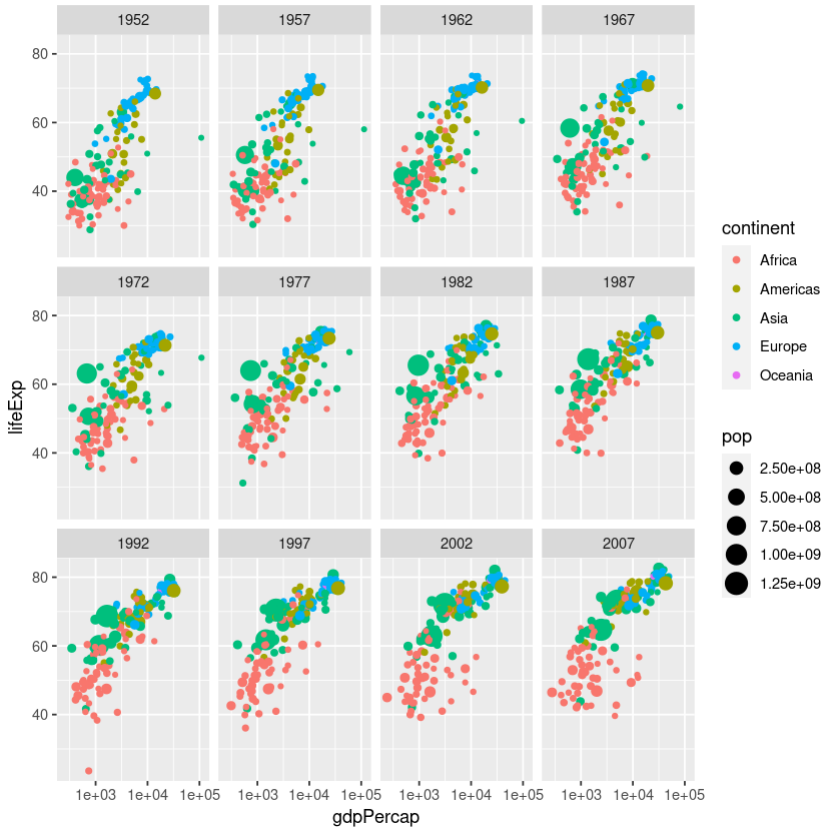
In R, the tilde (~) symbol typically means "by"

In [18]:
```
gapminder_2007 <- gapminder %>%
  filter(year == 2007)

# Change to put pop on the x-axis and gdpPercap on the y-axis
ggplot(gapminder_2007, aes(x = gdpPercap , y = lifeExp, color = continent, size =
  geom_point() +
scale_x_log10() +
facet_wrap(~ continent)
```



In [19]:
```
ggplot(gapminder, aes(x = gdpPercap , y = lifeExp, color = continent, size = pop)
  geom_point() +
scale_x_log10() +
facet_wrap(~ year)
```



3 Continue with Tidyverse (dplyr)

3.1 Summarize()

Analyses will usually involve a cycle between these steps of data transformation and visualization, as well as additional components of the data science workflow.

Now you'll learn how to **summarize** many observations into a single data point.

For example, suppose you want to know the average life expectancy across all countries and years in the dataset. You would do this with the **summarize** function.

In [23]:
```
gapminder
```

A tibble: 1704 × 6

country	continent	year	lifeExp	pop	gdpPercap
<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
Afghanistan	Asia	1952	28.801	8425333	779.4453
Afghanistan	Asia	1957	30.332	9240934	820.8530
Afghanistan	Asia	1962	31.997	10267083	853.1007
Afghanistan	Asia	1967	34.020	11537966	836.1971
Afghanistan	Asia	1972	36.088	13079460	739.9811
Afghanistan	Asia	1977	38.438	14880372	786.1134
Afghanistan	Asia	1982	39.854	12881816	978.0114
Afghanistan	Asia	1987	40.822	13867957	852.3959
Afghanistan	Asia	1992	41.674	16317921	649.3414
Afghanistan	Asia	1997	41.763	22227415	635.3414
Afghanistan	Asia	2002	42.129	25268405	726.7341
Afghanistan	Asia	2007	43.828	31889923	974.5803
Albania	Europe	1952	55.230	1282697	1601.0561
Albania	Europe	1957	59.280	1476505	1942.2842
Albania	Europe	1962	64.820	1728137	2312.8890
Albania	Europe	1967	66.220	1984060	2760.1969
Albania	Europe	1972	67.690	2263554	3313.4222
Albania	Europe	1977	68.930	2509048	3533.0039
Albania	Europe	1982	70.420	2780097	3630.8807
Albania	Europe	1987	72.000	3075321	3738.9327
Albania	Europe	1992	71.581	3326498	2497.4379
Albania	Europe	1997	72.950	3428038	3193.0546
Albania	Europe	2002	75.651	3508512	4604.2117
Albania	Europe	2007	76.423	3600523	5937.0295
Algeria	Africa	1952	43.077	9279525	2449.0082
Algeria	Africa	1957	45.685	10270856	3013.9760
Algeria	Africa	1962	48.303	11000948	2550.8169
Algeria	Africa	1967	51.407	12760499	3246.9918
Algeria	Africa	1972	54.518	14760787	4182.6638
Algeria	Africa	1977	58.014	17152804	4910.4168
⋮	⋮	⋮	⋮	⋮	⋮
Yemen, Rep.	Asia	1982	49.113	9657618	1977.5570

country	continent	year	lifeExp	pop	gdpPercap
<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
Yemen, Rep.	Asia	1987	52.922	11219340	1971.7415
Yemen, Rep.	Asia	1992	55.599	13367997	1879.4967
Yemen, Rep.	Asia	1997	58.020	15826497	2117.4845
Yemen, Rep.	Asia	2002	60.308	18701257	2234.8208
Yemen, Rep.	Asia	2007	62.698	22211743	2280.7699
Zambia	Africa	1952	42.038	2672000	1147.3888
Zambia	Africa	1957	44.077	3016000	1311.9568
Zambia	Africa	1962	46.023	3421000	1452.7258
Zambia	Africa	1967	47.768	3900000	1777.0773
Zambia	Africa	1972	50.107	4506497	1773.4983
Zambia	Africa	1977	51.386	5216550	1588.6883
Zambia	Africa	1982	51.821	6100407	1408.6786
Zambia	Africa	1987	50.821	7272406	1213.3151
Zambia	Africa	1992	46.100	8381163	1210.8846
Zambia	Africa	1997	40.238	9417789	1071.3538
Zambia	Africa	2002	39.193	10595811	1071.6139
Zambia	Africa	2007	42.384	11746035	1271.2116
Zimbabwe	Africa	1952	48.451	3080907	406.8841
Zimbabwe	Africa	1957	50.469	3646340	518.7643
Zimbabwe	Africa	1962	52.358	4277736	527.2722
Zimbabwe	Africa	1967	53.995	4995432	569.7951
Zimbabwe	Africa	1972	55.635	5861135	799.3622
Zimbabwe	Africa	1977	57.674	6642107	685.5877
Zimbabwe	Africa	1982	60.363	7636524	788.8550
Zimbabwe	Africa	1987	62.351	9216418	706.1573
Zimbabwe	Africa	1992	60.377	10704340	693.4208
Zimbabwe	Africa	1997	46.809	11404948	792.4500
Zimbabwe	Africa	2002	39.989	11926563	672.0386
Zimbabwe	Africa	2007	43.487	12311143	469.7093

In [34]:
```
library(ggplot2)
library(gapminder)
library(dplyr)

gapminder %>%
summarize(meanLifeExp = mean(lifeExp))
```

meanLifeExp
59.47444

In the output, we see the answer to our question: the mean life expectancy was about 59.47 years.

If you think about it, it doesn't really make sense to summarize across all countries and across all years. It may make more sense to ask questions about averages in a particular year, such as 2007.

To answer this, you can combine the summarize verb with filter: filter your data for a particular year first, then summarize the result.

In [35]:
```
gapminder %>%
filter(year == 2007) %>%
summarise(meanLifeExp = mean(lifeExp))
```

meanLifeExp
67.00742

This shows you that the average life expectancy in the year 2007 was about 67 years.

You can create multiple summaries at once with the summarize. For example, suppose that along with finding the average life expectancy in 2007, you want to find the total population in that year.

In [41]:
```
gapminder %>%
filter(year == 2007) %>%
summarise(meanLifeExp = mean(lifeExp),
          totalpop = sum(as.numeric(pop))
          )
```

meanLifeExp	totalpop
67.00742	6251013179

In [18]:
```
?summarise
```

Useful functions

- Center: mean(), median()
- Spread: sd(), IQR(), mad()
- Range: min(), max(), quantile()

- Position: first(), last(), nth(),
- Count: n(), n_distinct()
- Logical: any(), all()

3.2 Group_by()

We use the **summarize** function to answer questions about the entire dataset, or about a particular year. For example, here you're finding the average life expectancy and the total population in the year 2007.

What if we weren't interested just in the average for the year 2007, but for each of the years in the dataset?

You could rerun this code and change the year each time, but that's very tedious...or

we can use the **group_by** function

In [42]:
```
gapminder %>%
group_by(year) %>%
summarise(meanLifeExp = mean(lifeExp),
          totalpop = sum(as.numeric(pop)))
```

year	meanLifeExp	totalpop
1952	49.05762	2406957150
1957	51.50740	2664404580
1962	53.60925	2899782974
1967	55.67829	3217478384
1972	57.64739	3576977158
1977	59.57016	3930045807
1982	61.53320	4289436840
1987	63.21261	4691477418
1992	64.16034	5110710260
1997	65.01468	5515204472
2002	65.69492	5886977579
2007	67.00742	6251013179

This shows us that the total population started at 2.4 billion, and went up to 6.25 billion in 2007.

We can also see that average life expectancy went up from 49 years in 1952 to 67 in 2007.

How to determinate the average life expectancy and the total population in 2007 within each continent.

In [43]:
```
gapminder %>%
filter(year == 2007) %>%
group_by(continent) %>%
summarise(meanLifeExp = mean(lifeExp),
          totalpop = sum(as.numeric(pop)))
```

continent	meanLifeExp	totalpop
Africa	54.80604	929539692
Americas	73.60812	898871184
Asia	70.72848	3811953827
Europe	77.64860	586098529
Oceania	80.71950	24549947

We can see that Europe and Oceania have the highest life expectancy, and that Asia and Africa are lower.

Now that you've calculated these statistics for each continent in 2007, you might be interested in how they changed for each continent over time.

In [44]:
```
gapminder %>%
group_by(continent, year) %>%
summarise(meanLifeExp = mean(lifeExp),
          totalpop = sum(as.numeric(pop)))
```

continent	year	meanLifeExp	totalpop
Africa	1952	39.13550	237640501
Africa	1957	41.26635	264837738
Africa	1962	43.31944	296516865
Africa	1967	45.33454	335289489
Africa	1972	47.45094	379879541
Africa	1977	49.58042	433061021
Africa	1982	51.59287	499348587
Africa	1987	53.34479	574834110
Africa	1992	53.62958	659081517
Africa	1997	53.59827	743832984
Africa	2002	53.32523	833723916
Africa	2007	54.80604	929539692
Americas	1952	53.27984	345152446
Americas	1957	55.96028	386953916
Americas	1962	58.39876	433270254
Americas	1967	60.41092	480746623
Americas	1972	62.39492	529384210
Americas	1977	64.39156	578067699
Americas	1982	66.22884	630290920
Americas	1987	68.09072	682753971
Americas	1992	69.56836	739274104
Americas	1997	71.15048	796900410
Americas	2002	72.42204	849772762
Americas	2007	73.60812	898871184
Asia	1952	46.31439	1395357351
Asia	1957	49.31854	1562780599
Asia	1962	51.56322	1696357182
Asia	1967	54.66364	1905662900
Asia	1972	57.31927	2150972248
Asia	1977	59.61056	2384513556
Asia	1982	62.61794	2610135582
Asia	1987	64.85118	2871220762

continent	year	meanLifeExp	totalpop
Asia	1992	66.53721	3133292191
Asia	1997	68.02052	3383285500
Asia	2002	69.23388	3601802203
Asia	2007	70.72848	3811953827
Europe	1952	64.40850	418120846
Europe	1957	66.70307	437890351
Europe	1962	68.53923	460355155
Europe	1967	69.73760	481178958
Europe	1972	70.77503	500635059
Europe	1977	71.93777	517164531
Europe	1982	72.80640	531266901
Europe	1987	73.64217	543094160
Europe	1992	74.44010	558142797
Europe	1997	75.50517	568944148
Europe	2002	76.70060	578223869
Europe	2007	77.64860	586098529
Oceania	1952	69.25500	10686006
Oceania	1957	70.29500	11941976
Oceania	1962	71.08500	13283518
Oceania	1967	71.31000	14600414
Oceania	1972	71.91000	16106100
Oceania	1977	72.85500	17239000
Oceania	1982	74.29000	18394850
Oceania	1987	75.32000	19574415
Oceania	1992	76.94500	20919651
Oceania	1997	78.19000	22241430
Oceania	2002	79.74000	23454829
Oceania	2007	80.71950	24549947

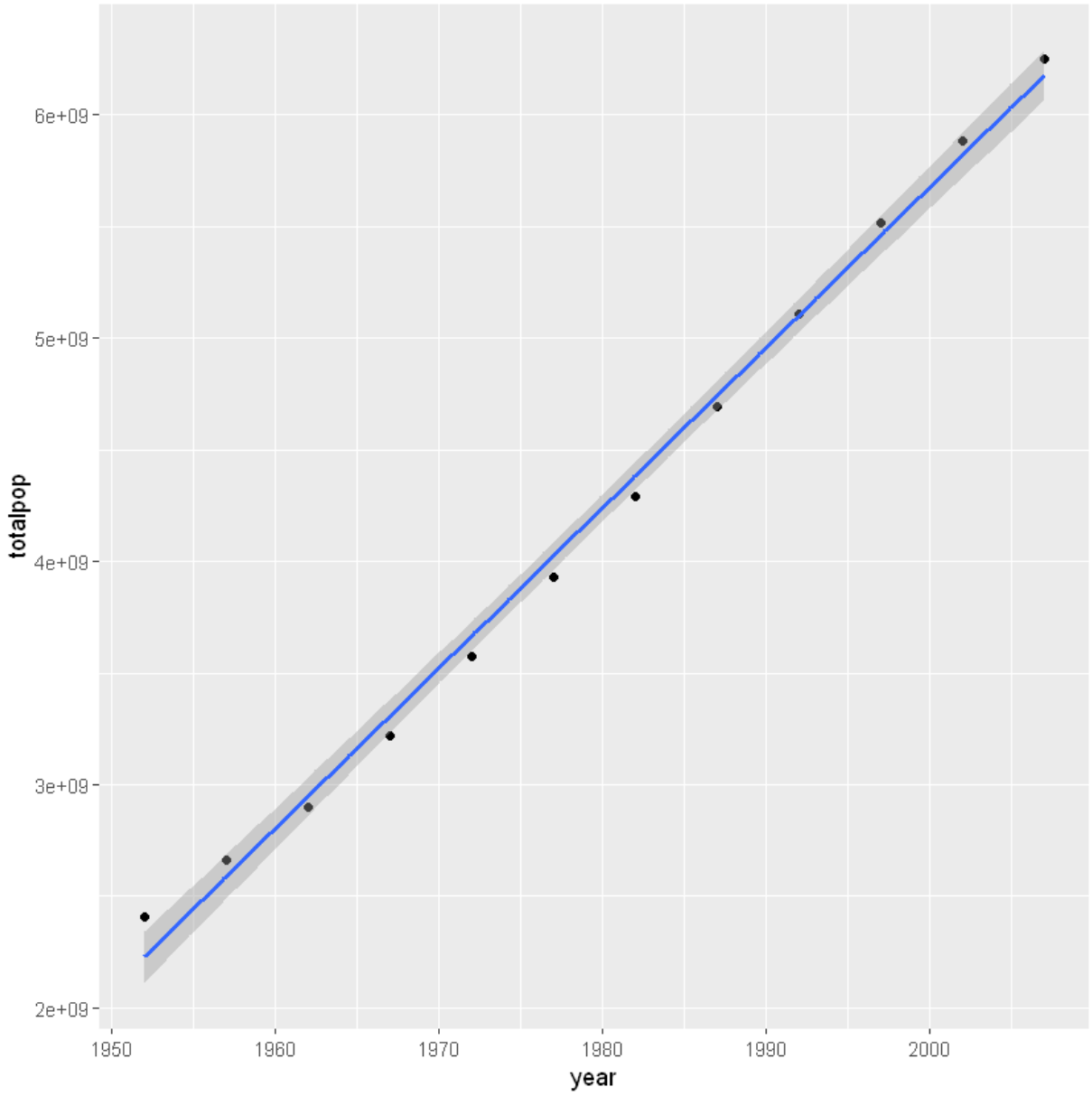
4 Visualizing summarized data

In [46]:
```
by_year <- gapminder %>%
group_by(year) %>%
summarise(meanLifeExp = mean(lifeExp),
          totalpop = sum(as.numeric(pop)))

by_year
```

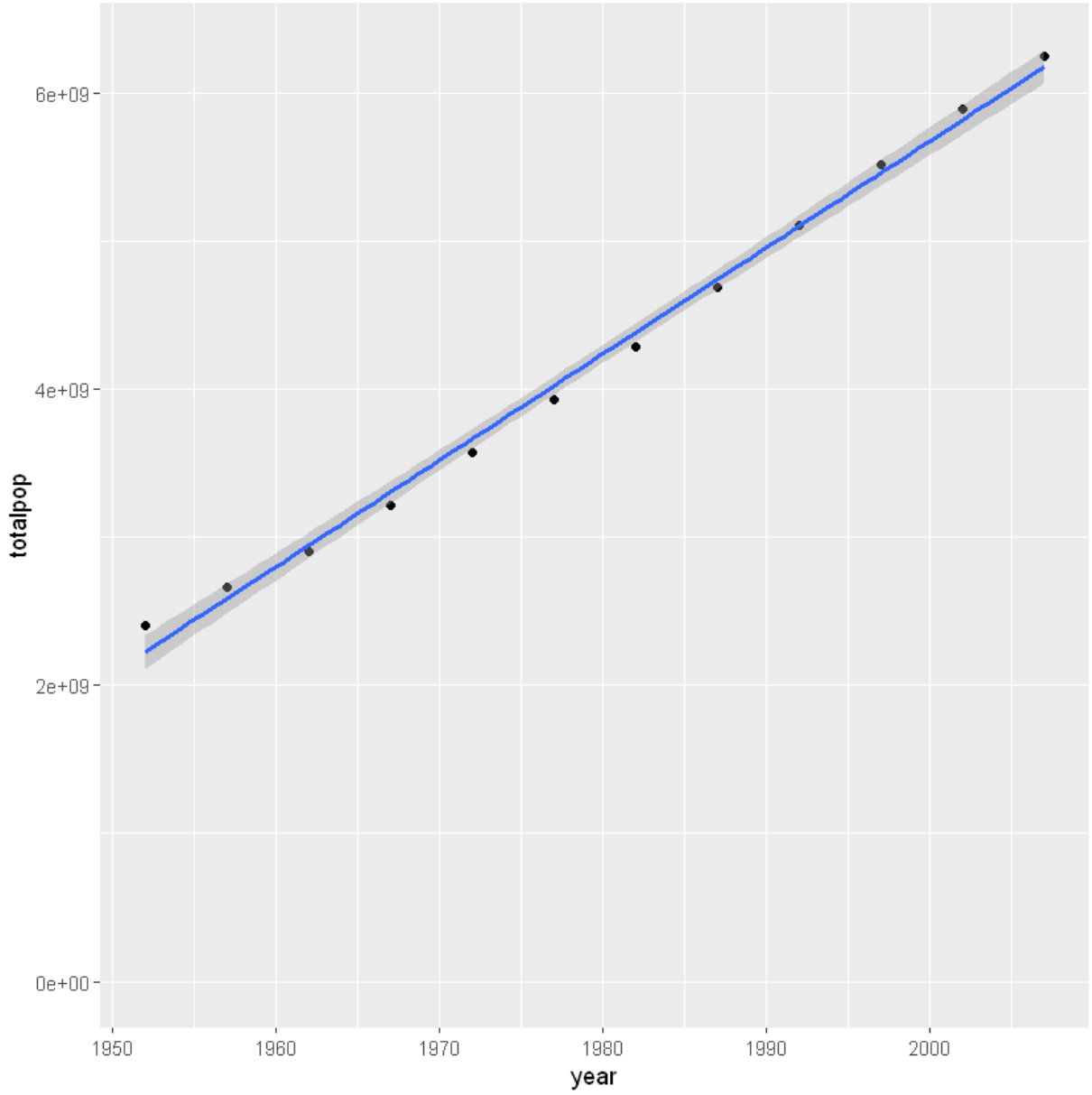
year	meanLifeExp	totalpop
1952	49.05762	2406957150
1957	51.50740	2664404580
1962	53.60925	2899782974
1967	55.67829	3217478384
1972	57.64739	3576977158
1977	59.57016	3930045807
1982	61.53320	4289436840
1987	63.21261	4691477418
1992	64.16034	5110710260
1997	65.01468	5515204472
2002	65.69492	5886977579
2007	67.00742	6251013179

In [47]:
```
ggplot(by_year, aes(x = year, y = totalpop)) +
geom_point()+
geom_smooth(method="lm")
```



The resulting graph of population by year shows the change in the total population, which is going up over time. ggplot2 puts the y-axis is in scientific notation, since showing it with nine zeros would be hard to read.

In [48]:
```
ggplot(by_year, aes(x = year, y = totalpop)) +
geom_point() +
expand_limits(y = 0) +
geom_smooth(method="lm")
```

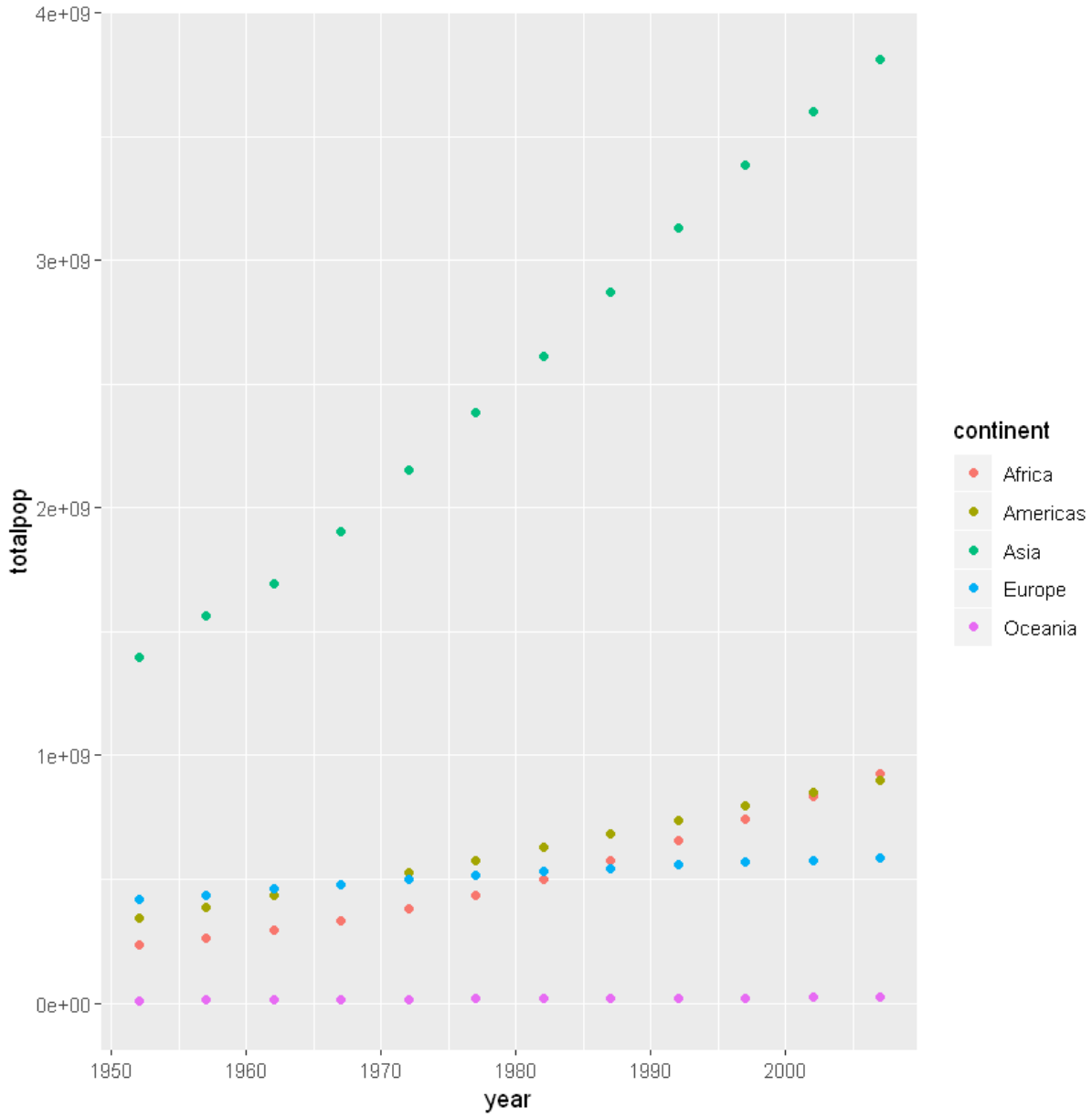


In [50]:
```
library(ggplot2)
library(gapminder)
library(dplyr)


by_year_continent <- gapminder %>%
group_by(year, continent) %>%
summarise(meanLifeExp = mean(lifeExp),
          totalpop = sum(as.numeric(pop)))

#by_year_continent

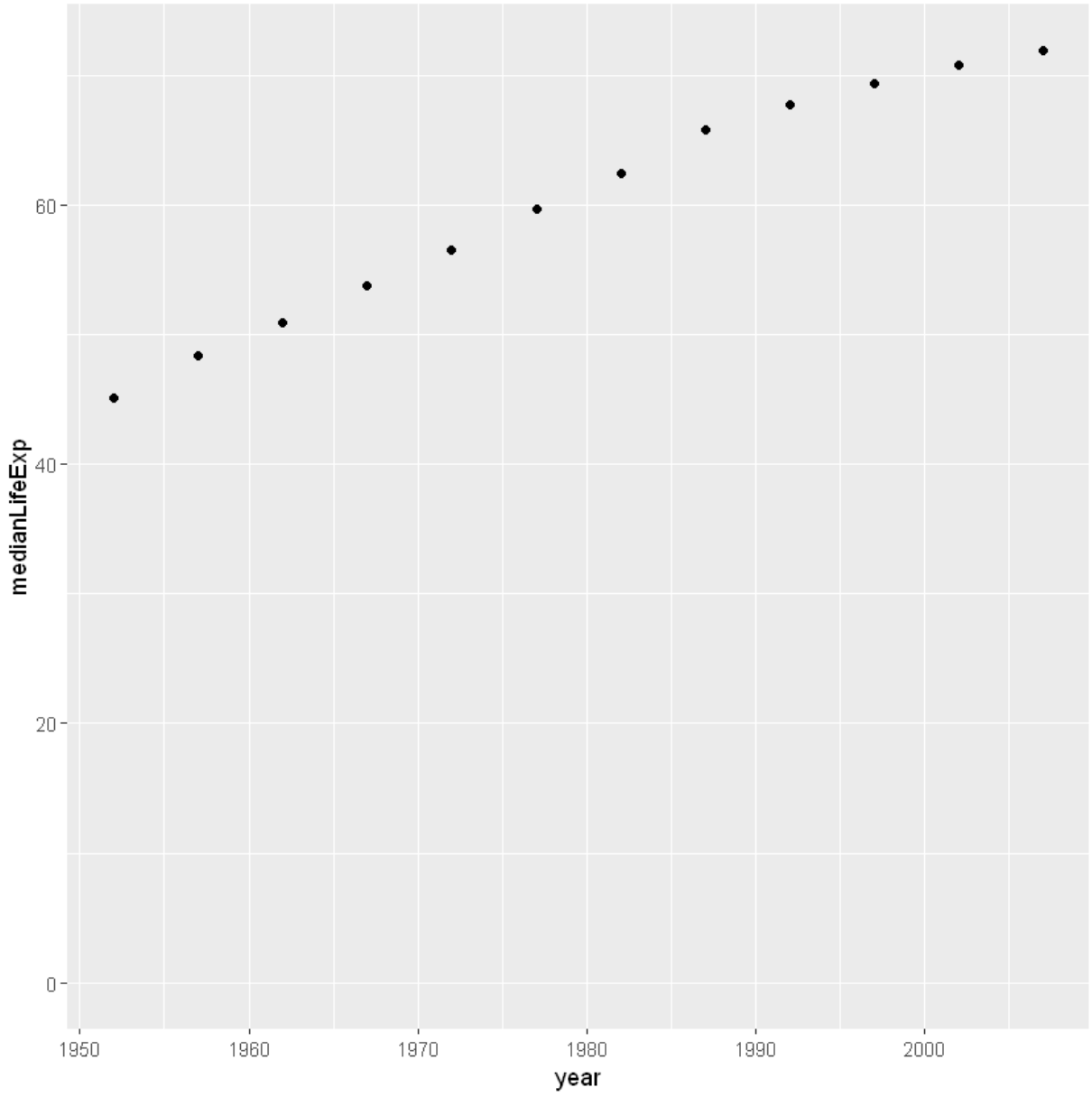
ggplot(by_year_continent, aes(x = year, y = totalpop, color = continent)) +
geom_point() +
expand_limits(y = 0)
```



This lets us see that Asia was always the most populated continent and has been growing the most rapidly, that Oceania has a slower rate of growth, and that Africa has grown to surpass both Europe and the Americas in terms of population.

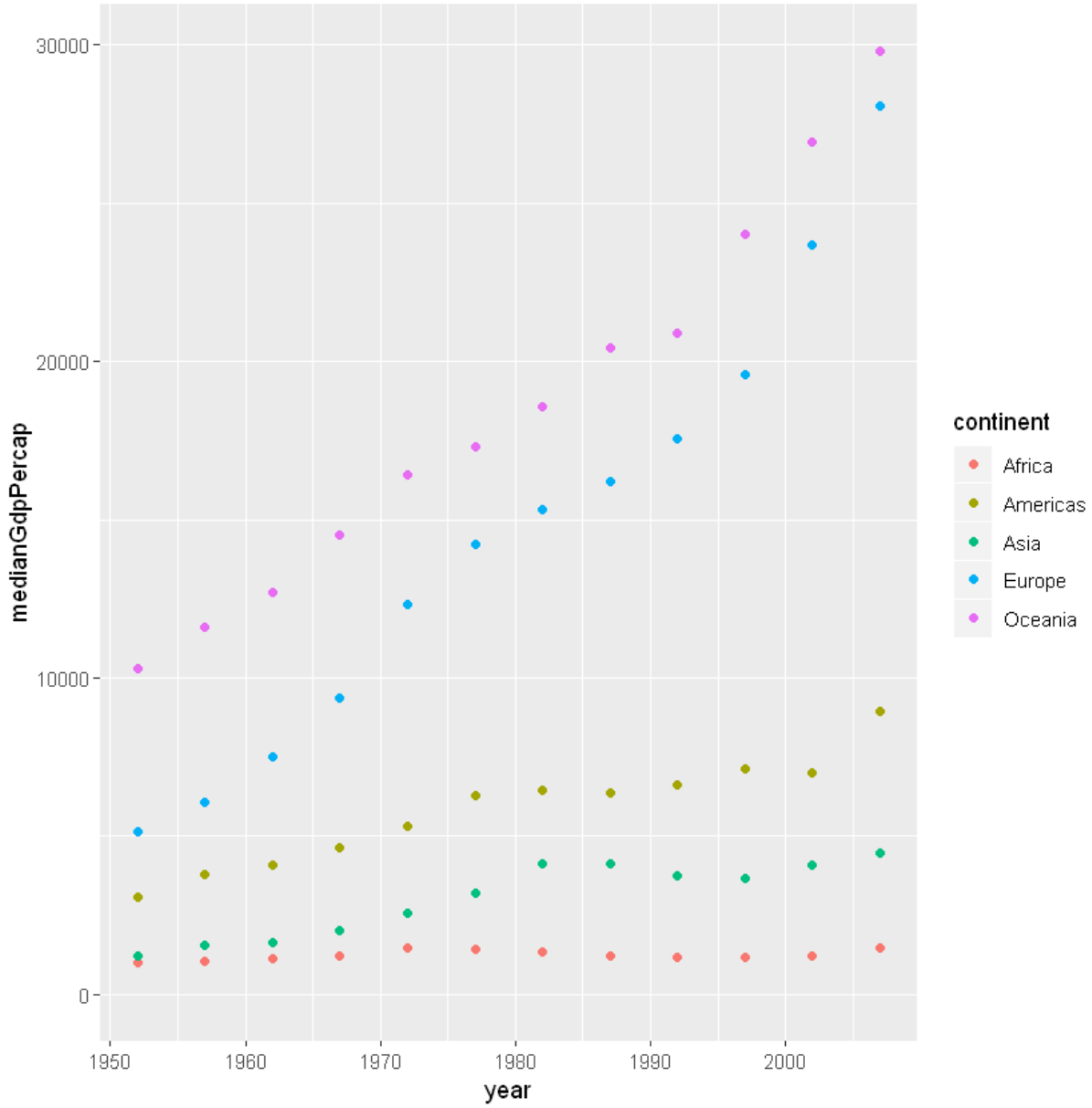
In [51]:
```
by_year <- gapminder %>%
  group_by(year) %>%
  summarize(medianLifeExp = median(lifeExp),
            maxGdpPercap = max(gdpPercap))

ggplot(by_year, aes(x = year, y = medianLifeExp)) +
geom_point() +
expand_limits(y = 0)
```



In [52]:
```
by_year_continent <- gapminder %>%
  group_by(continent, year) %>%
  summarize(medianGdpPercap = median(gdpPercap))

ggplot(by_year_continent, aes(x = year, y = medianGdpPercap, color = continent))
geom_point() +
expand_limits(y = 0)
```

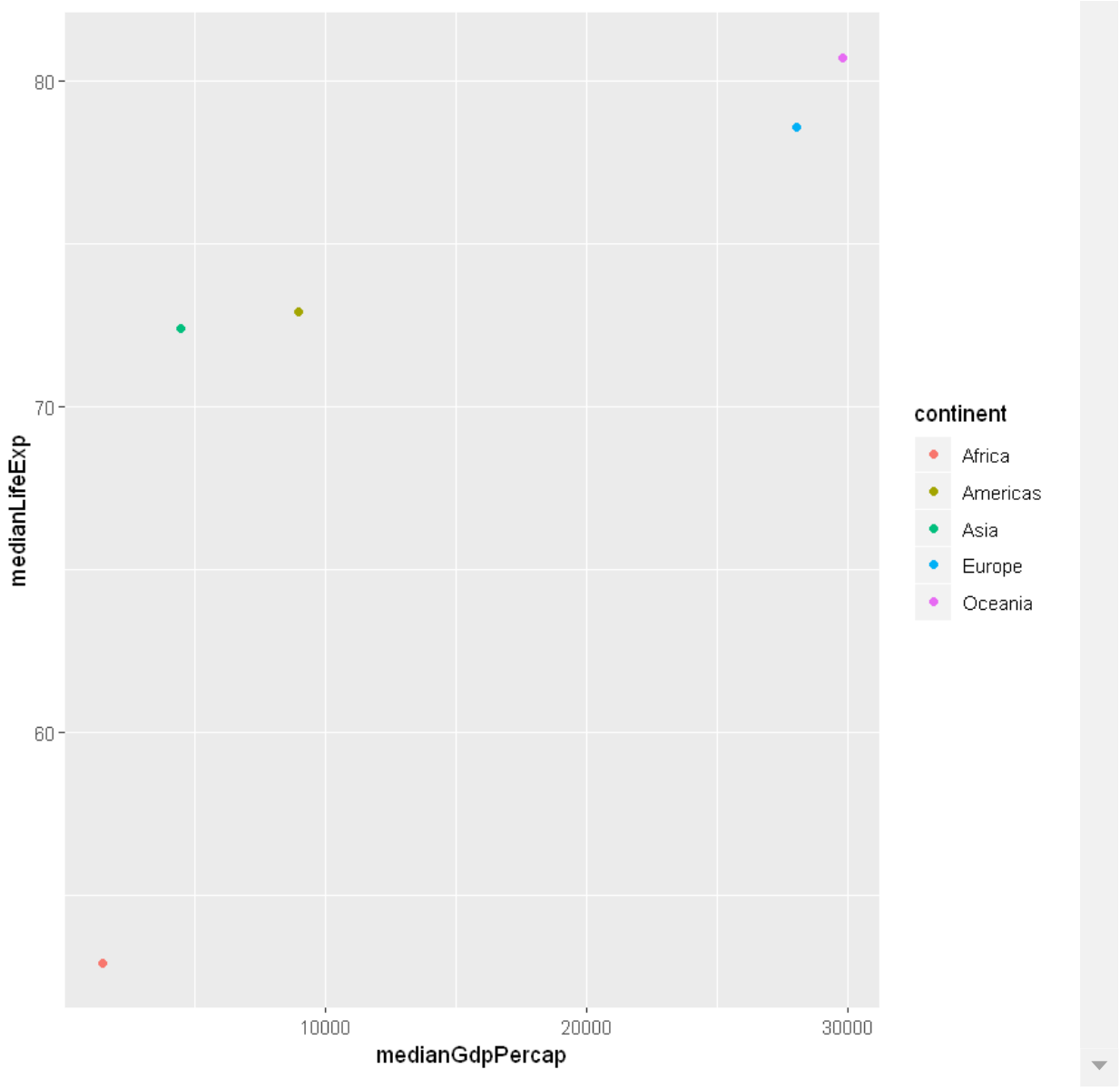


In [53]:
```
by_continent_2007 <- gapminder %>%
filter(year == 2007) %>%
  group_by(continent) %>%
  summarize( medianLifeExp = median(lifeExp),
      medianGdpPercap = median(gdpPercap))

by_continent_2007

ggplot(by_continent_2007, aes(x = medianGdpPercap, y = medianLifeExp, color = cor
geom_point()
```

continent	medianLifeExp	medianGdpPercap
Africa	52.9265	1452.267
Americas	72.8990	8948.103
Asia	72.3960	4471.062
Europe	78.6085	28054.066
Oceania	80.7195	29810.188



5 Types of visualizations

5.1 Line plots

Line plots, which are useful for showing change over time.

5.1.1 What is a line plot?

A line chart (aka line plot, line graph) uses points connected by line segments from left to right to demonstrate changes in value. The horizontal axis depicts a continuous progression, often that of time, while the vertical axis reports values for a metric of interest across that progression.

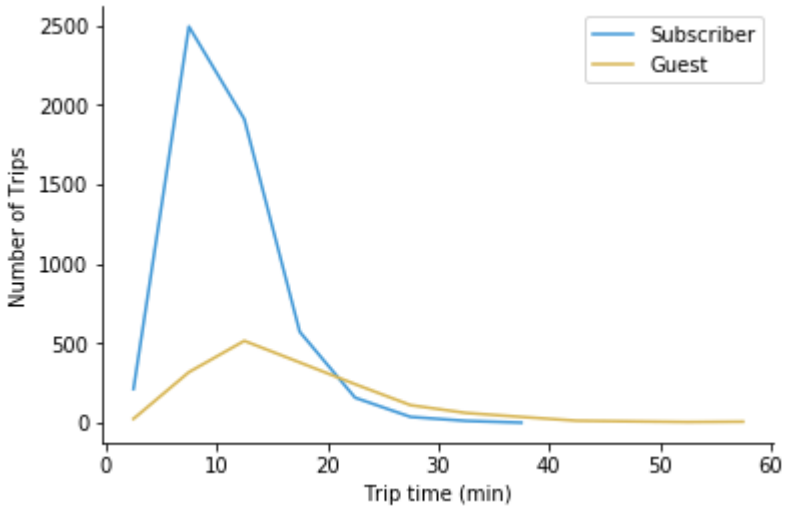
5.1.2 When you should use a line plot?

You will use a line chart when you want to emphasize changes in values for one variable (plotted on the vertical axis) for continuous values of a second variable (plotted on the horizontal). This emphasis on patterns of change is sold by line segments moving consistently from left to right and observing the slopes of the lines moving up or down.

On the horizontal axis, you need a variable that depicts continuous values that have a regular interval of measurement. Very commonly, this variable is a temporal one, generating an observation every minute, hour, day, week, or month. The choice of interval size, or bin, is a decision that the analyst will usually need to make for the data.

On the vertical axis, you will report the value of a second numeric variable for points that fall in each of the intervals defined by the horizontal-axis variable. Often, this will be a statistical summary like a total or average value across events within each bin.

Multiple lines can also be plotted in a single line chart to compare the trend between series. A common use case for this is to observe the breakdown of the data across different subgroups. The ability to plot multiple lines also provides the line chart a special use case where it might not usually be selected. Normally, we would use a histogram to depict the frequency distribution of a single numeric variable. However, since it's tricky to plot two histograms on the same set of axes, the line chart serves as a good mode of comparison as a substitute.



To use a line chart, data often needs to be aggregated into a table with two or more columns. Values in the first column indicate positions for points on the horizontal axis for each line to be plotted. Each following column indicates the vertical position for points of a single line.

In [13]:
```
library(gapminder)
library(dplyr)
#gapminder

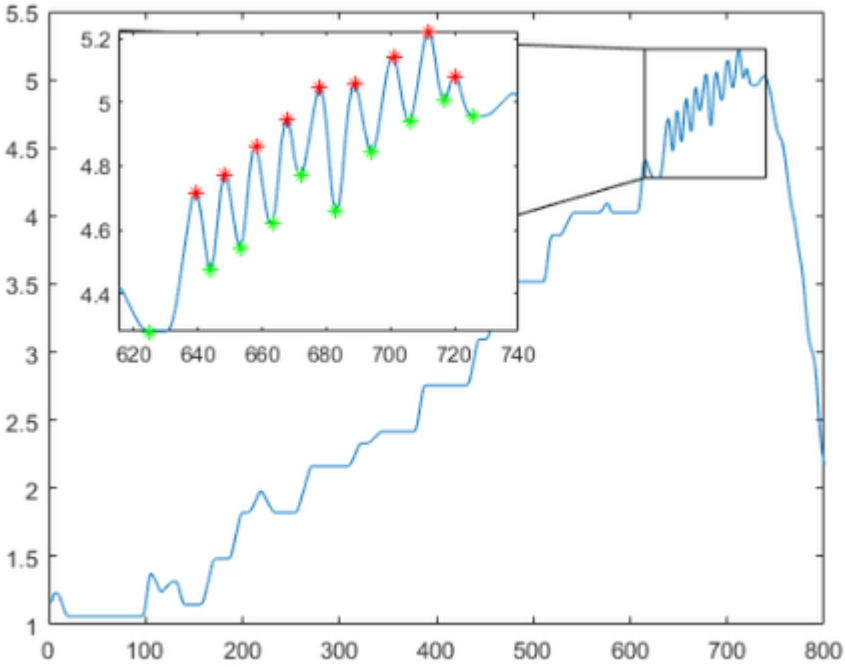

gapminder %>%
filter(country == "Chile")
```

A tibble: 12 × 6

country	continent	year	lifeExp	pop	gdpPercap
<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
Chile	Americas	1952	54.745	6377619	3939.979
Chile	Americas	1957	56.074	7048426	4315.623
Chile	Americas	1962	57.924	7961258	4519.094
Chile	Americas	1967	60.523	8858908	5106.654
Chile	Americas	1972	63.441	9717524	5494.024
Chile	Americas	1977	67.052	10599793	4756.764
Chile	Americas	1982	70.565	11487112	5095.666
Chile	Americas	1987	72.492	12463354	5547.064
Chile	Americas	1992	74.126	13572994	7596.126
Chile	Americas	1997	75.816	14599929	10118.053
Chile	Americas	2002	77.860	15497046	10778.784
Chile	Americas	2007	78.553	16284741	13171.639

5.1.3 Choose an appropriate measurement interval

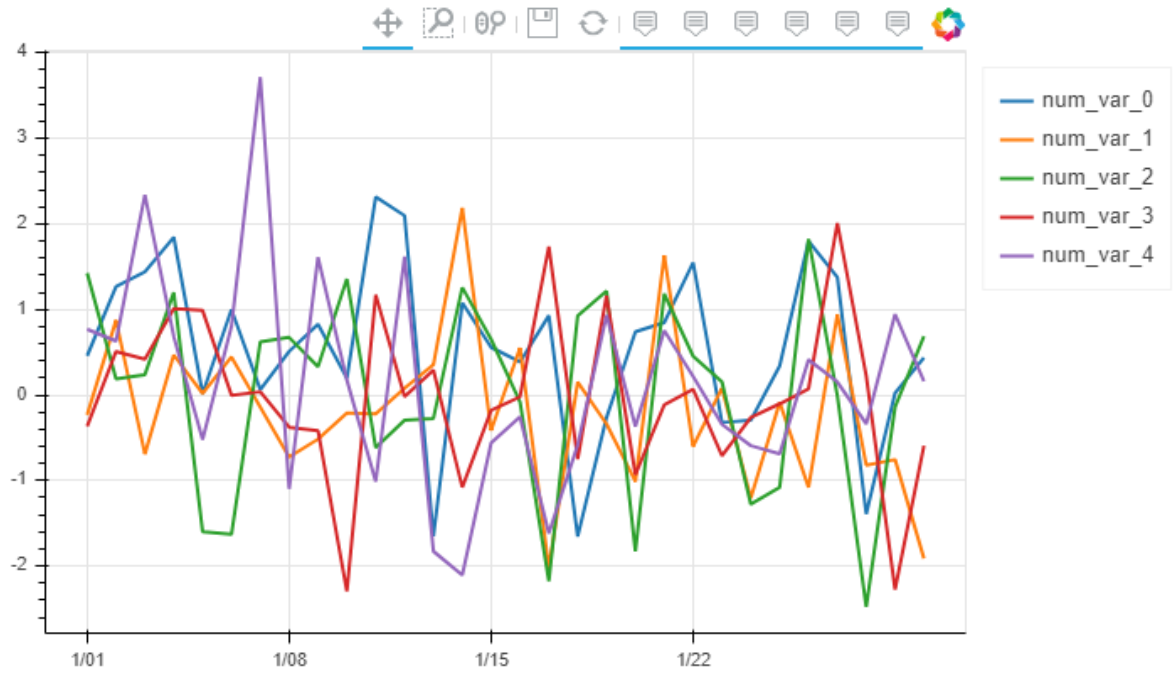
An important aspect of creating a line chart is selecting the right interval or bin size. For temporal data, a too-broad of a measurement interval may mean that it takes too long to see where the data trend is leading, hiding away the useful signal. On the flip side of the coin, a too-short a measurement interval may only reveal noise rather than signal.



5.1.4 Don’t plot too many lines

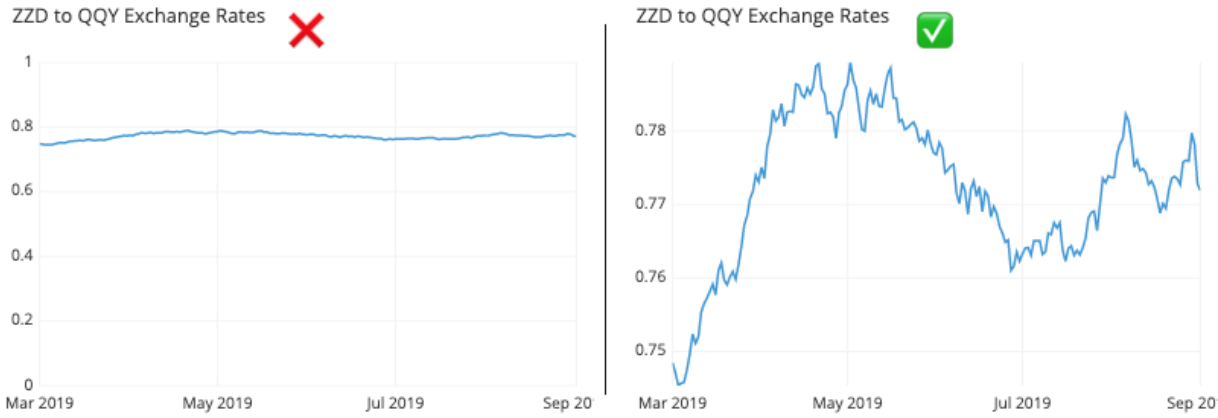
While there is the technical capacity to put many lines onto a single line chart, it is a good idea to be judicious in the amount of data that you plot. A good rule of thumb is to limit yourself to five or fewer lines.

If you find the need to plot more lines than can be read in a single axis, then you might consider faceting the plots into a grid of smaller line charts. It will be more difficult to see details in these plots, so it’s a good idea to sort them by some important characteristic (like average or final value) to help draw out important points. If you are using a tool that allows for interactive plots.



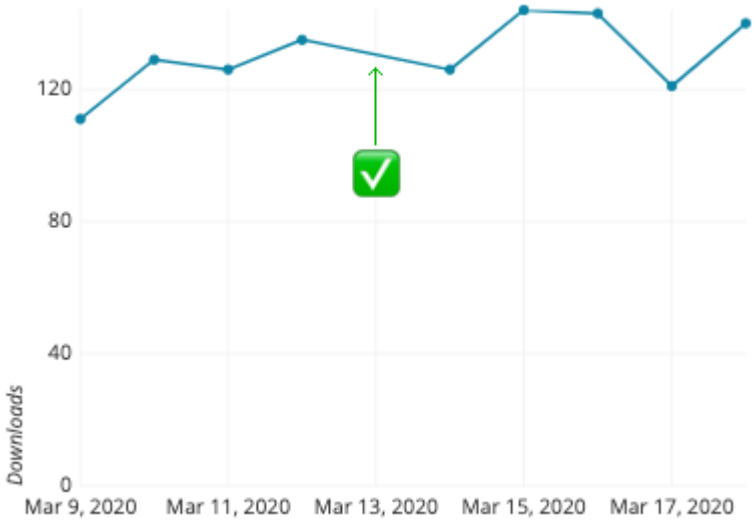
5.1.5 Strictly using a zero-value baseline

Despite the zero baseline for the vertical axis being a requirement for bar charts and histograms, you do not need to include a zero baseline for a line chart. Recall that the main goal of a line chart is to emphasize changes in value, rather than the magnitude of the values themselves. In cases where a zero line is not meaningful or useful, it's fine to zoom the vertical axis range into what will make the changes in value most informative.



5.1.6 Failing to identify uneven gaps between points

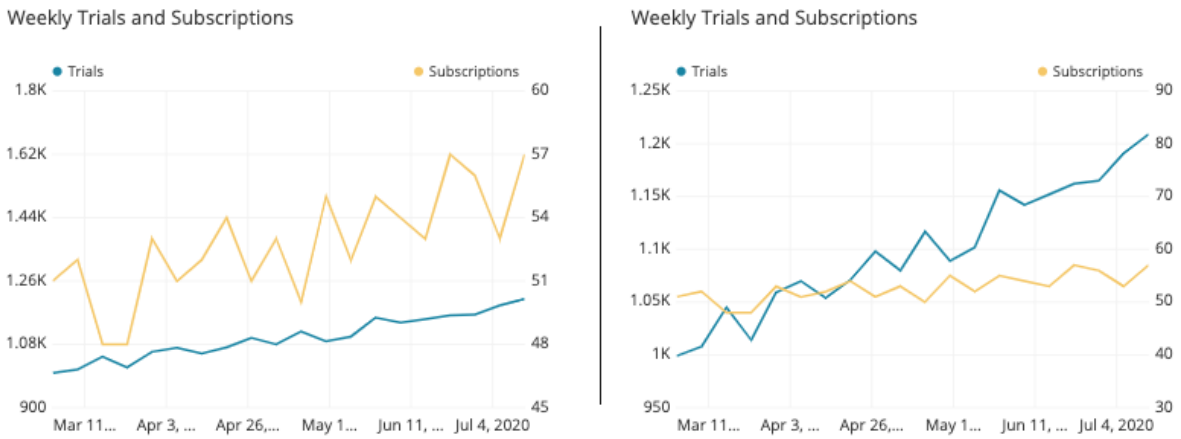
When the line chart is missing information for certain bins, gaps in the record may be interpreted as phantom values if the line does not include distinct dots at each observation. When there aren't many points to plot, try showing all of the points and not just the line. If including the points would muddy up the interpretability of the plot, another alternative is to include a gap in the line to show where there are missing values.



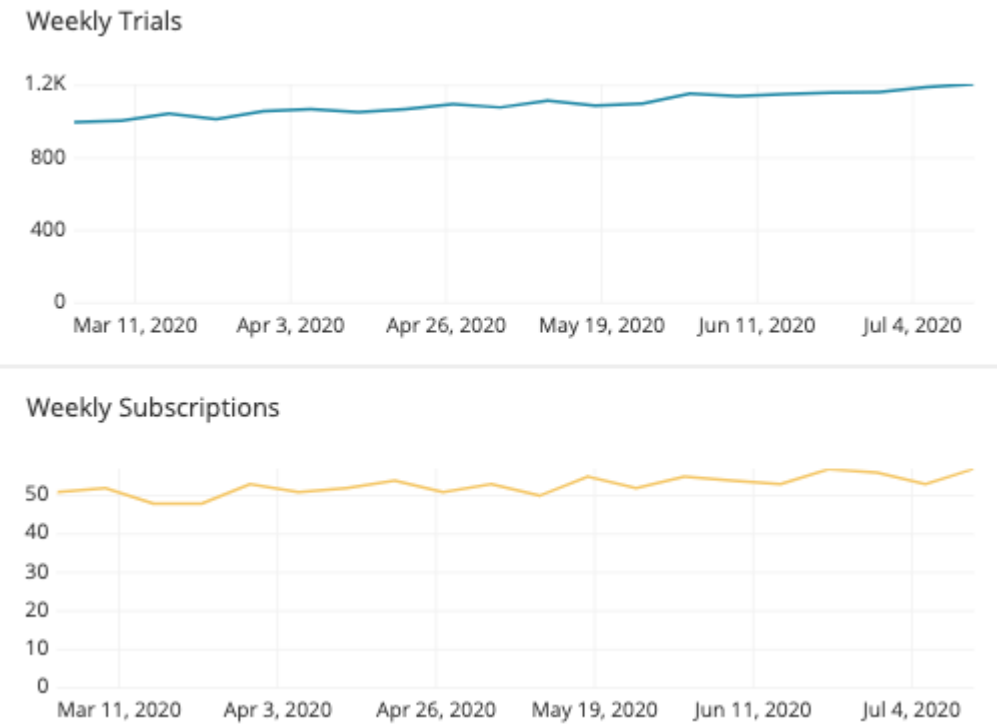
5.1.7 Using a misleading dual axis

Examples of line charts with multiple lines have thus far had each line be part of the same domain, and thus plottable on the same axis. There's nothing that limits each line to depict values on the same units, however. When a line plot includes two series, each depicting a summary of a different variable, then we end up with a dual axis plot.

The problem with a dual-axis plot is that it can easily be manipulated to be misleading. Depending on how each axis is scaled, the perceived relationship between the two lines can be changed.



Instead, faceting the two lines into separate plots still allows for the general patterns of change to be observed for both variables, while reducing the temptations to compare them in misleading ways.



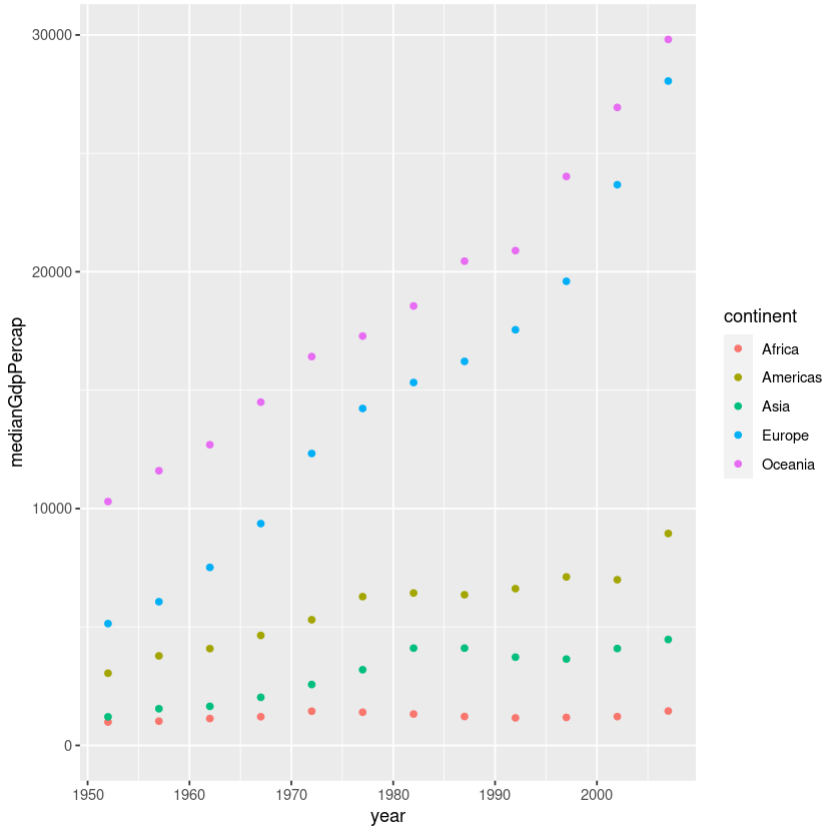
5.1.8 let us go back to the code in R!

In [15]:
```
library(ggplot2)
library(gapminder)
library(dplyr)


by_year_continent <- gapminder %>%
  group_by(continent, year) %>%
  summarize(medianGdpPercap = median(gdpPercap))

ggplot(by_year_continent, aes(x = year, y = medianGdpPercap, color = continent)) +
geom_point() +
expand_limits(y = 0)
```

```
`summarise()` has grouped output by 'continent'. You can override using the `.g
roups` argument.
```

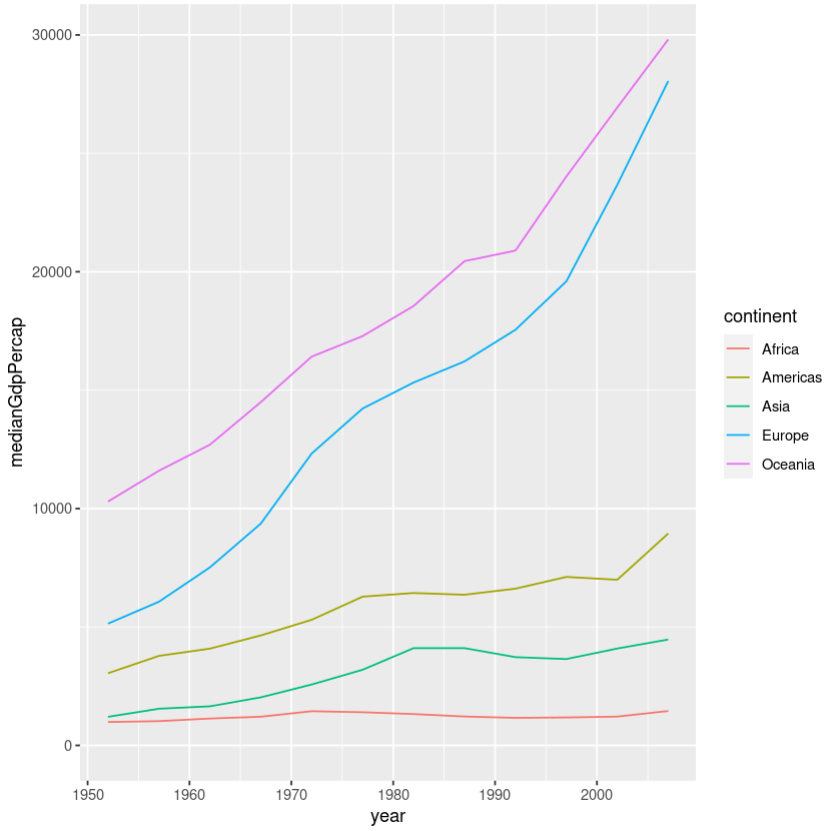


In [2]:
```
library(ggplot2)
library(gapminder)
library(dplyr)


by_year_continent <- gapminder %>%
  group_by(continent, year) %>%
  summarize(medianGdpPercap = median(gdpPercap))

ggplot(by_year_continent, aes(x = year, y = medianGdpPercap, color = continent)) +
geom_line() +
expand_limits(y = 0)
```

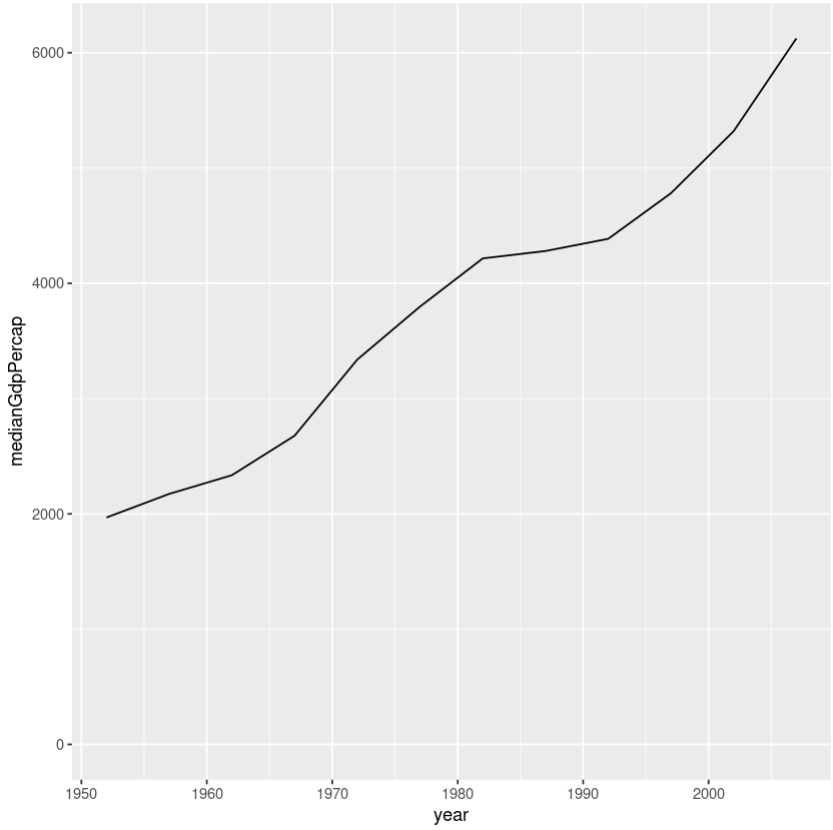
```
`summarise()` has grouped output by 'continent'. You can override using the `.g
roups` argument.
```



it's a bit easier to understand as a line plot, where the observations within each continent are connected. This makes it clearer that what we care about is the upward or downward trend over time.

In [3]:
```
by_year <- gapminder %>%
group_by(year) %>%
summarise(medianGdpPercap = median(gdpPercap))

ggplot(by_year, aes(x = year, y = medianGdpPercap)) +
geom_line() +
expand_limits(y = 0)
```

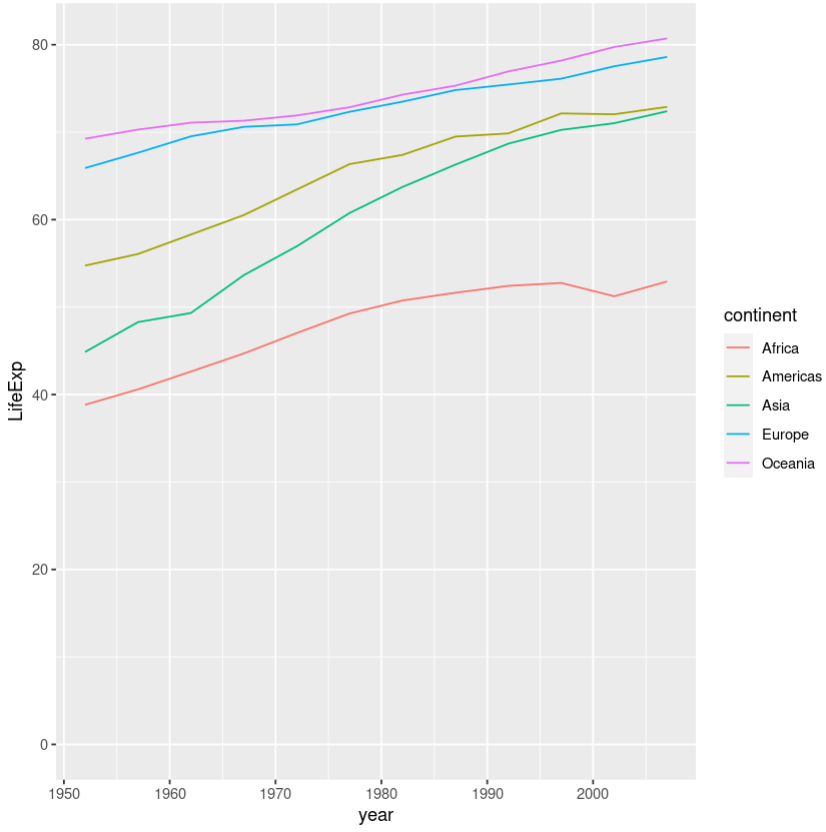


In [2]:
```
by_year_continent <- gapminder %>%
group_by(year, continent) %>%
summarise(LifeExp = median(lifeExp))



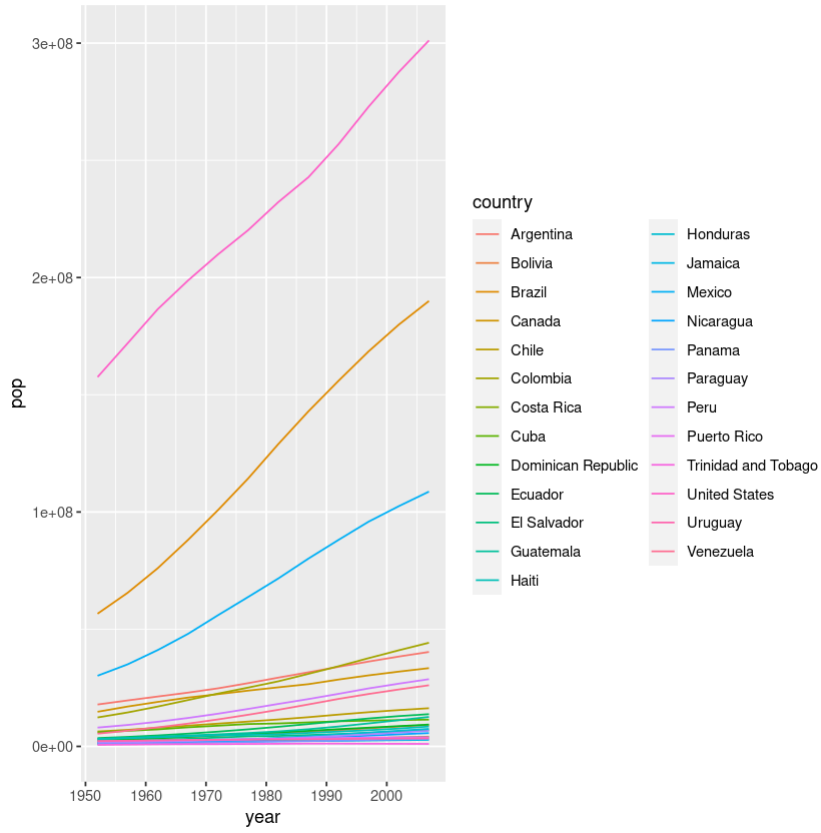
ggplot(by_year_continent, aes(x = year, y = LifeExp, color = continent)) +
geom_line() +
expand_limits(y = 0)
```

```
`summarise()` has grouped output by 'year'. You can override using the `.groups
` argument.
```



In [25]:
```
a1 <- gapminder %>%
filter(continent == "Americas") %>%
select(country, year, pop)


ggplot(a1, aes(x = year, y = pop, color = country)) +
geom_line() +
expand_limits(y = 0)
```

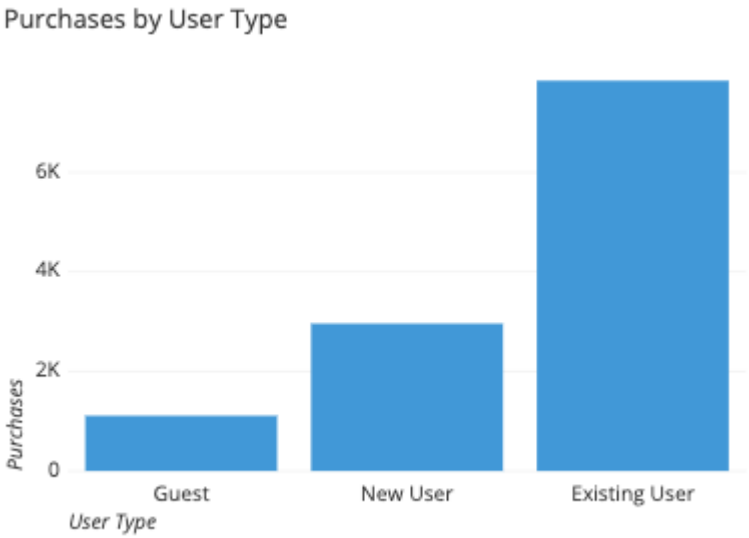


5.2 Bar plots

Bar plots, which are good at comparing statistics for each of several categories.

5.2.1 What is a bar plot?

A bar chart (aka bar graph, column chart) plots numeric values for levels of a categorical feature as bars. Levels are plotted on one chart axis, and values are plotted on the other axis. Each categorical value claims one bar, and the length of each bar corresponds to the bar's value. Bars are plotted on a common baseline to allow for easy comparison of values.



5.2.2 When you should use a bar chart?

A bar chart is used when you want to show a distribution of data points or perform a comparison of metric values across different subgroups of your data. From a bar chart, we can see which groups are highest or most common, and how other groups compare against the others. Since this is a fairly common task, bar charts are a fairly ubiquitous chart type.

The primary variable of a bar chart is its categorical variable. A categorical variable takes discrete values, which can be thought of as labels. Examples include state or country, industry type, website access method (desktop, mobile), and visitor type (free, basic, premium). Some categorical variables have ordered values, like dividing objects by size (small, medium, large). In addition, some non-categorical variables can be converted into groups, like aggregating temporal data based on date (eg. dividing by quarter into 20XX-Q1, 20XX-Q2, 20XX-Q3, 20XX-Q4, etc.) The important point for this primary variable is that the groups are distinct.

In contrast, the secondary variable will be numeric in nature. The secondary variable's values determine the length of each bar. These values can come from a great variety of sources. In its simplest form, the values may be a simple frequency count or proportion for how much of the data is divided into each category. Other times, the values may be an average, total, or some other summary measure computed separately for each group.

5.2.3 Example of data structure

Data rendered as a bar chart might come in a compact form like the above table, with one column for the categories and the second column for their values.

In [38]:
```
america <- gapminder %>%
filter(continent == "Americas") %>%
group_by(country) %>%
summarise(averagepop = mean(pop))

america
```

A tibble: 25 × 2

country	averagepop
<fct>	<dbl>
Argentina	28602240
Bolivia	5610395
Brazil	122312127
Canada	24462967
Chile	11205725
Colombia	27256100
Costa Rica	2400008
Cuba	9232576
Dominican Republic	5735697
Ecuador	8212243
El Salvador	4343441
Guatemala	6858695
Haiti	5401634
Honduras	3920593
Jamaica	2135123
Mexico	68414297
Nicaragua	3040975
Panama	1985290
Paraguay	3555486
Peru	17559093
Puerto Rico	3115178
Trinidad and Tobago	1006470
United States	228211232
Uruguay	2912487
Venezuela	15129801

Other times, data may come in its unaggregated form...

In [39]:
```
america <- gapminder %>%
filter(continent == "Americas") %>%
group_by(country) %>%
select(country, pop)

america
```

A grouped_df: 300 × 2

country	pop
<fct>	<int>
Argentina	17876956
Argentina	19610538
Argentina	21283783
Argentina	22934225
Argentina	24779799
Argentina	26983828
Argentina	29341374
Argentina	31620918
Argentina	33958947
Argentina	36203463
Argentina	38331121
Argentina	40301927
Bolivia	2883315
Bolivia	3211738
Bolivia	3593918
Bolivia	4040665
Bolivia	4565872
Bolivia	5079716
Bolivia	5642224
Bolivia	6156369
Bolivia	6893451
Bolivia	7693188
Bolivia	8445134
Bolivia	9119152
Brazil	56602560
Brazil	65551171
Brazil	76039390
Brazil	88049823
Brazil	100840058

country	pop
<fct>	<int>
Brazil	114313951
⋮	⋮
United States	232187835
United States	242803533
United States	256894189
United States	272911760
United States	287675526
United States	301139947
Uruguay	2252965
Uruguay	2424959
Uruguay	2598466
Uruguay	2748579
Uruguay	2829526
Uruguay	2873520
Uruguay	2953997
Uruguay	3045153
Uruguay	3149262
Uruguay	3262838
Uruguay	3363085
Uruguay	3447496
Venezuela	5439568
Venezuela	6702668
Venezuela	8143375
Venezuela	9709552
Venezuela	11515649
Venezuela	13503563
Venezuela	15620766
Venezuela	17910182
Venezuela	20265563
Venezuela	22374398
Venezuela	24287670
Venezuela	26084662

5.2.4 Use a common zero-valued baseline

First and foremost, make sure that all of your bars are being plotted against a zero-value baseline. Not only does that baseline make it easier for readers to compare bar lengths, it also maintains the truthfulness of your data visualization. A bar chart with a non-zero baseline or some other gap in the axis scale can easily misrepresent the comparison between groups since the ratio in bar lengths will not match the ratio in actual bar values.

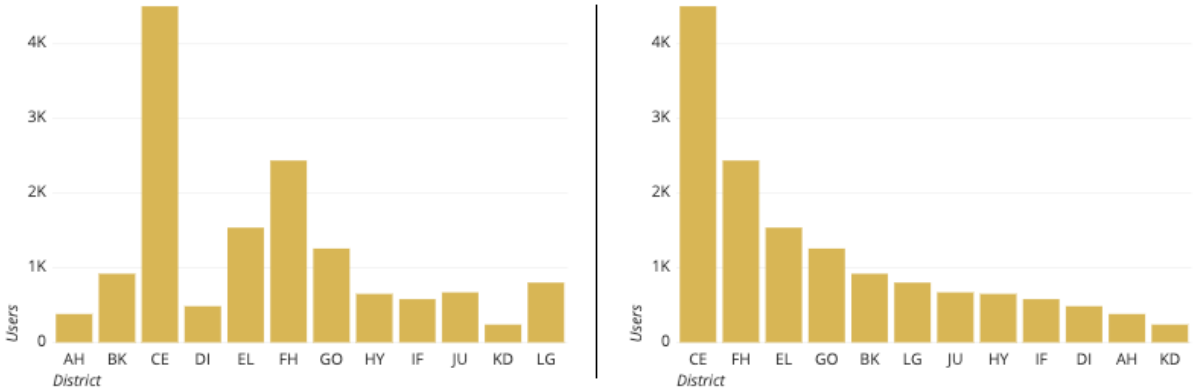






5.2.5 Consider the ordering of category levels

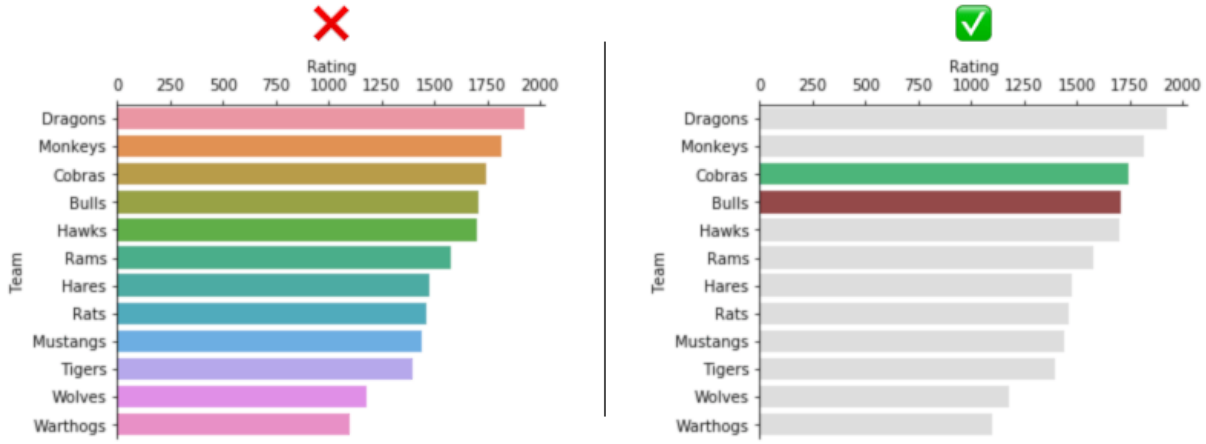
One consideration you should have when putting together a bar chart is what order in which you will plot the bars. A standard convention to take is to sort the bars from longest to shortest: while it is always possible to compare the bar lengths no matter the order, this can reduce the burden on the reader to make those comparisons themselves. The major exception to this is if the category labels are inherently ordered in some way.



5.2.6 Use color wisely

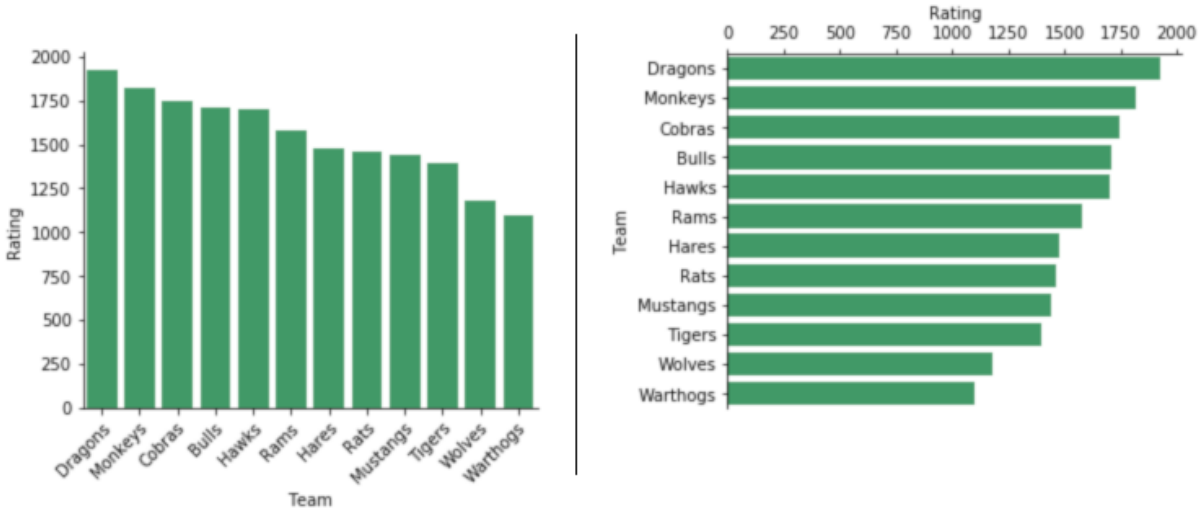
Another consideration is on how you should use color in your bar charts. Certain tools will color

each bar differently by default, but this can distract the reader by implying additional meaning where none exists. Instead, color should be used with purpose. For example, you might use color to highlight specific columns for storytelling. Colors can also be used if they are meaningful for the categories posted (e.g. to match company or team colors).



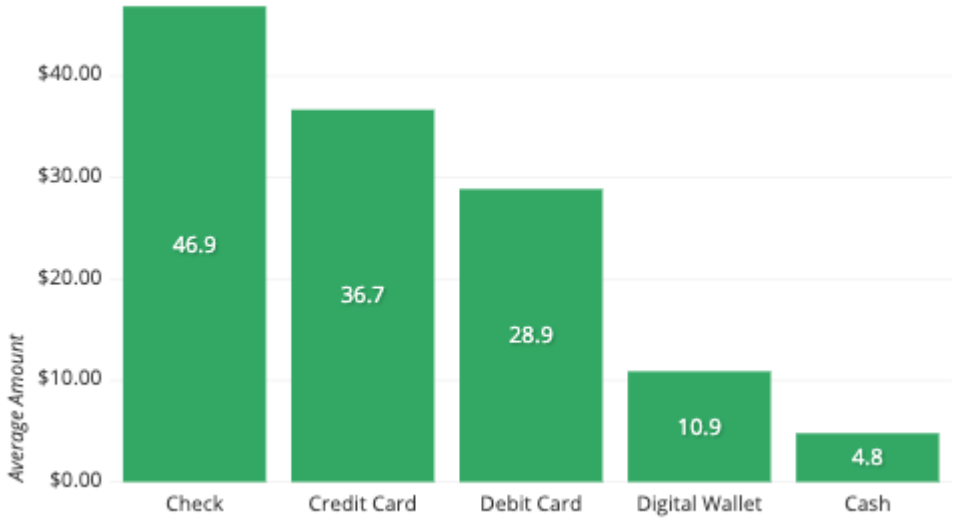
5.2.7 Horizontal bars vs. vertical bars

A common bar chart variation is whether or not the bar chart should be oriented vertically (with categories on the horizontal axis) or horizontally (with categories on the vertical axis). While the vertical bar chart is usually the default, it's a good idea to use a horizontal bar chart when you are faced with long category labels. In a vertical chart, these labels might overlap, and would need to be rotated or shifted to remain legible; the horizontal orientation avoids this issue.



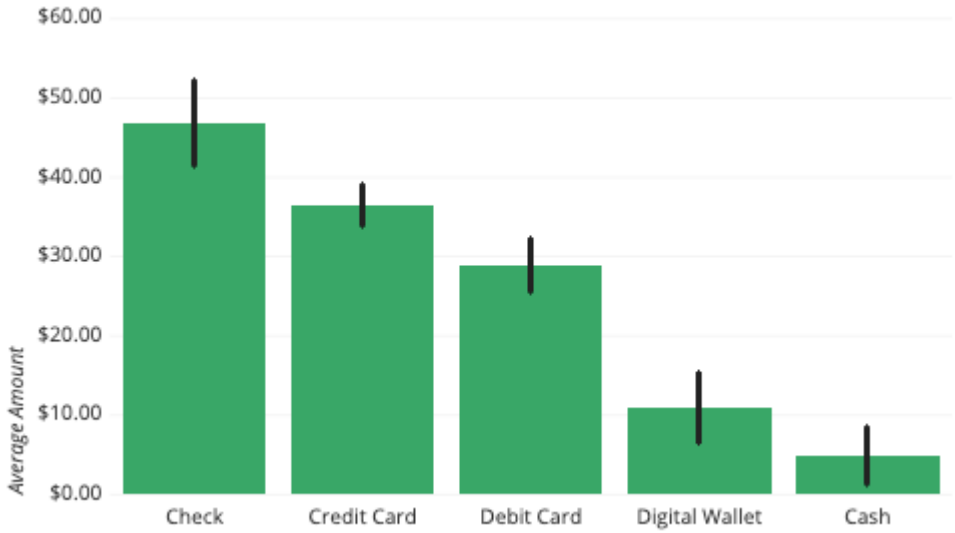
5.2.8 Include value annotations

A common addition to bar charts are value annotations. While it is fairly easy for readers to compare bar lengths and gauge approximate values from a bar chart, exact values aren't necessarily easy to state. Annotations can report these values where they are important, and are usually placed in the middle of the bar or at their ends.



5.2.9 Include variability whiskers

When the numeric values are a summary measure, a frequent consideration is whether or not to include error bars in the plot. Error bars are additional whiskers added to the end of each bar to indicate variability in the individual data points that contributed to the summary measure. Since there are many choices for uncertainty measure (e.g. standard deviation, confidence interval, interquartile range) it is important that when you display error bars, that you note in an annotation or comment what the error bars represent.



5.2.10 let us go back to the code in R!

In [40]:
```
by_continent <- gapminder %>%
filter(year == 2007) %>%
group_by(continent) %>%
summarise(meanLifeExp = mean(lifeExp))

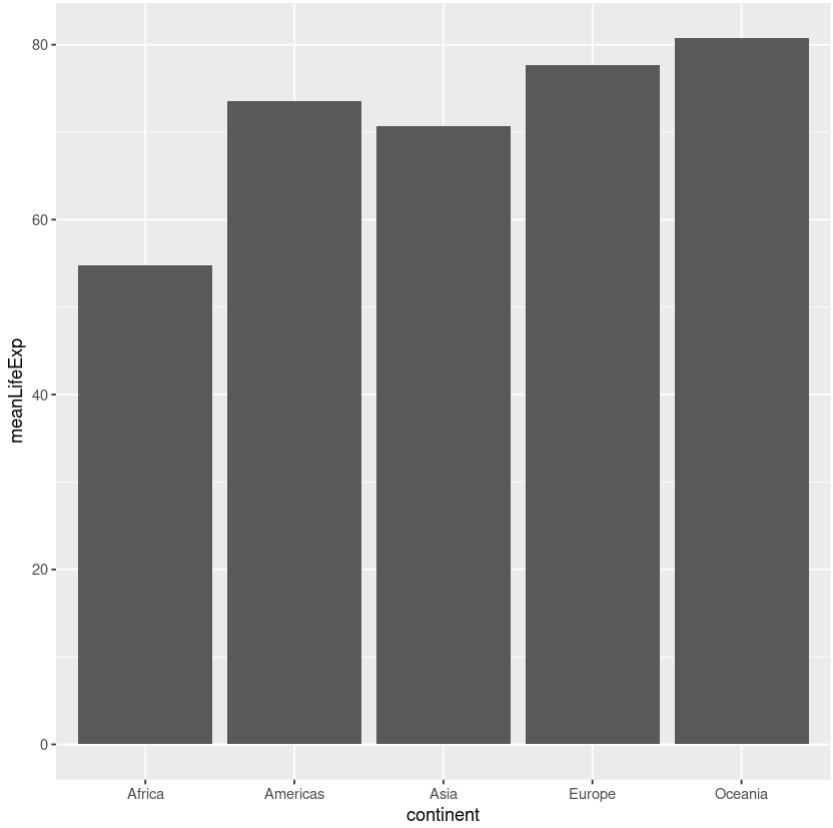
by_continent
```

A tibble: 5 × 2

continent	meanLifeExp
<fct>	<dbl>
Africa	54.80604
Americas	73.60812
Asia	70.72848
Europe	77.64860
Oceania	80.71950

This code finds the average life expectancy within each continent in the year 2007. That creates a table that looks like this, with one observation for each continent. Instead of just printing the table, you might want to represent the summary visually. For that, you would use a bar plot.

In [41]:
```
ggplot(by_continent, aes(x= continent, y = meanLifeExp)) +
geom_col()
```



In [44]:
```
by_continent1 <- gapminder %>%
filter(year == 2007) %>%
group_by(continent) %>%
summarise(medianGdpPercap = median(gdpPercap), meanGdpPercap = mean(gdpPercap))

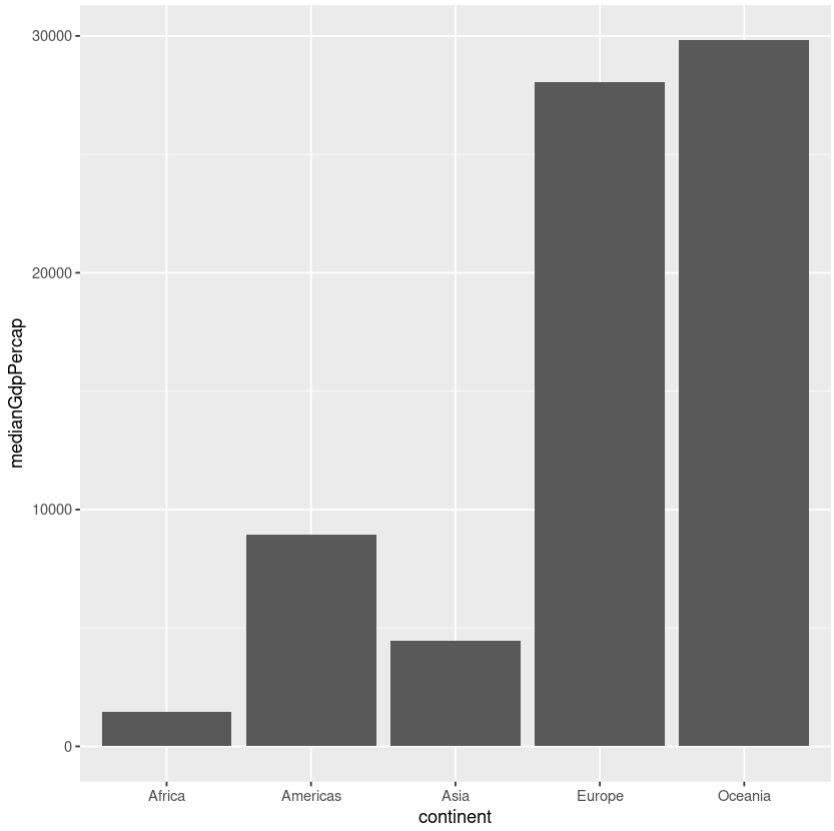
by_continent1

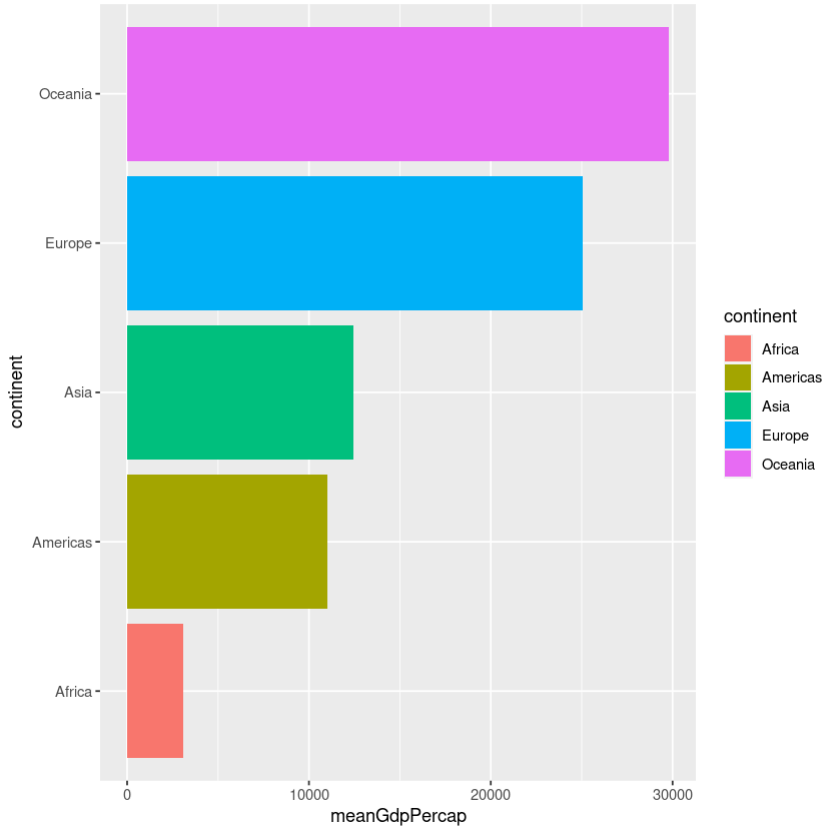
ggplot(by_continent1, aes(x= continent, y = medianGdpPercap)) +
geom_col()

ggplot(by_continent1, aes(x= continent, y = meanGdpPercap, fill=continent)) +
geom_col()+ coord_flip()
```

A tibble: 5 × 3

continent	medianGdpPercap	meanGdpPercap
<fct>	<dbl>	<dbl>
Africa	1452.267	3089.033
Americas	8948.103	11003.032
Asia	4471.062	12473.027
Europe	28054.066	25054.482
Oceania	29810.188	29810.188




Oceania
Europe
Asia
Americas
Africa
continent
0
10000
20000
30000
meanGdpPercap
continent
Africa
Americas
Asia
Europe
Oceania

In [107]:
```
oceania <- gapminder %>%
filter(continent == "Oceania") %>%
group_by(country) %>%
summarise(meanGdpPercap = mean(gdpPercap), sdGdpPercap = sd(gdpPercap))

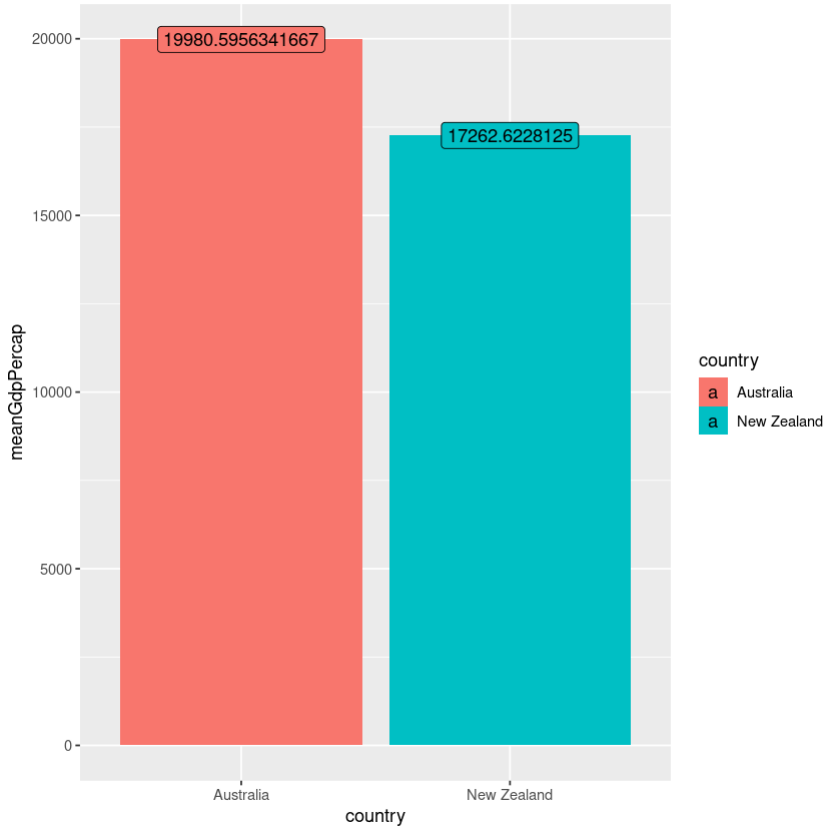
oceania

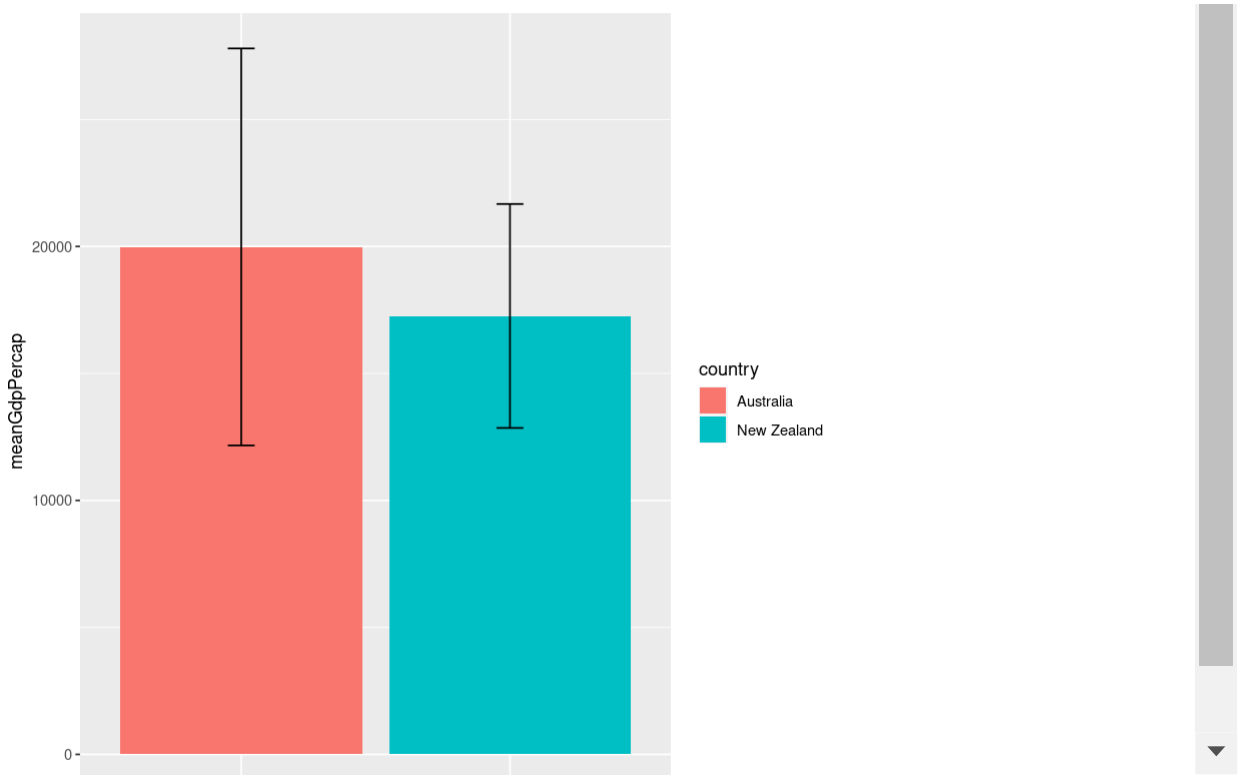
# Create a bar plot with labels
ggplot(oceania, aes(x= country, y = meanGdpPercap, fill = country)) +
geom_col() +
geom_label(aes(x=country, label=meanGdpPercap))
#geom_text(aes(label = meanGdpPercap), vjust = -1)


# Create a bar plot with errors
ggplot(oceania, aes(x= country, y = meanGdpPercap, fill = country)) +
geom_col() +
geom_errorbar(aes(ymin=meanGdpPercap-sdGdpPercap, ymax=meanGdpPercap+sdGdpPercap)
                 position=position_dodge(.9))
```

A tibble: 2 × 3

country	meanGdpPercap	sdGdpPercap
<fct>	<dbl>	<dbl>
Australia	19980.60	7815.405
New Zealand	17262.62	4409.009

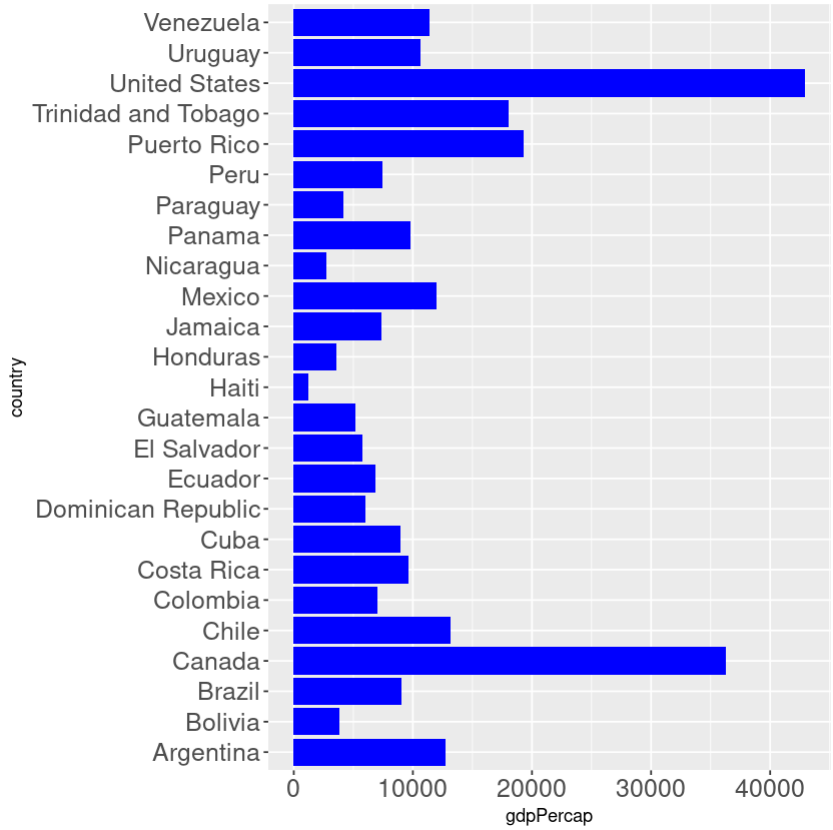




In [94]:
```
?geom_label
```

In [120]:
```
america <- gapminder %>%
filter(year == 2007, continent == "Americas")


# Create a bar plot of gdpPercap by country
ggplot(america, aes(x= country, y = gdpPercap)) +
geom_col(fill = "blue") +
coord_flip() +
theme(axis.text = element_text(size = 15))
```



In [121]:
```
#Sorting????

america2 <- gapminder %>%
filter(year == 2007, continent == "Americas") %>%
arrange(gdpPercap)

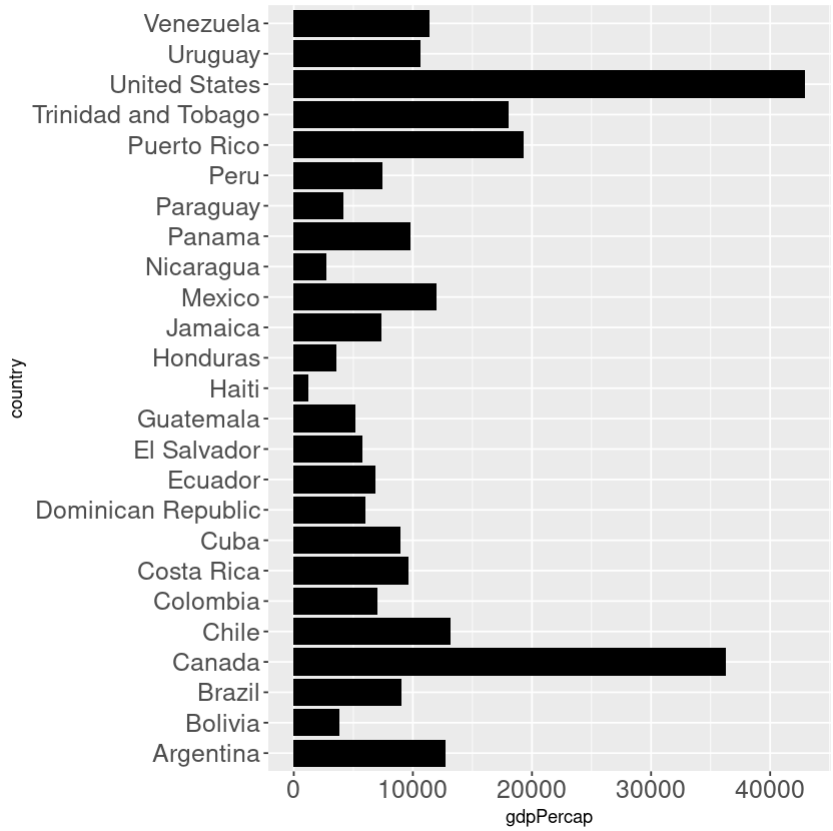
america2

# Create a bar plot of gdpPercap by country
ggplot(america2, aes(x= country, y = gdpPercap)) +
geom_col(fill = "black") +
coord_flip()  +
theme(axis.text = element_text(size = 15))
```

A tibble: 25 × 6

country	continent	year	lifeExp	pop	gdpPercap
<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
Haiti	Americas	2007	60.916	8502814	1201.637
Nicaragua	Americas	2007	72.899	5675356	2749.321
Honduras	Americas	2007	70.198	7483763	3548.331
Bolivia	Americas	2007	65.554	9119152	3822.137
Paraguay	Americas	2007	71.752	6667147	4172.838
Guatemala	Americas	2007	70.259	12572928	5186.050
El Salvador	Americas	2007	71.878	6939688	5728.354
Dominican Republic	Americas	2007	72.235	9319622	6025.375
Ecuador	Americas	2007	74.994	13755680	6873.262
Colombia	Americas	2007	72.889	44227550	7006.580
Jamaica	Americas	2007	72.567	2780132	7320.880
Peru	Americas	2007	71.421	28674757	7408.906
Cuba	Americas	2007	78.273	11416987	8948.103
Brazil	Americas	2007	72.390	190010647	9065.801
Costa Rica	Americas	2007	78.782	4133884	9645.061
Panama	Americas	2007	75.537	3242173	9809.186
Uruguay	Americas	2007	76.384	3447496	10611.463
Venezuela	Americas	2007	73.747	26084662	11415.806
Mexico	Americas	2007	76.195	108700891	11977.575
Argentina	Americas	2007	75.320	40301927	12779.380
Chile	Americas	2007	78.553	16284741	13171.639
Trinidad and Tobago	Americas	2007	69.819	1056608	18008.509
Puerto Rico	Americas	2007	78.746	3942491	19328.709
Canada	Americas	2007	80.653	33390141	36319.235

	country	continent	year	lifeExp	pop	gdpPercap
	<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
	United States	Americas	2007	78.242	301139947	42951.653



In [122]:
```
#Now is sorted

america3 <- gapminder %>%
filter(year == 2007, continent == "Americas")

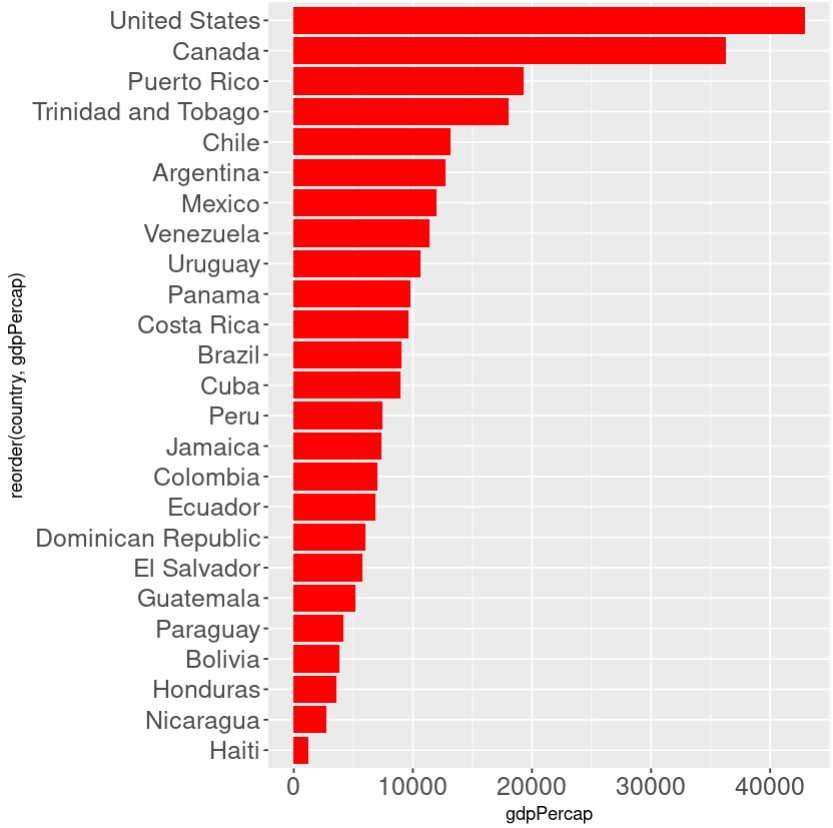
america3

# Create a bar plot of gdpPercap by country
ggplot(america3, aes(x= reorder(country, gdpPercap), y = gdpPercap)) +
geom_col(fill = "red") +
coord_flip()  +
theme(axis.text = element_text(size = 15))
```

A tibble: 25 × 6

country	continent	year	lifeExp	pop	gdpPercap
<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
Argentina	Americas	2007	75.320	40301927	12779.380
Bolivia	Americas	2007	65.554	9119152	3822.137
Brazil	Americas	2007	72.390	190010647	9065.801
Canada	Americas	2007	80.653	33390141	36319.235
Chile	Americas	2007	78.553	16284741	13171.639
Colombia	Americas	2007	72.889	44227550	7006.580
Costa Rica	Americas	2007	78.782	4133884	9645.061
Cuba	Americas	2007	78.273	11416987	8948.103
Dominican Republic	Americas	2007	72.235	9319622	6025.375
Ecuador	Americas	2007	74.994	13755680	6873.262
El Salvador	Americas	2007	71.878	6939688	5728.354
Guatemala	Americas	2007	70.259	12572928	5186.050
Haiti	Americas	2007	60.916	8502814	1201.637
Honduras	Americas	2007	70.198	7483763	3548.331
Jamaica	Americas	2007	72.567	2780132	7320.880
Mexico	Americas	2007	76.195	108700891	11977.575
Nicaragua	Americas	2007	72.899	5675356	2749.321
Panama	Americas	2007	75.537	3242173	9809.186
Paraguay	Americas	2007	71.752	6667147	4172.838
Peru	Americas	2007	71.421	28674757	7408.906
Puerto Rico	Americas	2007	78.746	3942491	19328.709
Trinidad and Tobago	Americas	2007	69.819	1056608	18008.509
United States	Americas	2007	78.242	301139947	42951.653
Uruguay	Americas	2007	76.384	3447496	10611.463

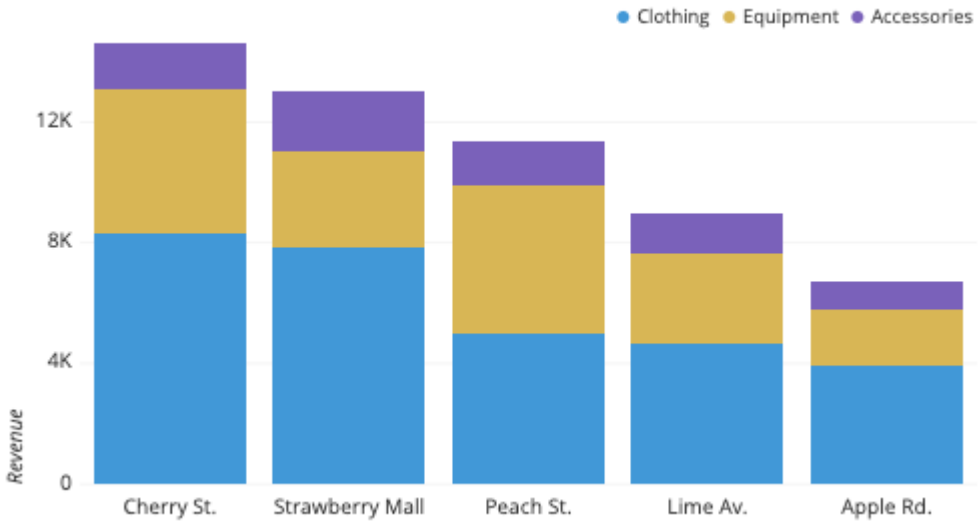
country	continent	year	lifeExp	pop	gdpPercap
<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
Venezuela	Americas	2007	73.747	26084662	11415.806



5.3 Stacked Bar plots

5.3.1 What is a stacked bar chart?

The stacked bar chart (aka stacked bar graph) extends the standard bar chart from looking at numeric values across two or more categorical variables. Each bar in a standard bar chart is divided into a number of sub-bars stacked end to end, each one corresponding to a level of the second categorical variable.



The stacked bar chart above depicts revenue from a fictional fitness retailer for a particular period of time, across two categorical variables: store location and department. The primary categorical variable is store location: we can see from the sorted overall bar heights that the Cherry St. location has the highest revenue and Apple Rd. lowest. Each bar is subdivided based on levels of the second categorical variable, department. We can see that for most locations, clothing is quite a bit larger in sales than equipment, which in turn is larger than accessories. The Strawberry Mall location appears to have a lower proportion of revenue attributed to equipment, while equipment has a larger share for Peach St.

5.3.2 When you should use a stacked bar chart

The main objective of a standard bar chart is to compare numeric values between levels of a categorical variable. One bar is plotted for each level of the categorical variable, each bar's length indicating numeric value. A stacked bar chart also achieves this objective, but also targets a second goal.

We want to move to a stacked bar chart when we care about the relative decomposition of each primary bar based on the levels of a second categorical variable. Each bar is now comprised of a number of sub-bars, each one corresponding with a level of a secondary categorical variable. The total length of each stacked bar is the same as before, but now we can see how the secondary groups contributed to that total.

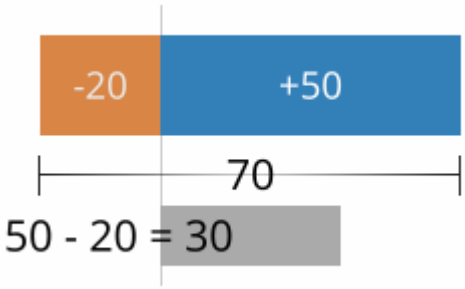
5.3.3 Order of categorical variables

One important consideration in building a stacked bar chart is to decide which of the two categorical variables will be the primary variable (dictating major axis positions and overall bar lengths) and which will be the secondary (dictating how each primary bar will be subdivided). The most 'important' variable should be the primary; use domain knowledge and the specific type of categorical variables to make a decision on how to assign your categorical variables. As another

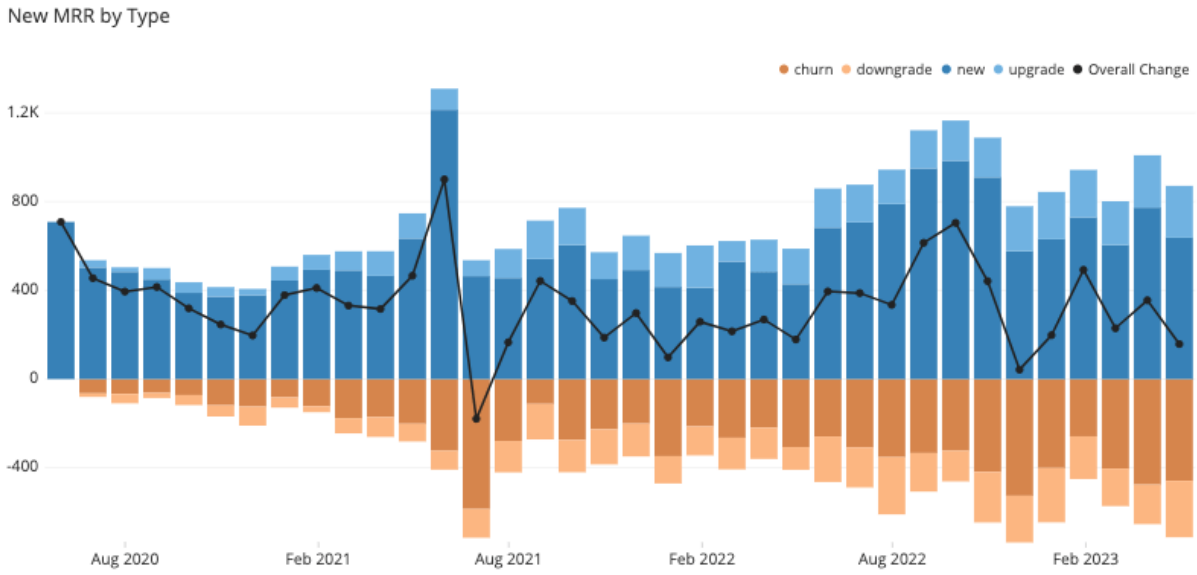
consideration, variables with more levels are generally better as primary variables; we want to limit the number of secondary levels to a fairly small amount to make the stack breakdowns easier to read.

5.3.4 Maintain a zero-baseline

When a standard bar chart encounters a negative value, the corresponding bar just gets plotted below or to the left of the baseline (depending on if the bars are vertically or horizontally oriented, respectively). In a stacked bar chart, a similar representation can be performed, just stacking bars in the negative direction.

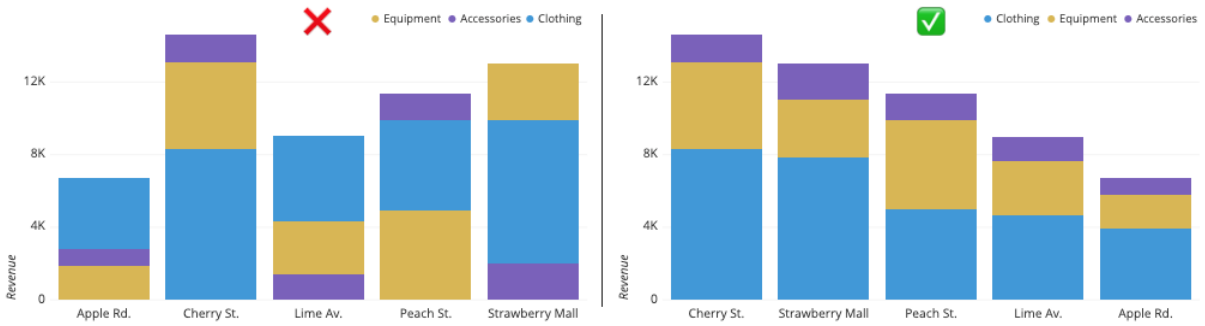


However, when positive bars and negative bars are combined, it is no longer the case that the overall length of the bar corresponds to the bar's total value. When this occurs, it is a good idea to plot an additional line or series of points on top of the bars to show the true total: the difference between the lengths of the positive bars and negative bars.



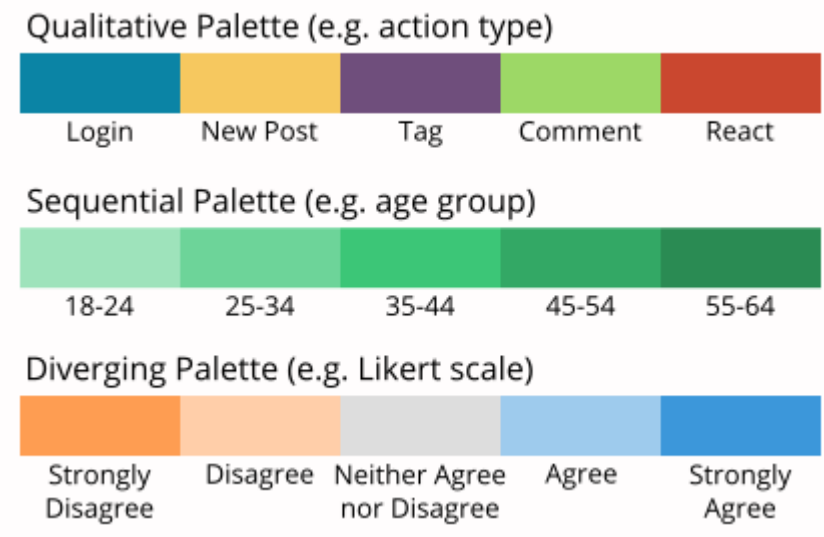
5.3.5 Ordering of category levels

With a stacked bar chart, you will need to consider the order of category levels for both categorical variables to be plotted. The rule of thumb for standard bar charts can be applied in both variables: order the bars from largest to smallest unless there is an intrinsic order of levels.



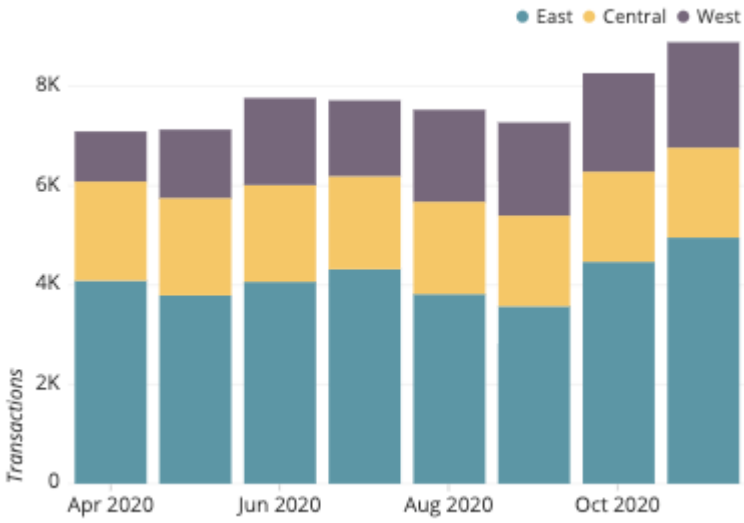
5.3.6 Choosing effective colors

While the general recommendation is to stick with a single color in a standard bar chart, use of color to distinguish secondary variable levels is an inevitability for a stacked bar chart. The important point is to make sure that the choice of color palette to assign to each categorical level matches the variable type: a qualitative palette for purely categorical variables, and sequential, or diverging for variables with a meaningful order.



5.3.7 Interpreting values on individual groups in a stacked area chart

While it is straightforward to compare the total numeric values across the levels of the primary categorical variable, it is less straightforward to gauge other divisions or comparisons using the secondary categorical variable. If we want to see the change in a secondary level across the primary categorical variable, this can only be easily done for the level plotted against the baseline. For all other secondary levels, their baseline will experience shifts, making it more difficult to judge how the sub-bar lengths change across primary bars. In the below example, it can be difficult to tell that the central yellow group is actually decreasing slightly over time.



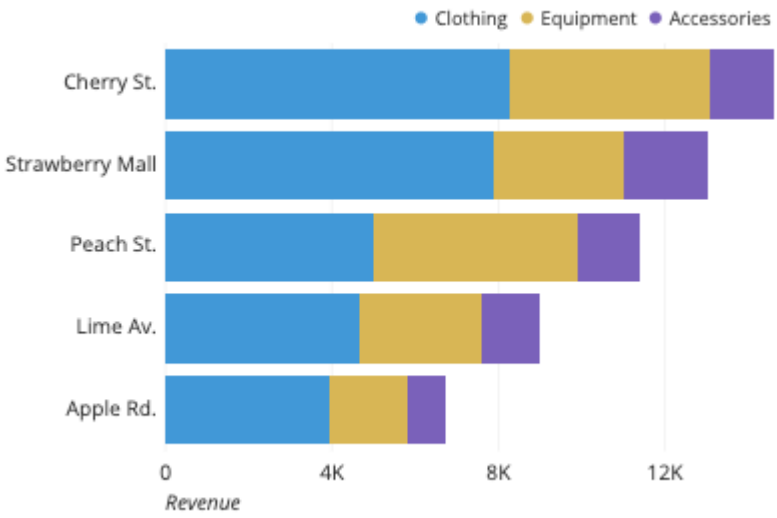
Even trying to compare sub-bars within each primary bar can be difficult. Even when following a guideline to sort secondary category levels by overall size, this presents no guarantee that they will be sorted by size within a particular primary bar. In the same image above, it can be hard to tell where the purple “West” group overtakes the yellow “Central” group in size.

Keep in mind that one of the standard goals of a stacked bar chart is to make relative judgements about the secondary categorical variable, and that making precise judgments are not as important. If comparing the sub-groups is important, then a different chart type like the **line chart** or **grouped bar chart** is warranted.

5.3.8 Common stacked bar chart options

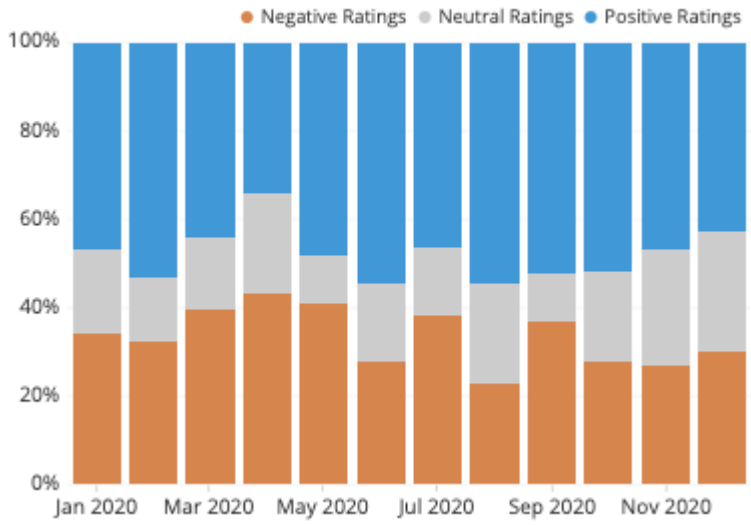
5.3.8.1 Horizontal stacked bar chart

Just like the standard bar chart, the bars in a stacked bar chart can be oriented horizontally (with primary categories on the vertical axis) as well as vertically (with primary categories on the horizontal axis). The horizontal orientation serves the same benefits as before, allowing for the easy display of long category levels without rotation or truncation.



5.3.8.2 Percentage stacked bar chart

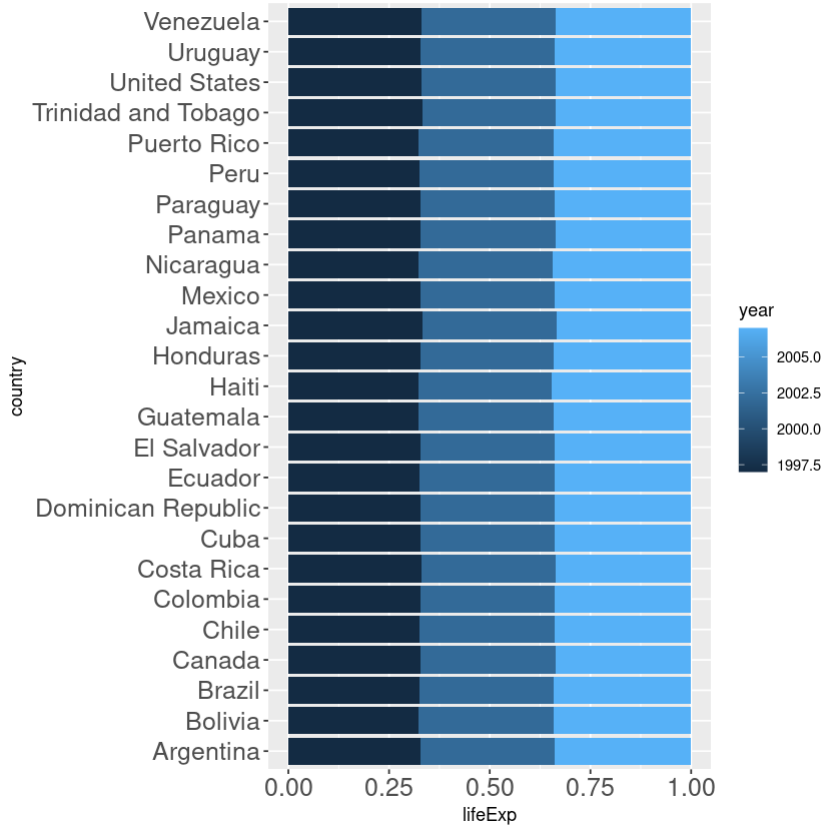
Another common option for stacked bar charts is the percentage, or relative frequency, stacked bar chart. Here, each primary bar is scaled to have the same height, so that each sub-bar becomes a percentage contribution to the whole at each primary category level. This removes our ability to compare the primary category levels' totals, but allows us to perform a better analysis of the secondary groups' relative distributions. The fixing of the heights of each primary bar to be the same also creates another baseline at the top of the chart where a second subgroup can be tracked across primary bars.



In [241]:
```
Americas <- gapminder %>%
filter(continent == "Americas", year >= 1997, year <= 2007)

#Americas

# Create a bar plot with labels
ggplot(Americas, aes(x= country, y = lifeExp, fill = year)) +
geom_bar(position = "fill", stat = "identity") +
coord_flip()  +
theme(axis.text = element_text(size = 15))
```

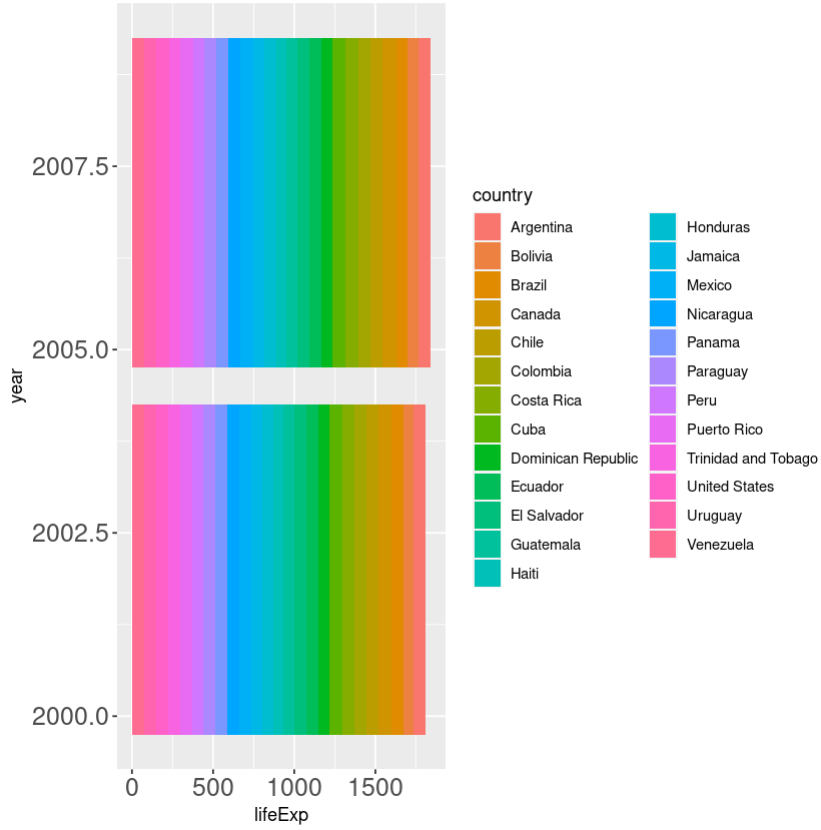


5.3.8.3 Stacked bar

In [242]:
```
Americas <- gapminder %>%
filter(continent == "Americas", year >= 2002, year <= 2007)

#Americas

# Create a bar plot with labels
ggplot(Americas, aes(x= year, y = lifeExp, fill = country)) +
geom_bar(position = "stack", stat = "identity") +
coord_flip()  +
theme(axis.text = element_text(size = 15))
```

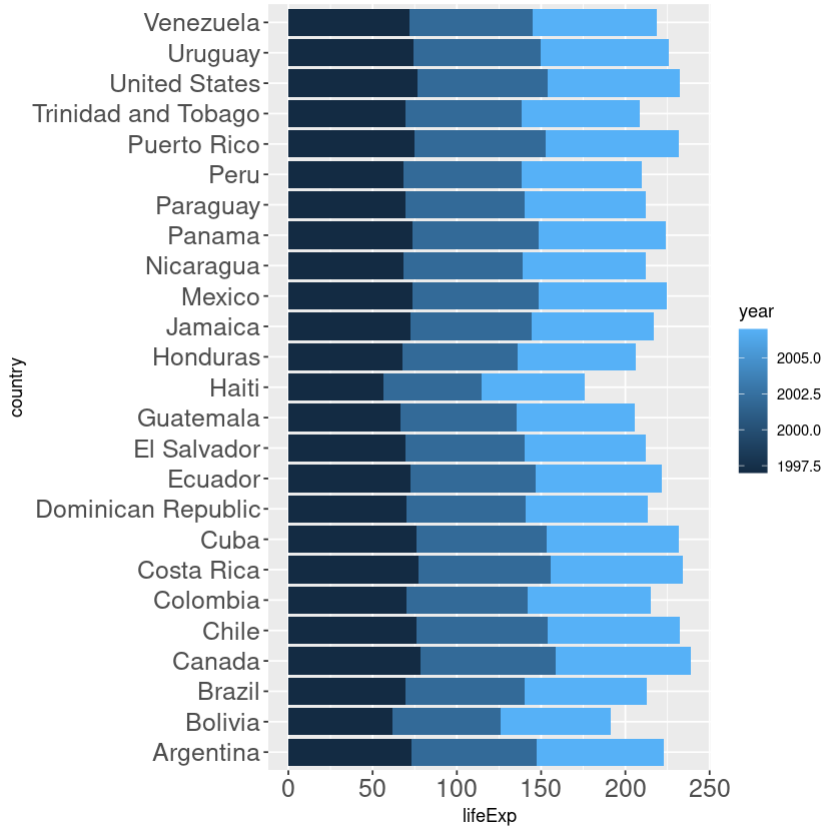


This plot sucks cause there are too many levels in the secondary category!. A better way is:

In [243]:
```
Americas <- gapminder %>%
filter(continent == "Americas", year >= 1997, year <= 2007)

#Americas

# Create a bar plot with labels
ggplot(Americas, aes(x= country, y = lifeExp, fill = year)) +
geom_bar(position = "stack", stat = "identity") +
coord_flip()  +
theme(axis.text = element_text(size = 15))
```



5.3.8.4 Percentage stacked bar

5.3.8.5 Grouped stack bar

A grouped bar chart (aka clustered bar chart, multi-series bar chart) extends the bar chart, plotting numeric values for levels of two categorical variables instead of one. Bars are grouped by position

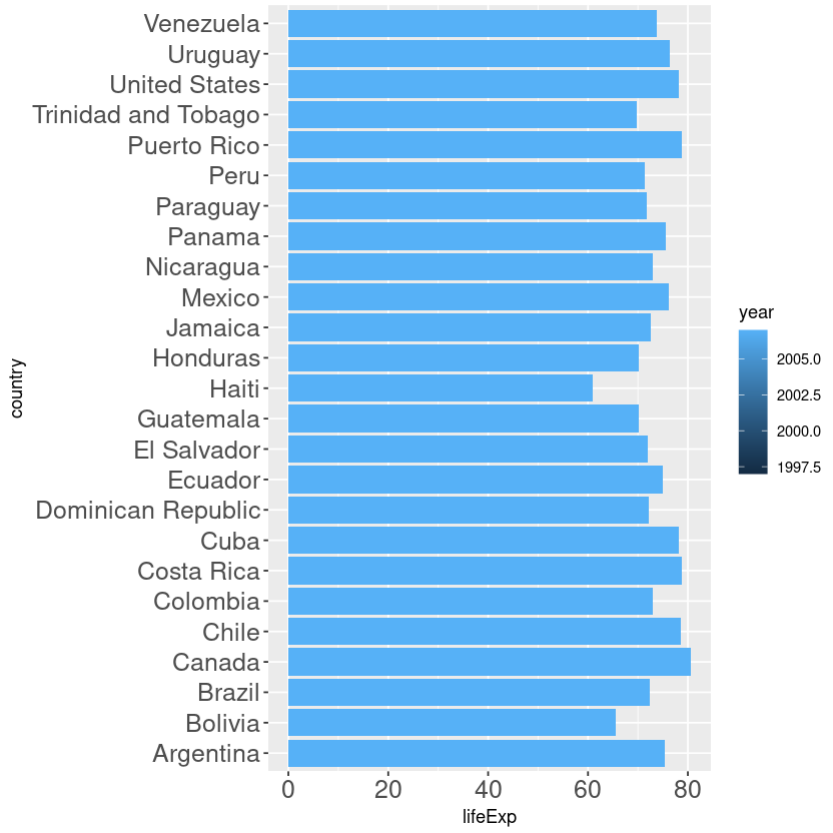
for levels of one categorical variable, with color indicating the secondary category level within each group.

In [244]:

```
Americas <- gapminder %>%
filter(continent == "Americas", year >= 1997, year <= 2007)

#Americas

# Create a bar plot with labels
ggplot(Americas, aes(x= country, y = lifeExp, fill = year)) +
geom_bar(position = position_dodge(), stat = "identity") +
coord_flip()  +
theme(axis.text = element_text(size = 15))
```

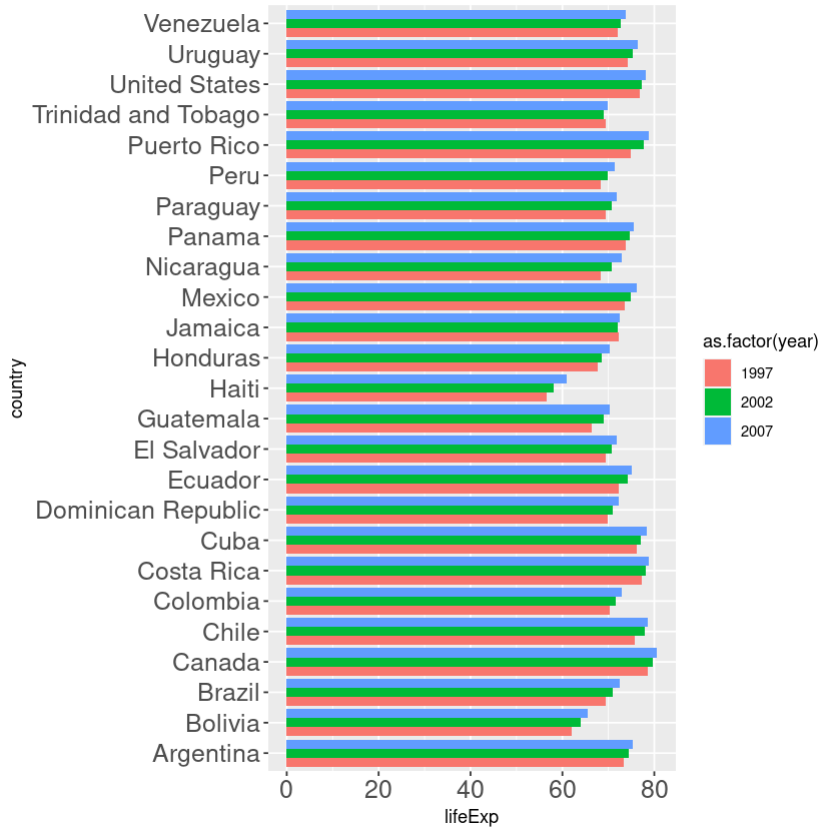


The plot dont work cause the year variable must be categorical (factor), for this:

In [245]:
```
Americas <- gapminder %>%
filter(continent == "Americas", year >= 1997, year <= 2007)

#Americas

# Create a bar plot with labels
ggplot(Americas, aes(x= country, y = lifeExp, fill = as.factor(year))) +
geom_bar(position = position_dodge(), stat = "identity") +
coord_flip()  +
theme(axis.text = element_text(size = 15))
```

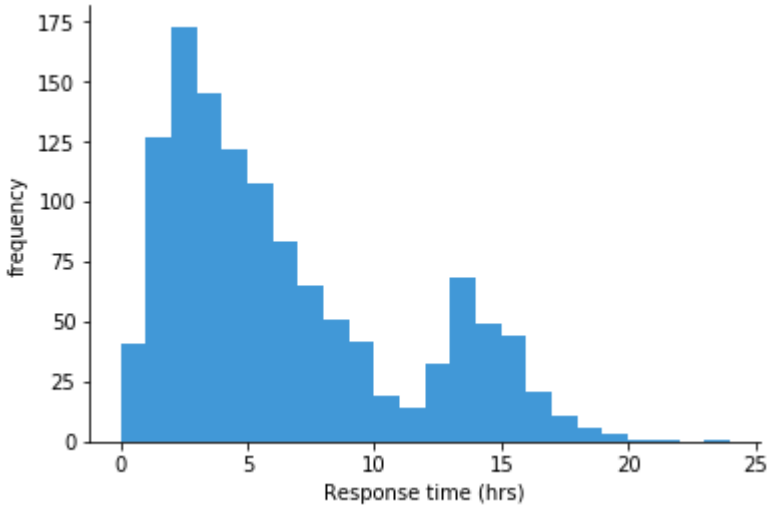


5.4 Histograms

Histograms, which describe the distribution of a one-dimensional numeric variable.

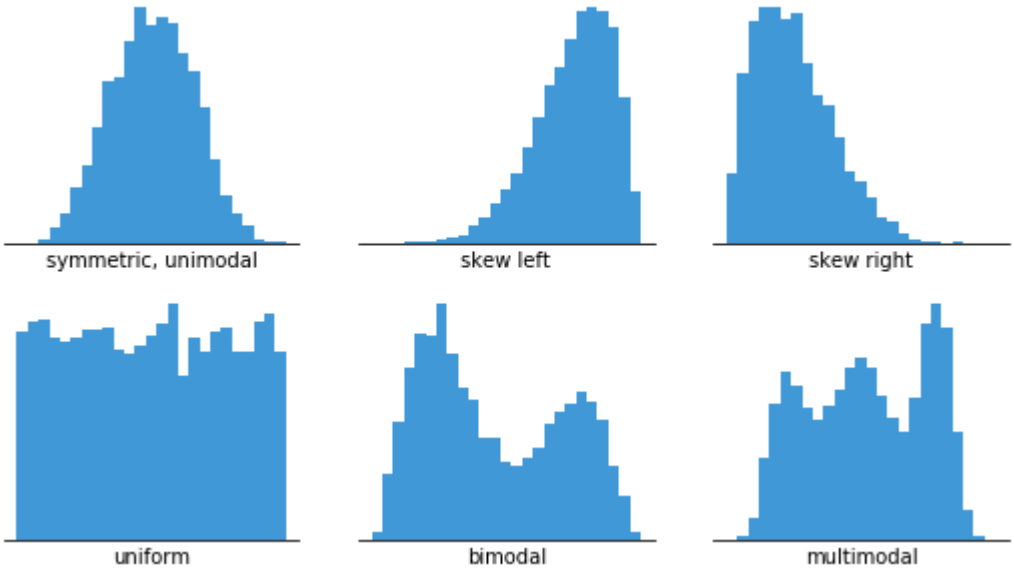
5.4.1 What is a histogram?

A histogram is a chart that plots the distribution of a numeric variable's values as a series of bars. Each bar typically covers a range of numeric values called a bin or class; a bar's height indicates the frequency of data points with a value within the corresponding bin.



5.4.2 When you should use a histogram?

Histograms are good for showing general distributional features of dataset variables. You can see roughly where the peaks of the distribution are, whether the distribution is skewed or symmetric, and if there are any outliers.



In order to use a histogram, we simply require a variable that takes continuous numeric values. This means that the differences between values are consistent regardless of their absolute values. For example, even if the score on a test might take only integer values between 0 and 100, a same-sized gap has the same meaning regardless of where we are on the scale: the difference between 60 and 65 is the same 5-point size as the difference between 90 to 95.

Information about the number of bins and their boundaries for tallying up the data points is not inherent to the data itself. Instead, setting up the bins is a separate decision that we have to make when constructing a histogram. The way that we specify the bins will have a major effect on how the histogram can be interpreted

5.4.3 Example of data structure

Bin Edges
0
1
2
...
22
23
24

Frequencies
41
127
173
...
0
1

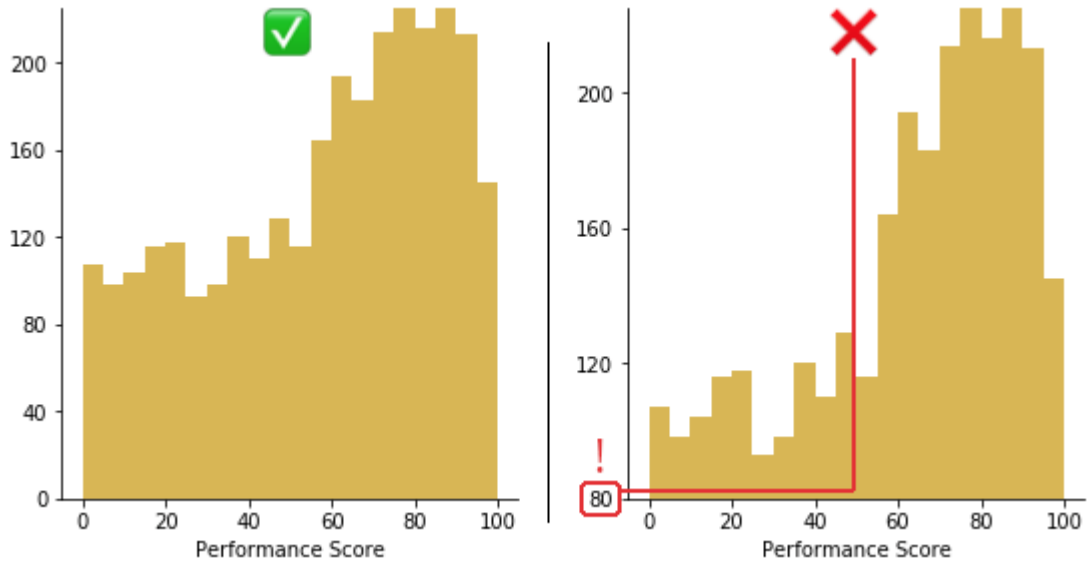
One way that visualization tools can work with data to be visualized as a histogram is from a summarized form like above. Here, the first column indicates the bin boundaries, and the second the number of observations in each bin. Alternatively, certain tools can just work with the original, unaggregated data column, then apply specified binning parameters to the data when the histogram is created.

...	response_delta	(other data columns ...)
...	1.6414	...
...	5.3931	...
...	14.3728	...
...	7.3064	...
...	2.0908	...

first_bin_edge	0
last_bin_edge	24
bin_size	1

5.4.4 Use a zero-valued baseline

An important aspect of histograms is that they must be plotted with a zero-valued baseline. Since the frequency of data in each bin is implied by the height of each bar, changing the baseline or introducing a gap in the scale will skew the perception of the distribution of data.

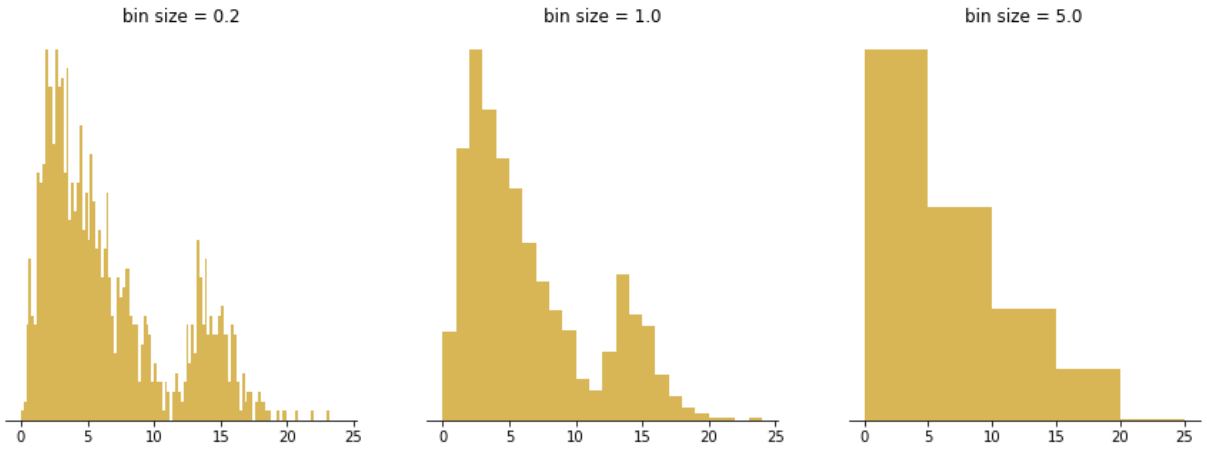


5.4.5 Choose an appropriate number of bins

While tools that can generate histograms usually have some default algorithms for selecting bin boundaries, you will likely want to play around with the binning parameters to choose something that is representative of your data.

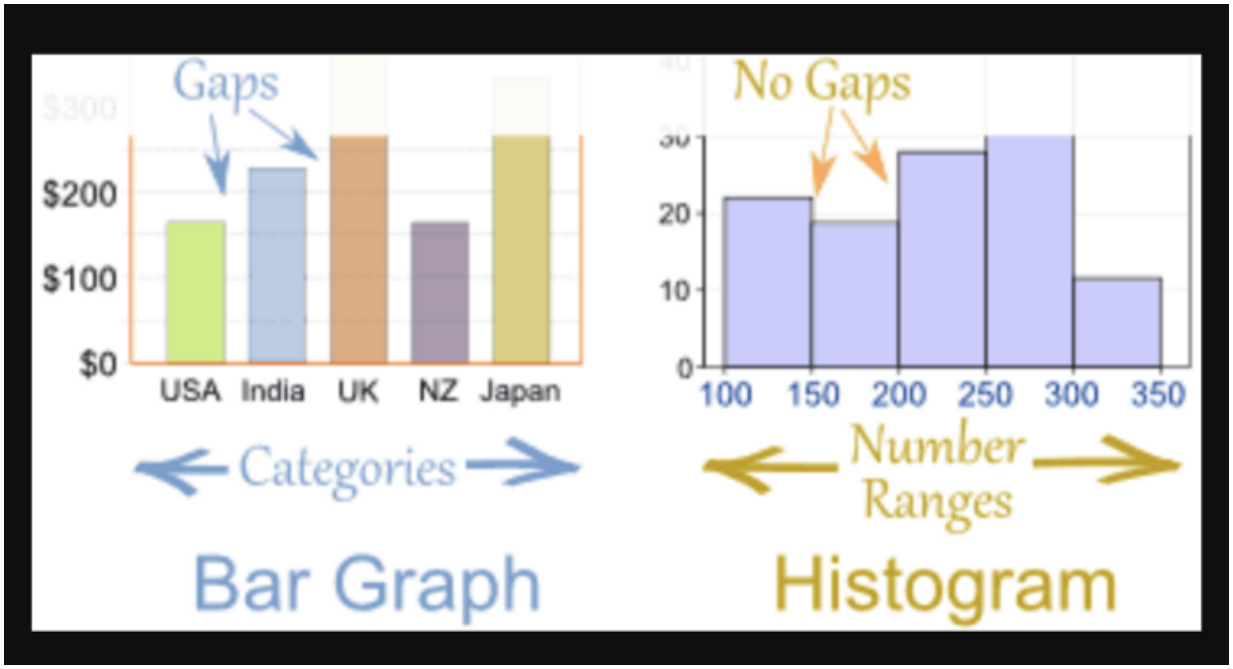
Choice of bin size has an inverse relationship with the number of bins. The larger the bin sizes, the fewer bins there will be to cover the whole range of data. With a smaller bin size, the more bins there will need to be. It is worth taking some time to test out different bin sizes to see how the distribution looks in each one, then choose the plot that represents the data best. If you have too

many bins, then the data distribution will look rough, and it will be difficult to discern the signal from the noise. On the other hand, with too few bins, the histogram will lack the details needed to discern any useful pattern from the data.



5.4.6 Measured variable is not continuous numeric

As noted in the opening sections, a histogram is meant to depict the frequency distribution of a continuous numeric variable. When our variable of interest does not fit this property, we need to use a different chart type instead: a bar chart. A variable that takes categorical values, like user type (e.g. guest, user) or location are clearly non-numeric, and so should use a bar chart. However, there are certain variable types that can be trickier to classify: those that take on discrete numeric values and those that take on time-based values.

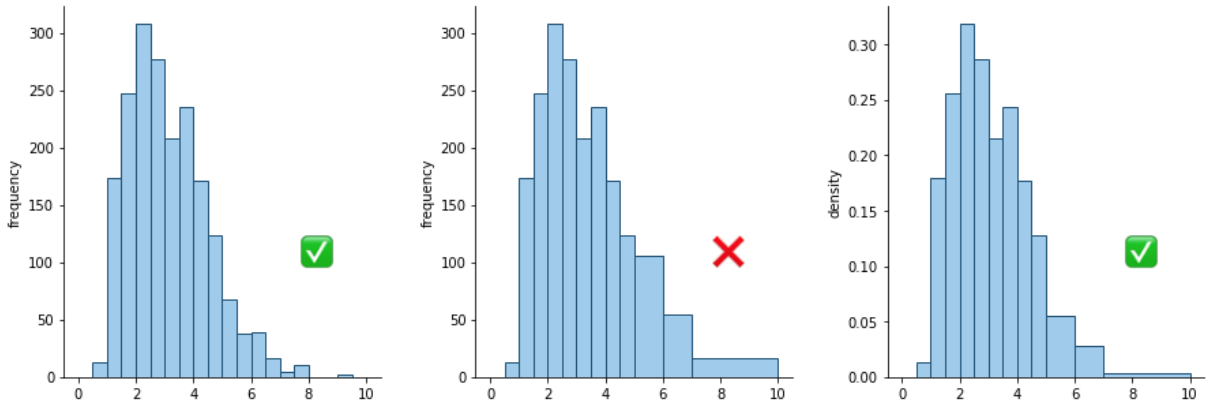


5.4.7 Using unequal bin sizes

When data is sparse, such as when there's a long data tail, the idea might come to mind to use larger bin widths to cover that space. However, creating a histogram with bins of unequal size is

not strictly a mistake, but doing so requires some major changes in how the histogram is created and can cause a lot of difficulties in interpretation.

The technical point about histograms is that the total area of the bars represents the whole, and the area occupied by each bar represents the proportion of the whole contained in each bin. When bin sizes are consistent, this makes measuring bar area and height equivalent. In a histogram with variable bin sizes, however, the height can no longer correspond with the total frequency of occurrences. Doing so would distort the perception of how many points are in each bin, since increasing a bin's size will only make it look bigger.

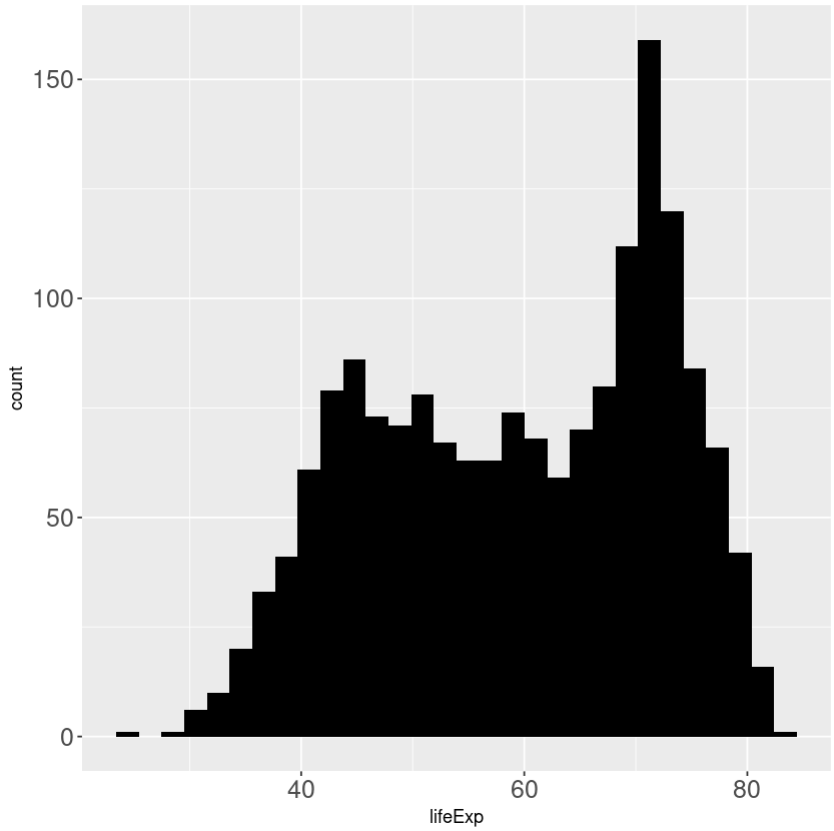


5.4.8 let us go back to the code in R!

In [246]:
```
library(ggplot2)
library(gapminder)
library(dplyr)


ggplot(gapminder, aes(x = lifeExp)) +
geom_histogram(fill = "black")+
theme(axis.text = element_text(size = 15))
```

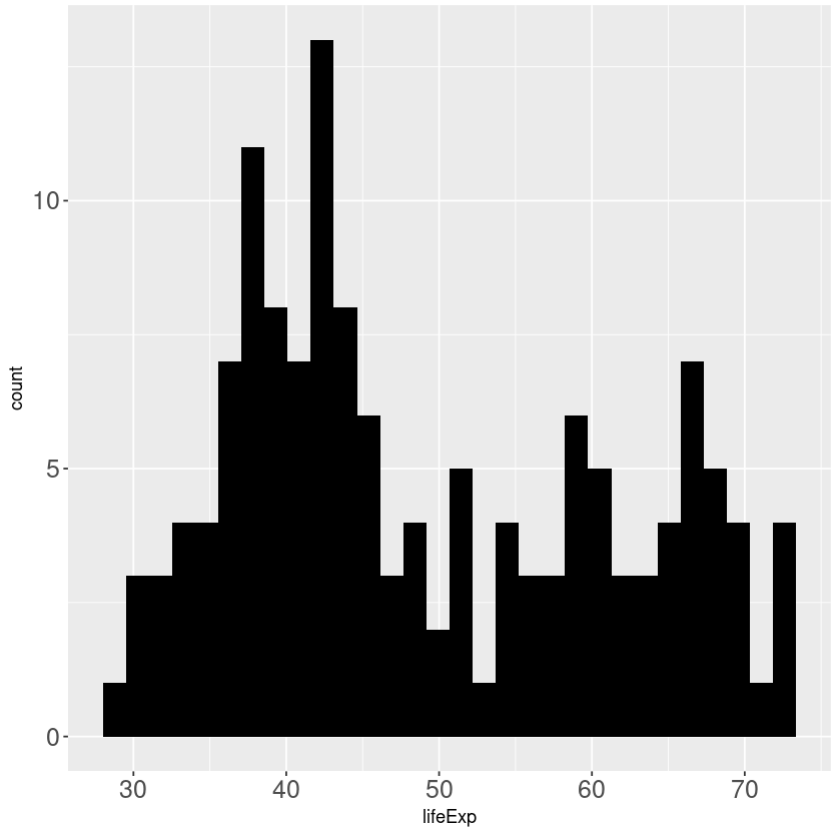
```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



In [247]:
```
gapminder_1952 <- gapminder %>%
  filter(year == 1952)

# Create a histogram of population (pop_by_mil)
ggplot(gapminder_1952, aes(x = lifeExp)) +
geom_histogram(fill = "black") +
theme(axis.text = element_text(size = 15))
```

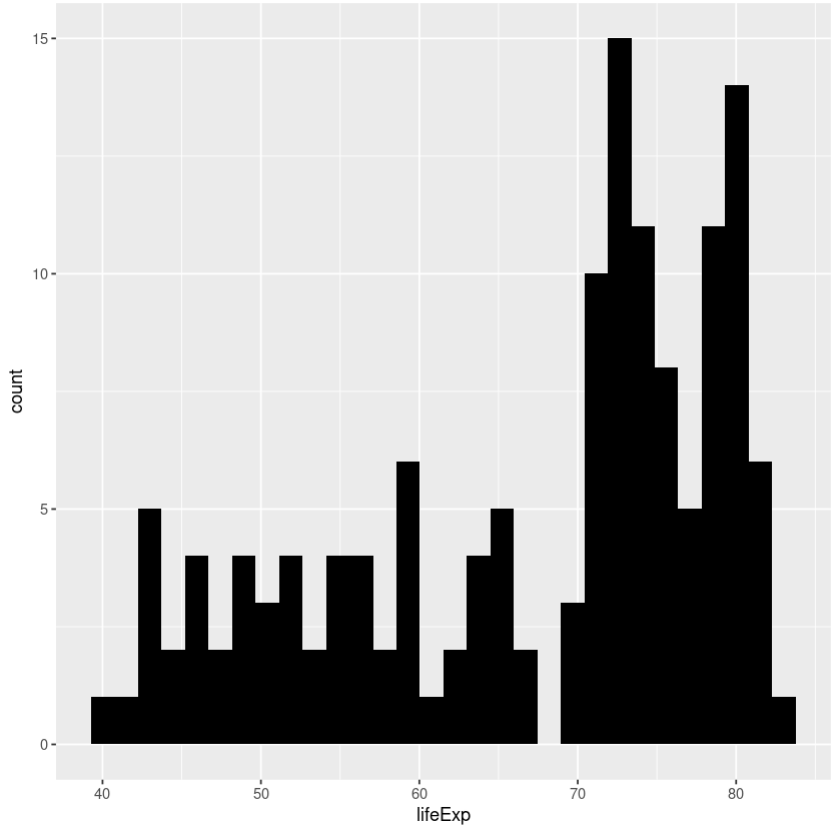
```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



In [234]:
```
gapminder_2007 <- gapminder %>%
  filter(year == 2007)

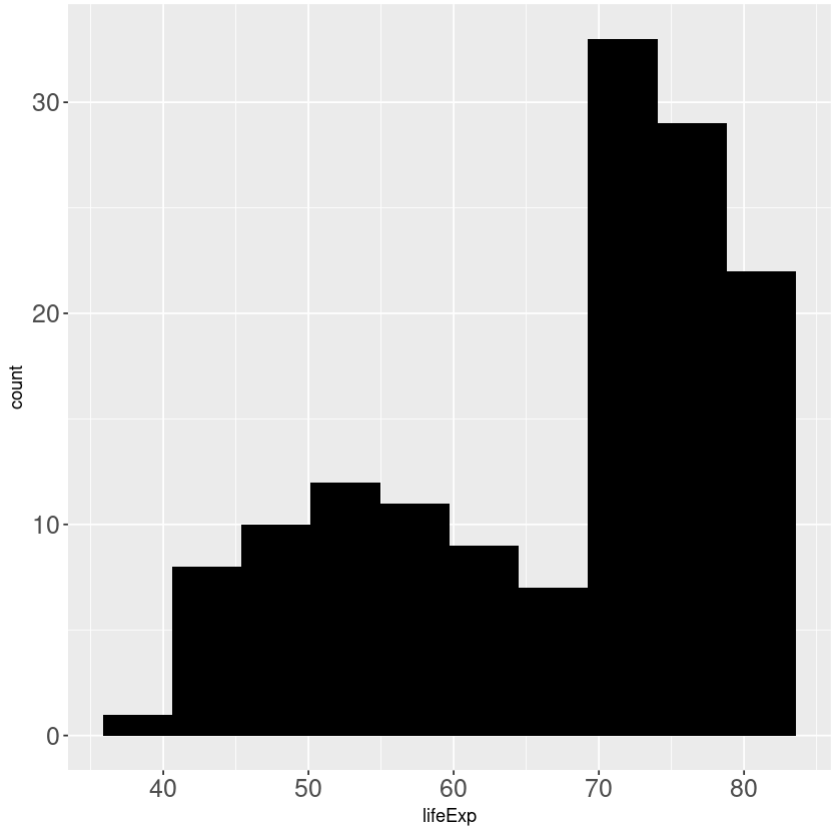
# Create a histogram of population (pop_by_mil)
ggplot(gapminder_2007, aes(x = lifeExp)) +
geom_histogram(fill = "black")
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



In [248]:
```
gapminder_2007 <- gapminder %>%
  filter(year == 2007)

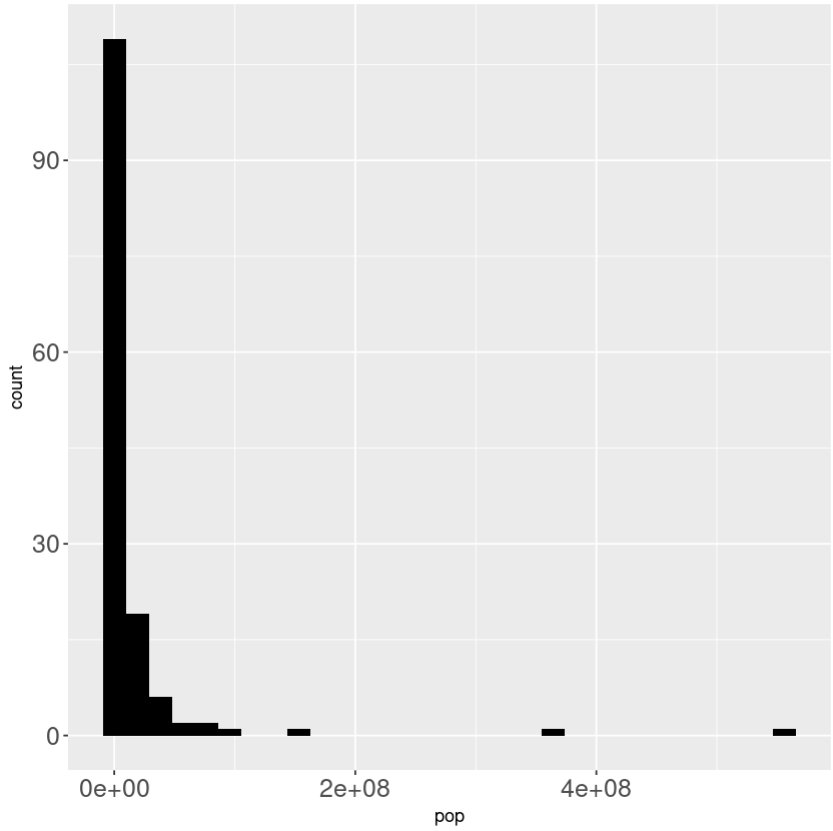
# Create a histogram of population (pop_by_mil)
ggplot(gapminder_2007, aes(x = lifeExp)) +
geom_histogram(bins = 10, fill = "black") +
theme(axis.text = element_text(size = 15))
```



In [249]:
```
gapminder_1952 <- gapminder %>%
  filter(year == 1952)

# Create a histogram of population (pop), with x on a log scale
ggplot(gapminder_1952, aes(x = pop)) +
geom_histogram(fill = "black") +
theme(axis.text = element_text(size = 15))
```

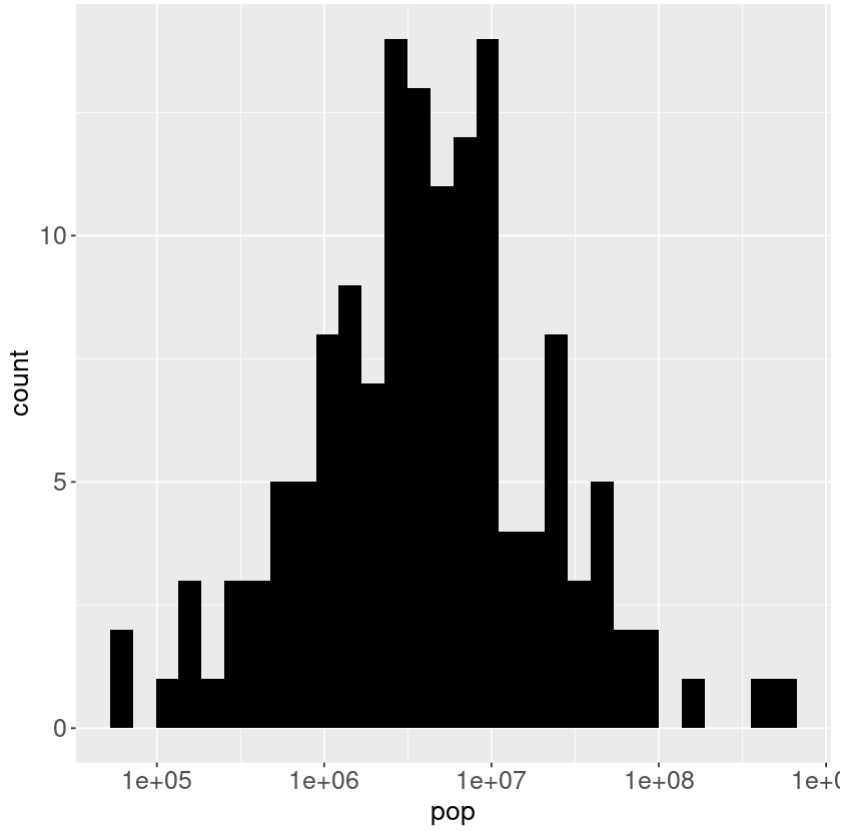
```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



In [259]:
```
gapminder_1952 <- gapminder %>%
  filter(year == 1952)

# Create a histogram of population (pop), with x on a log scale
ggplot(gapminder_1952, aes(x = pop)) +
geom_histogram(fill = "black")+
scale_x_log10() +
theme(axis.text = element_text(size = 15),
      axis.title.x = element_text(size = 16),
      axis.title.y = element_text(size = 16))
```

```
`stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



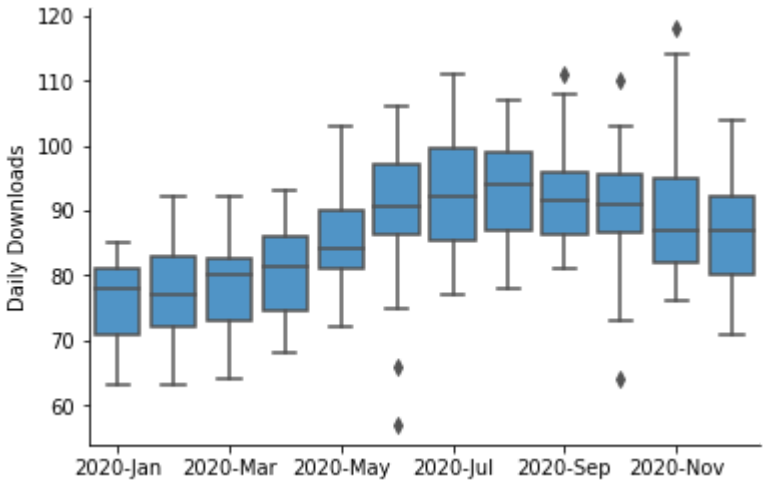
In [ ]:

5.5 Box plots

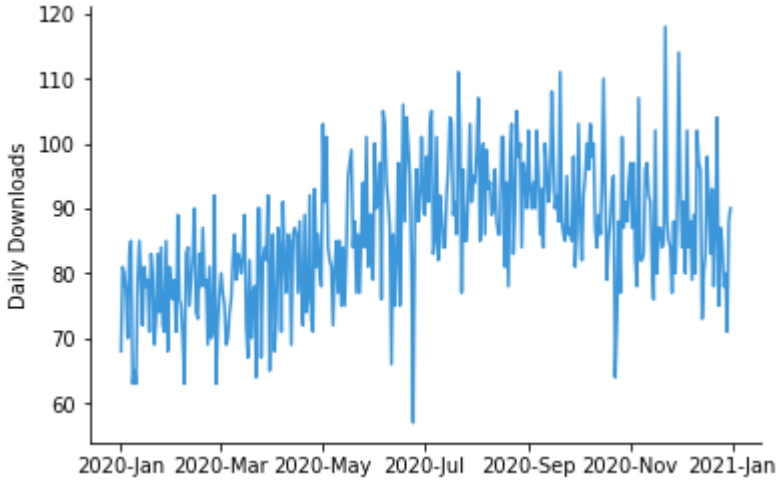
Box plots, which compare the distribution of a numeric variable among several categories.

5.5.1 What is a box plot?

A box plot (aka box and whisker plot) uses boxes and lines to depict the distributions of one or more groups of numeric data. Box limits indicate the range of the central 50% of the data, with a central line marking the median value. Lines extend from each box to capture the range of the remaining data, with dots placed past the line edges to indicate outliers.



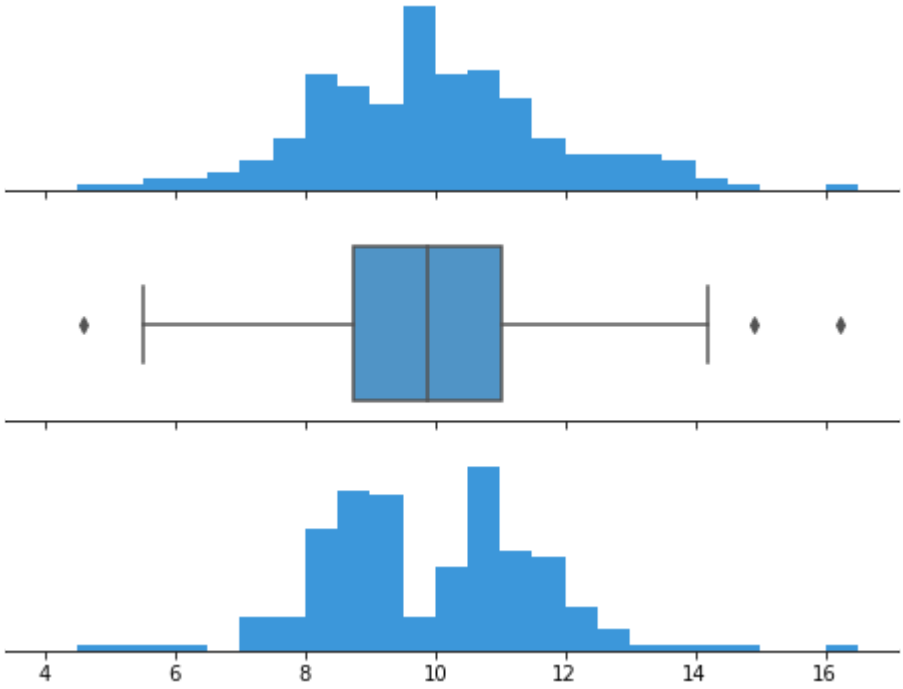
The example box plot above shows daily downloads for a fictional digital app, grouped together by month. From this plot, we can see that downloads increased gradually from about 75 per day in January to about 95 per day in August. There also appears to be a slight decrease in median downloads in November and December. Points show days with outlier download counts: there were two days in June and one day in October with low downloads compared to other days in the month. The box and whiskers plot provides a cleaner representation of the general trend of the data, compared to the equivalent line chart.



5.5.2 When you should use a box plot

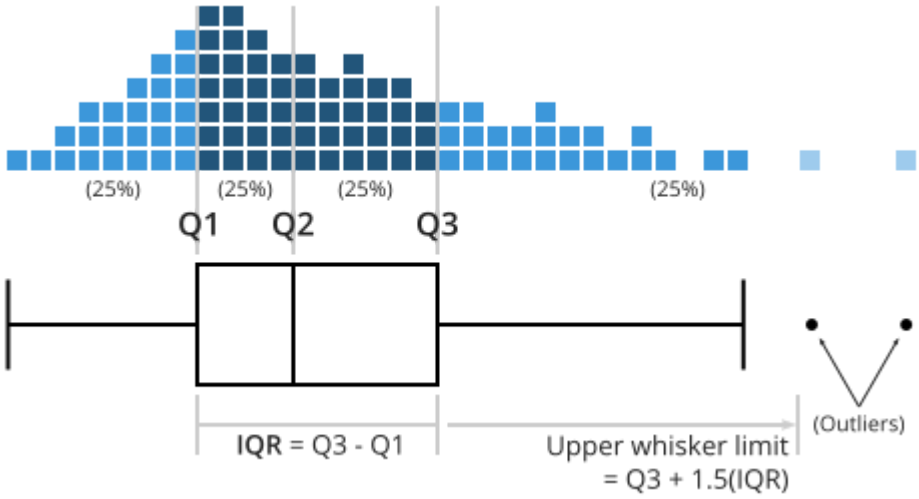
Box plots are used to show distributions of numeric data values, especially when you want to compare them between multiple groups. They are built to provide high-level information at a glance, offering general information about a group of data's symmetry, skew, variance, and outliers. It is easy to see where the main bulk of the data is, and make that comparison between different groups.

On the downside, a box plot's simplicity also sets limitations on the density of data that it can show. With a box plot, we miss out on the ability to observe the detailed shape of distribution, such as if there are oddities in a distribution's modality (number of 'humps' or peaks) and skew.



5.5.3 Interpreting a box and whiskers

Construction of a box plot is based around a dataset's quartiles, or the values that divide the dataset into equal fourths. The **first quartile (Q1)** is greater than 25% of the data and less than the other 75%. The **second quartile (Q2)** sits in the middle, dividing the data in half. Q2 is also known as the **median**. The **third quartile (Q3)** is larger than 75% of the data, and smaller than the remaining 25%. In a box and whiskers plot, the ends of the box and its center line mark the locations of these three quartiles.

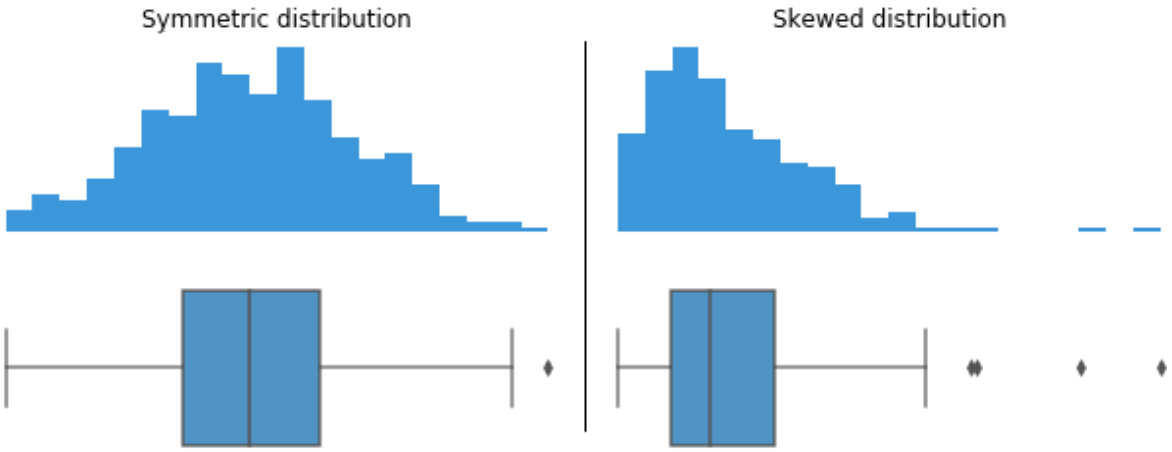


The distance between **Q3 and Q1** is known as the **interquartile range (IQR)** and plays a major part in how long the whiskers extending from the box are. Each whisker extends to the furthest data point in each wing that is within 1.5 times the IQR. Any data point further than that distance is considered an outlier, and is marked with a dot. There are other ways of defining the whisker lengths, which are discussed below.

https://muse.union.edu/dvorakt/what-drives-the-length-of-whiskers-in-a-box-plot/ (https://muse.union.edu/dvorakt/what-drives-the-length-of-whiskers-in-a-box-plot/)

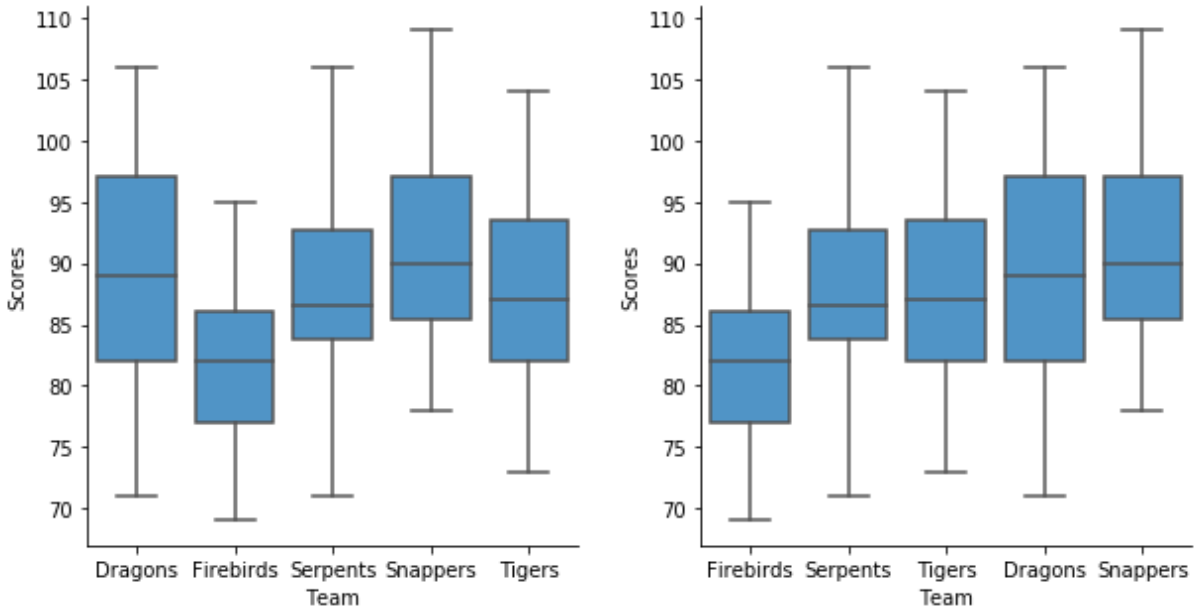
https://towardsdatascience.com/why-1-5-in-iqr-method-of-outlier-detection-5d07fdc82097 (https://towardsdatascience.com/why-1-5-in-iqr-method-of-outlier-detection-5d07fdc82097)

When a data distribution is symmetric, you can expect the median to be in the exact center of the box: the distance between Q1 and Q2 should be the same as between Q2 and Q3. Outliers should be evenly present on either side of the box. If a distribution is skewed, then the median will not be in the middle of the box, and instead off to the side. You may also find an imbalance in the whisker lengths, where one side is short with no outliers, and the other has a long tail with many more outliers.

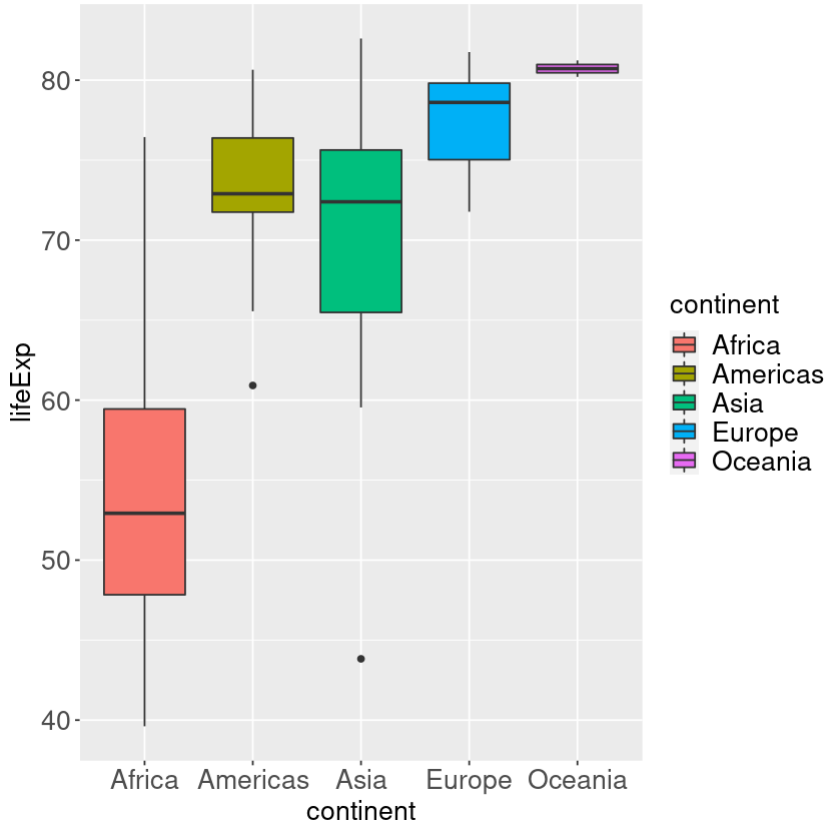


5.5.4 Consider the order of groups

If the groups plotted in a box plot do not have an inherent order, then you should consider arranging them in an order that highlights patterns and insights. One common ordering for groups is to sort them by median value.

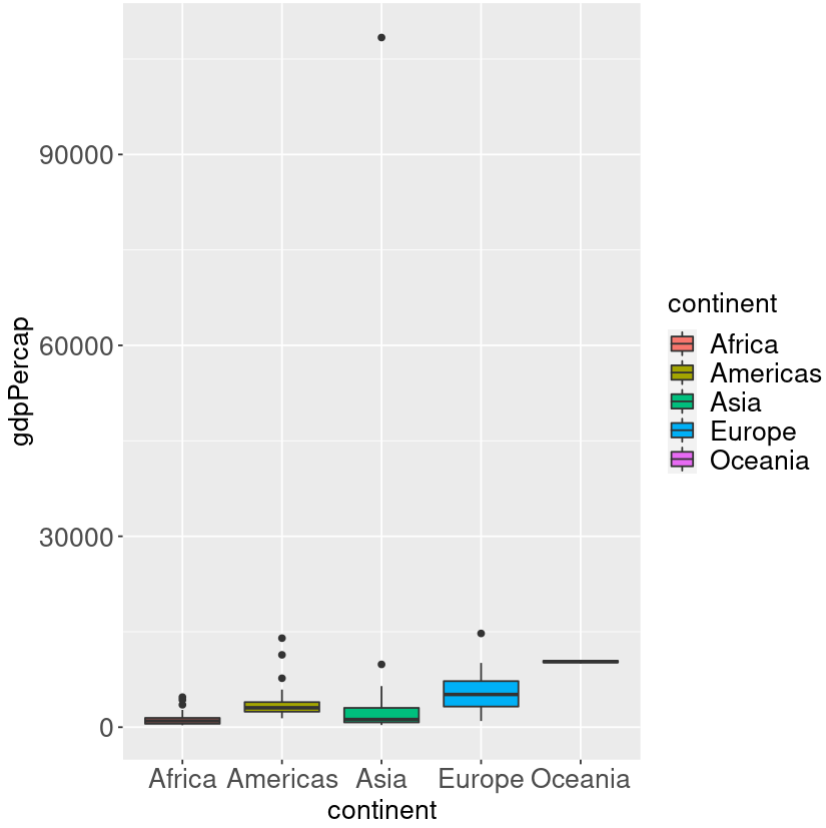


In [271]:
```
ggplot(gapminder_2007, aes(x = continent, y = lifeExp, fill = continent)) +
geom_boxplot() +
theme(axis.text = element_text(size = 16),
      axis.title.x = element_text(size = 16),
      axis.title.y = element_text(size = 16),
     legend.title = element_text(size = 16),
     legend.text = element_text(size = 16))
```



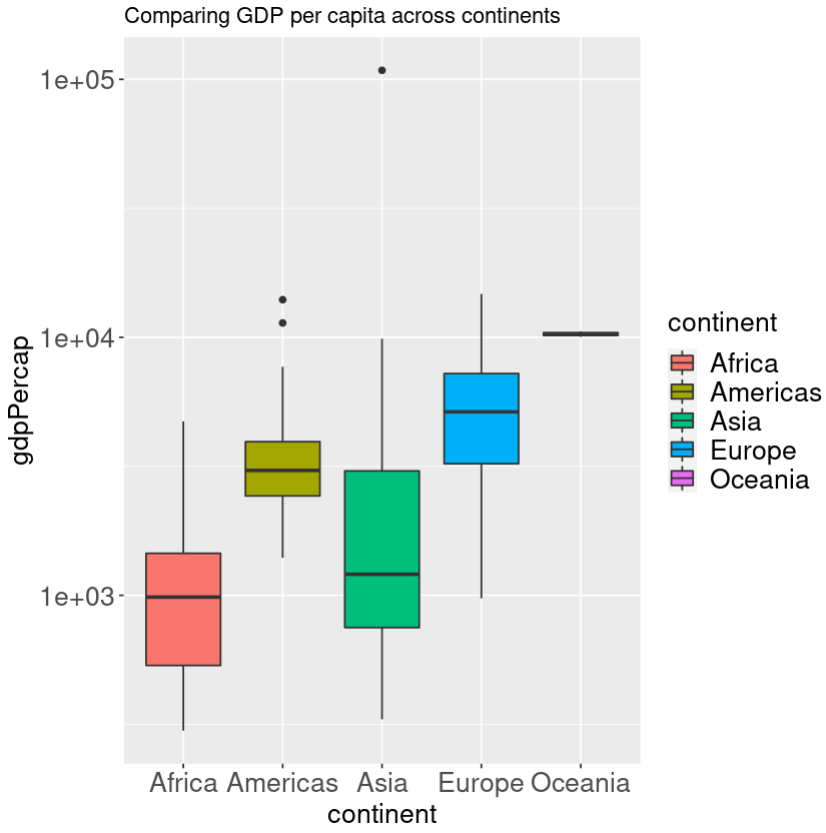
In [273]:
```
gapminder_1952 <- gapminder %>%
  filter(year == 1952)

# Create a boxplot comparing gdpPercap among continents
ggplot(gapminder_1952, aes(x = continent, y = gdpPercap, fill = continent)) +
geom_boxplot() +
theme(axis.text = element_text(size = 16),
      axis.title.x = element_text(size = 16),
      axis.title.y = element_text(size = 16),
     legend.title = element_text(size = 16),
     legend.text = element_text(size = 16))
```



In [275]:
```
gapminder_1952 <- gapminder %>%
  filter(year == 1952)

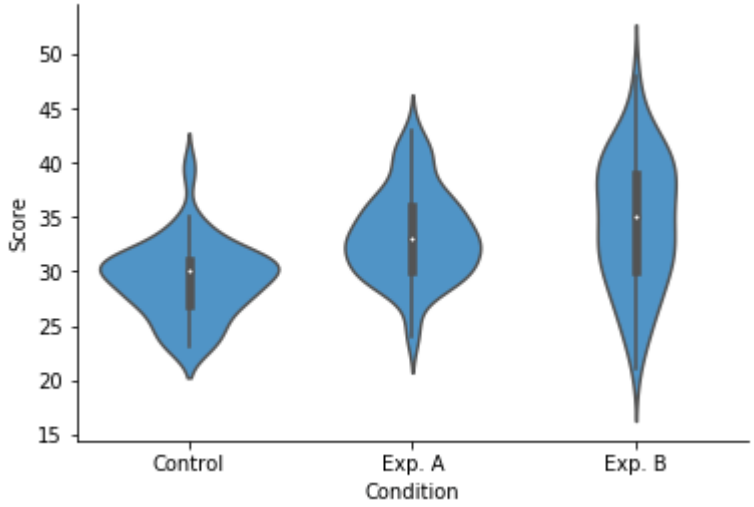
# Create a boxplot comparing gdpPercap among continents
ggplot(gapminder_1952, aes(x = continent, y = gdpPercap, fill = continent)) +
geom_boxplot() +
scale_y_log10() +
theme(axis.text = element_text(size = 16),
      axis.title.x = element_text(size = 16),
      axis.title.y = element_text(size = 16),
     legend.title = element_text(size = 16),
     legend.text = element_text(size = 16)) +
  labs(title = "Comparing GDP per capita across continents")
```



5.6 Violin plot

A violin plot depicts distributions of numeric data for one or more groups using density curves. The width of each curve corresponds with the approximate frequency of data points in each region. Densities are frequently accompanied by an overlaid chart type, such as box plot, to provide

additional information.



The example violin plot above depicts the results of a fictional experiment with one control group and two experimental conditions. In the middle of each density curve is a small box plot, with the rectangle showing the ends of the first and third quartiles and central dot the median. We can see from the plot that the two experimental techniques provided different benefits compared to the control. However, the second experimental condition (B) has a much more elongated distribution compared to the other two groups, without a distinct peak. The latter fact would have been missed with the box plot alone

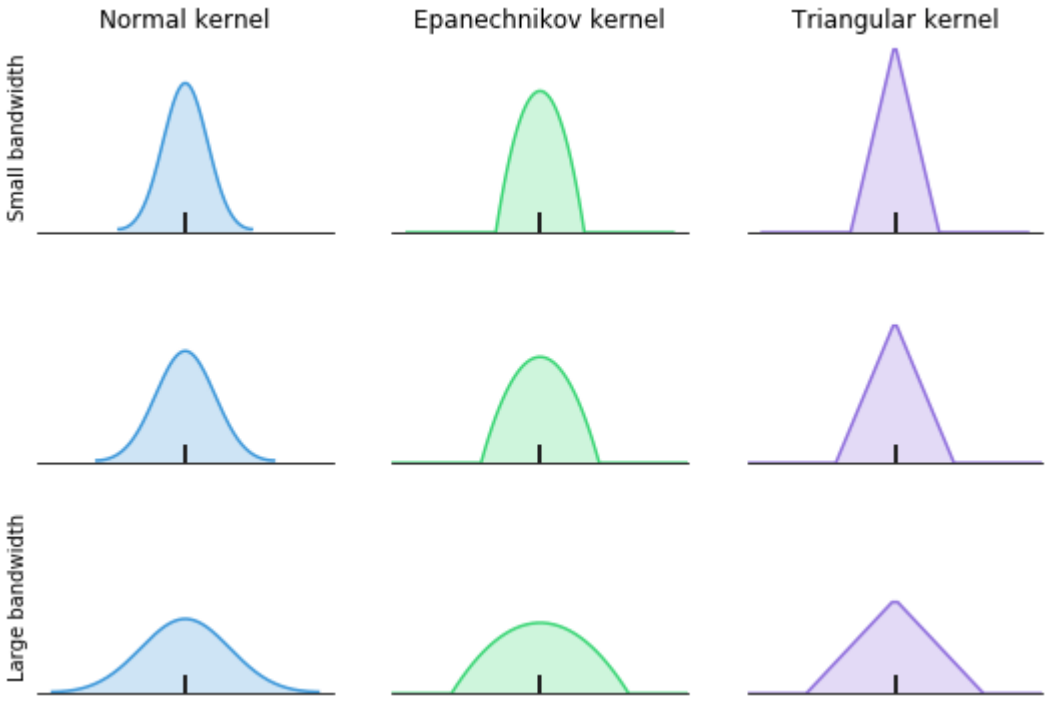
5.6.1 When you should use a violin plot

Violin plots are used when you want to observe the distribution of numeric data, and are especially useful when you want to make a comparison of distributions between multiple groups. The peaks, valleys, and tails of each group's density curve can be compared to see where groups are similar or different. Additional elements, like box plot quartiles, are often added to a violin plot to provide additional ways of comparing groups, and will be discussed below.

5.6.2 A brief explanation of density curves

The density curve, aka kernel density plot or kernel density estimate (KDE), is a less-frequently encountered depiction of data distribution, compared to the more common histogram. Below, we'll perform a brief explanation of how density curves are built.

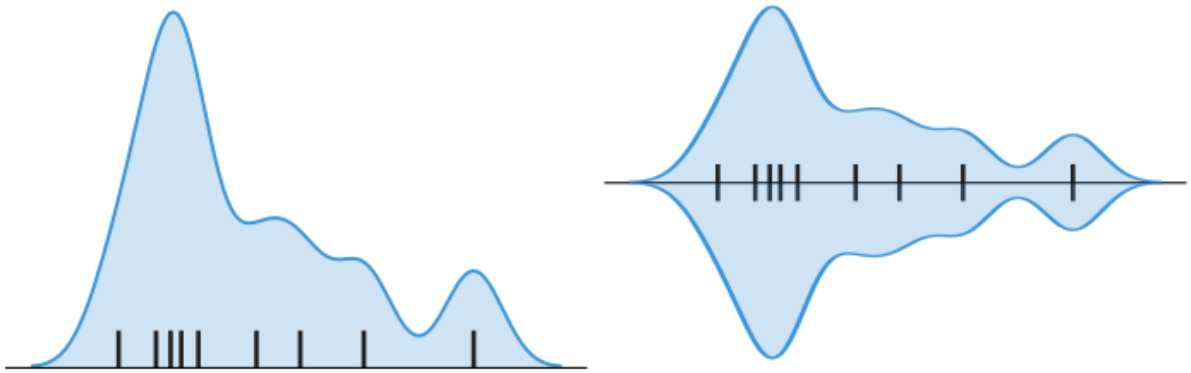
In a KDE, each data point contributes a small area around its true value. The shape of this area is called the kernel function. Kernels can take different shapes from smooth bell curves to sharp triangular peaks. In addition, kernels can have different width, or bandwidth, affecting the influence of each individual data point. Bandwidth size is usually determined by using mathematical rules of thumb, but can be tweaked depending on the shape and skew of the data to be plotted.



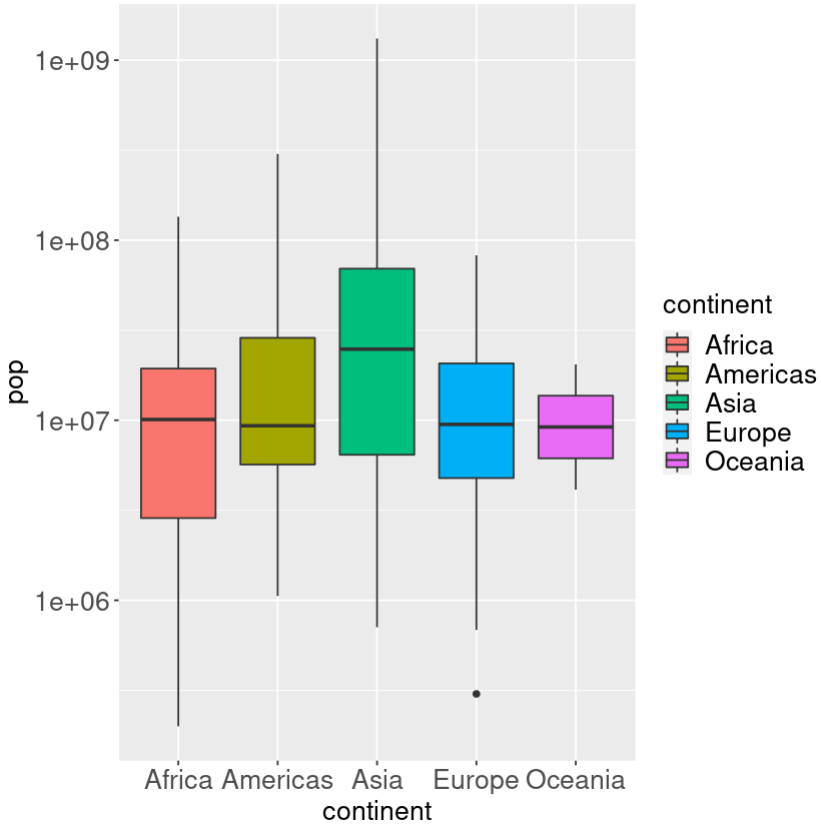
To build the final density curve, the areas for all data points are stacked up into a complete whole. Each data point has an equivalent influence on the final distribution. As there are more data points in a region, the height of the density curve in that area will increase.

Kernel density estimation is best used when a fair amount of data is available, resulting in more stable density estimates. With few data points available, it can be easy to be misled by the smoothness of the curve or the length of the tails past the largest and smallest points.

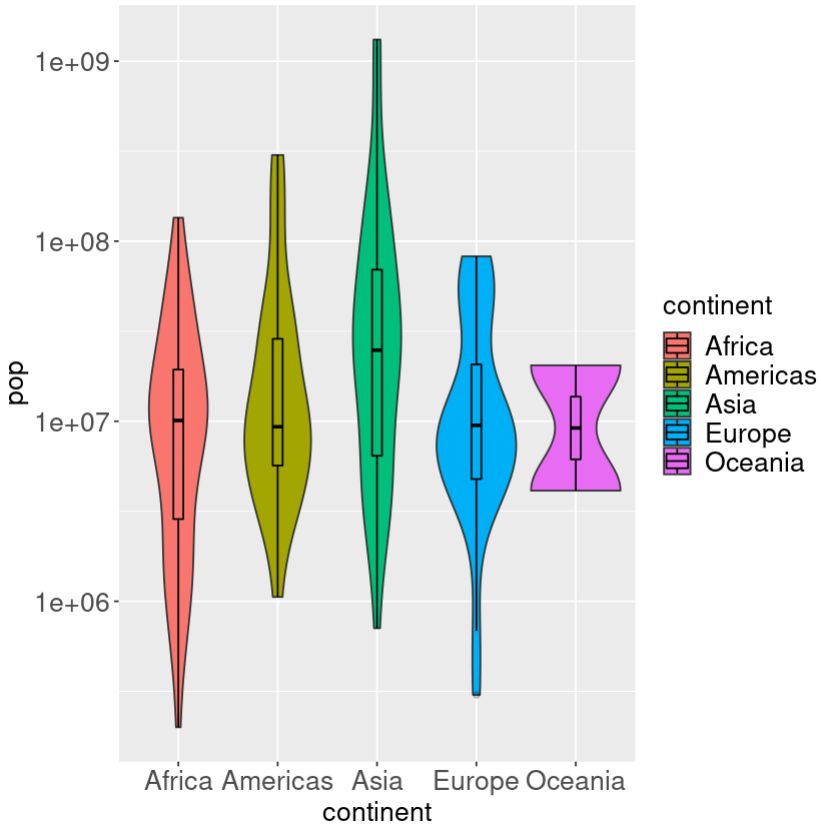
In a violin plot, individual density curves are built around center lines, rather than stacked on baselines. Other than this difference in display pattern, curves in a violin plot follow the exact same construction and interpretation.



In [289]:
```
ggplot(gapminder_2007, aes(x = continent, y = pop, fill = continent)) +
geom_boxplot() +
scale_y_log10() +
theme(axis.text = element_text(size = 16),
      axis.title.x = element_text(size = 16),
      axis.title.y = element_text(size = 16),
     legend.title = element_text(size = 16),
     legend.text = element_text(size = 16))
```



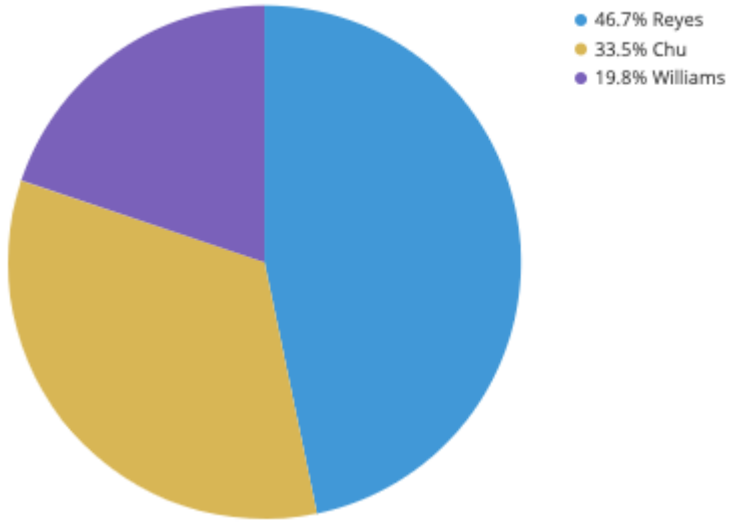
In [290]:
```
ggplot(gapminder_2007, aes(x = continent, y = pop, fill = continent))  +
geom_violin() +
scale_y_log10() +
geom_boxplot(width=0.1, color="black", alpha=0.2) +
theme(axis.text = element_text(size = 16),
      axis.title.x = element_text(size = 16),
      axis.title.y = element_text(size = 16),
     legend.title = element_text(size = 16),
     legend.text = element_text(size = 16))
```



5.7 Pie plots

5.7.1 What is a pie chart?

A pie chart shows how a total amount is divided between levels of a categorical variable as a circle divided into radial slices. Each categorical value corresponds with a single slice of the circle, and the size of each slice (both in area and arc length) indicates what proportion of the whole each category level takes.



The pie chart above depicts the distribution of votes for a fictional election for a small city. We can see that Reyes, represented by the first blue slice, has just less than half of the votes. Chu (yellow) is in second, with about a third of the votes, while Williams (purple) is last, with about a fifth of the votes. The annotations in the upper right give us a more precise judgment of the proportions, but the pie chart tells the overarching story of where the votes fell.

5.7.2 When you should use a pie chart

Pie charts have a fairly narrow use-case that is encapsulated particularly well by its definition. In order to use a pie chart, you must have some kind of whole amount that is divided into a number of distinct parts. Your primary objective in a pie chart should be to compare each group's contribution to the whole, as opposed to comparing groups to each other. If the above points are not satisfied, the pie chart is not appropriate, and a different plot type should be used instead.

The values that comprise a whole and the categories that divide the whole generally come in two major varieties. First of all, is when the 'whole' represents a total count. Examples of this include votes in an election divided by candidate, or number of transactions divided by user type (e.g. guest, new user, existing user).

Example of data structure

REGION	TOTAL REVENUE
North	491 064.51
East	283 445.43
South	128 753.87
West	263 391.13

Data for a pie chart can be summarized in a table like the above, where the first column indicates a category, and the second the proportion, frequency, or amount of that category. Usually, the total does not need to be specified separately unless it is to be listed somewhere else on a generated figure.

5.7.3 Best practices for using a pie chart

5.7.3.1 Include annotations

It is actually very difficult to discern exact proportions from pie charts, outside of small fractions like 1/2 (50%), 1/3 (33%), and 1/4 (25%). Furthermore, if the slice values are meant to depict amounts rather than proportions, pie charts typically lack the tick marks to allow for estimation of values directly from slice sizes. It is for these reasons that annotations are a standard inclusion for pie charts.

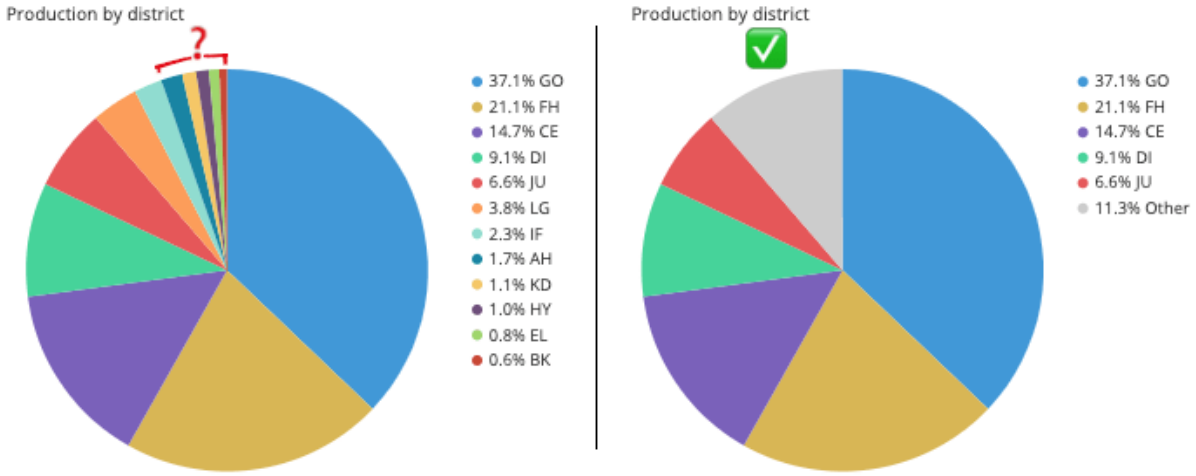
5.7.3.2 Consider the order of slices

A good order for slices can make it much easier for a reader to understand what the plot is saying. A typical ordering goes from the largest slice to the smallest slice, very useful when there are categories with very similar values. However, if the category levels have an inherent ordering, then plotting slices in that order is usually better.

5.7.3.3 Limit the number of pie slices

Pie charts with a large number of slices can be difficult to read. It can be difficult to see the

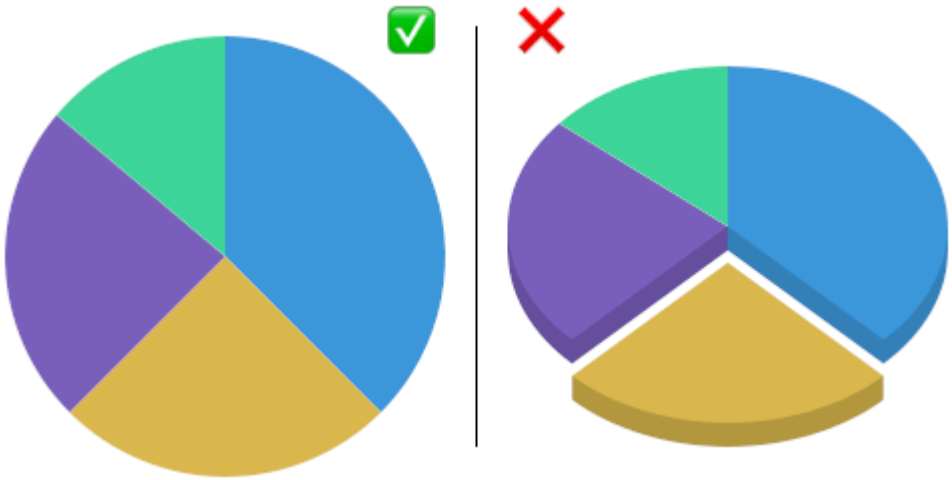
smallest slices, and it can be difficult to choose enough colors to make all of the slices distinct. Recommendations vary, but if you have more than about five categories, you might want to think about using a different chart type. As another option, you might consider lumping small slices into a single 'other' slice, colored in a neutral gray.



5.7.3.4 Avoid distorting effects

Reading a pie chart accurately requires that the slices' areas, arc lengths, and angles all point to an accurate representation of the data. While avoiding 3-d effects is a good idea for any plot, it is especially important for pie charts. Squashing or stretching the circle or adding unnecessary depth can easily distort how large each slice compares to the whole.

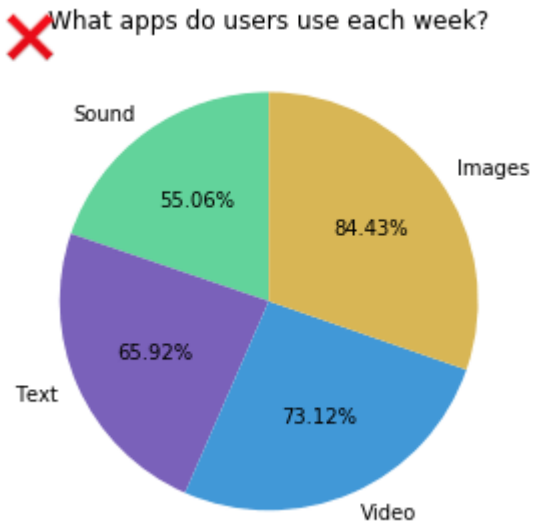
Another distortion can come from the 'exploded' pie chart, where slices are pulled out from the center for emphasis. This emphasis comes with a cost, where the gaps can make it more difficult to actually gauge the part-to-whole comparison.



5.7.3.5 Fitting a pie to incompatible data

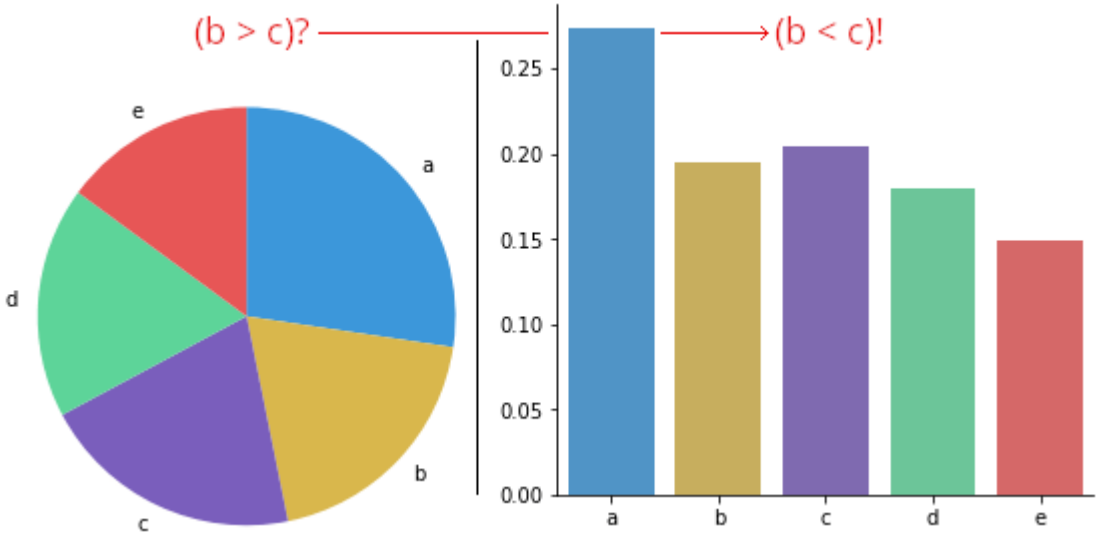
One of the most common mistakes with using a pie chart is to fit it to data that does not represent a parts-to-whole comparison. This confusion occurs most often when the values to be plotted are

percentages or proportions, but don't comprise a complete whole. The example below shows how frequently the people surveyed used each of four applications, but since many people used multiple apps, the proportions sum to much more than 100%.



5.7.3.6 Using pie charts to compare groups to one another

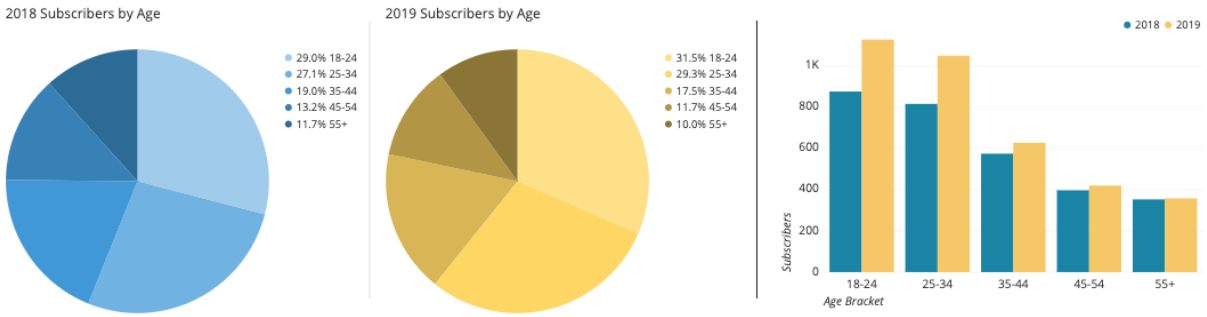
If you want to make comparisons between groups rather than from each group to the whole, then you are better off with a different chart type. Even when sorting slices by size, it can be difficult to tell how different two slices are, especially when they move away from the start / end point. In the below example, you might assume that the second slice is larger than the third due to order, but the corresponding bar chart actually shows the opposite. The main thing that you can really say from the pie chart is that both slices are approximately the same proportions from the whole.



5.7.3.7 Comparing values across multiple pie charts

here may be cases where you will want to compare multiple pies to one another: for example, comparing user demographic distribution across multiple years. However, this runs into a similar

issue as the previous section, where you want to compare groups to one another. Even worse, it's a comparison between pies, so you can't rely on the order of slices as easily for the comparison. Expressing the data using a different plot, like stacked bar chart, grouped bar chart, or line chart, is often a better choice when this kind of group-to-group comparison is desired.

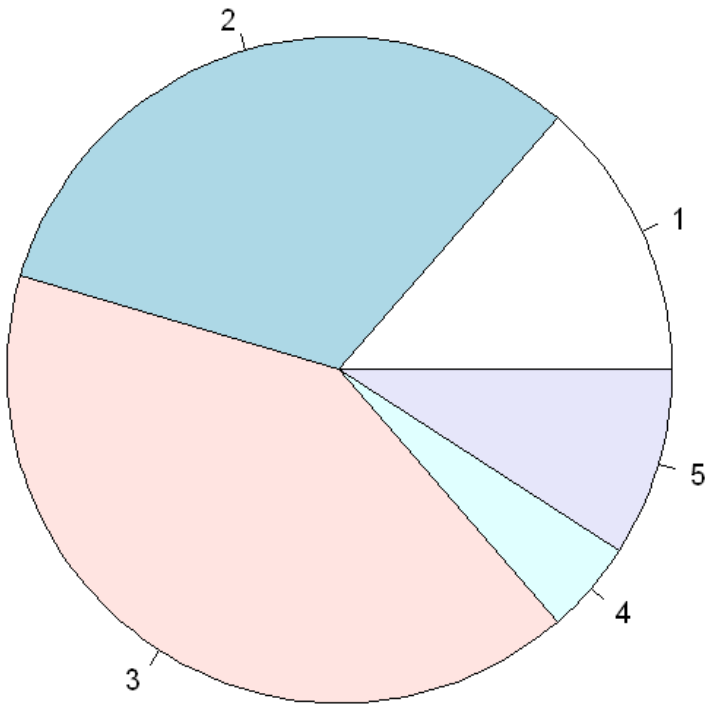


Now it is your turn to put into practice the lessons we have been learning in this course. Using the gapminder dataset, select two variables to display as a pie chart

a clue:

In [1]:
```
# Create Data
Prop <- c(3,7,9,1,2)

# Make the default Pie Plot
pie(Prop)
```



There is no specific **geom** to build piechart with **ggplot2**. The trick is to build a barplot and use **coord_polar** to make it circular. This is why the pie() function described above is probably a better alternative.

In [9]:
```
# Load ggplot2
library(ggplot2)

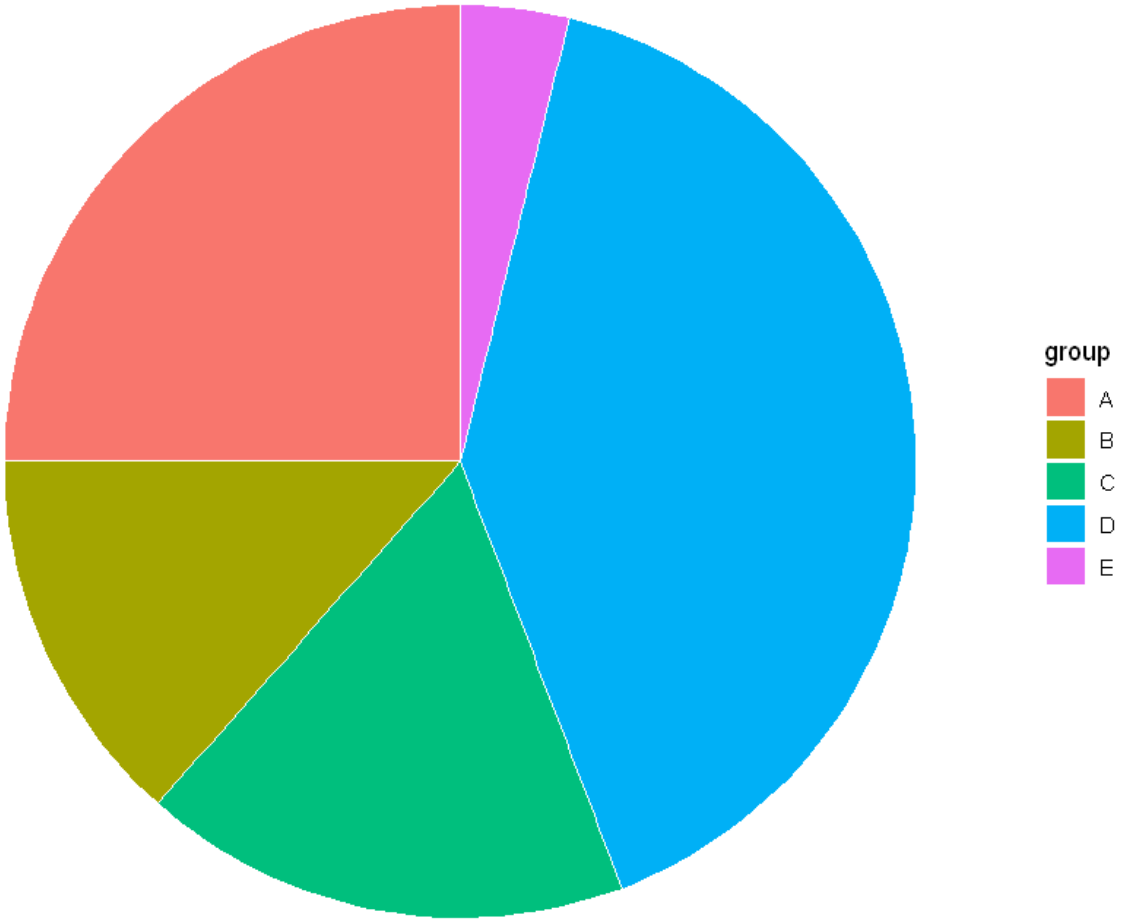
# Create Data
data <- data.frame(
  group=LETTERS[1:5],
  value=c(13,7,9,21,2)
)

data

# Basic piechart
ggplot(data, aes(x="", y=value, fill=group)) +
  geom_bar(stat="identity", width=1, color="white") +
  coord_polar("y", start=0) +

  theme_void() # remove background, grid, numeric labels
```

group	value
A	13
B	7
C	9
D	21
E	2


group
A
B
C
D
E

In [15]:
```
# Load ggplot2
library(ggplot2)
library(dplyr)

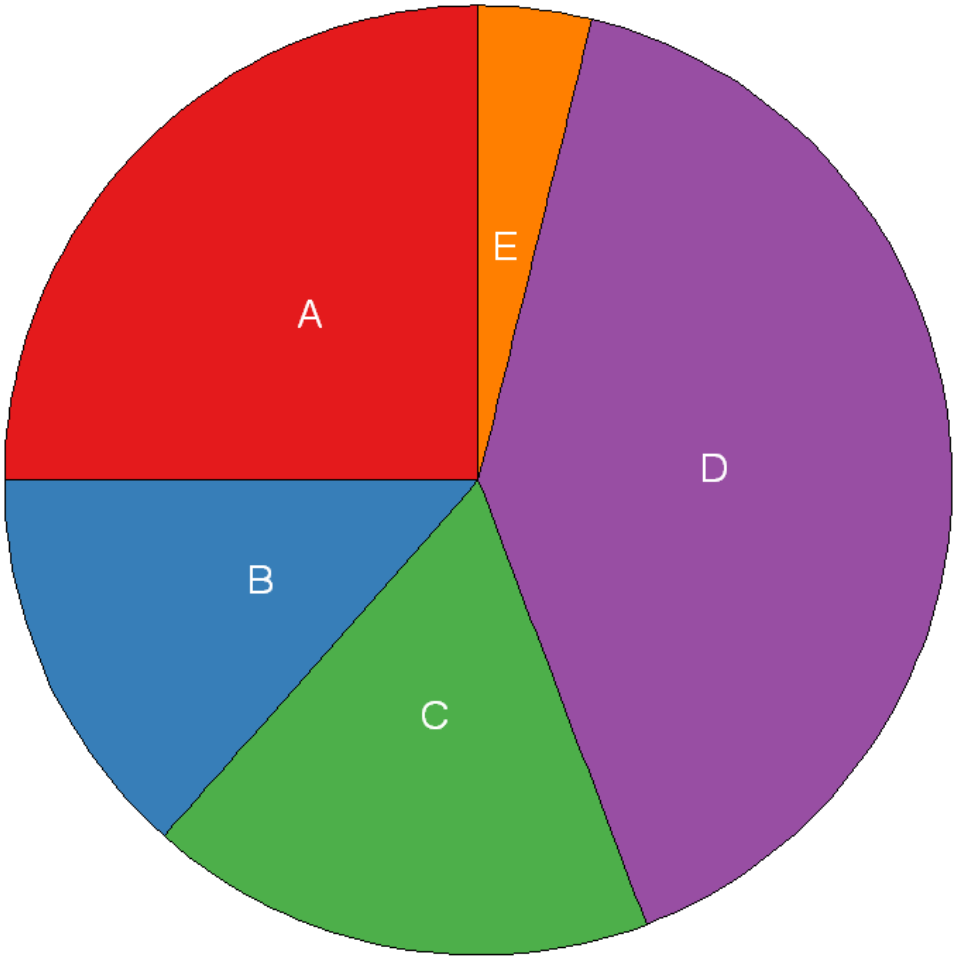
# Create Data
data <- data.frame(
  group=LETTERS[1:5],
  value=c(13,7,9,21,2)
)

# Compute the position of labels
data <- data %>%
  arrange(desc(group)) %>%
  mutate(prop = value / sum(data$value) *100) %>%
  mutate(ypos = cumsum(prop)- 0.5*prop )

data
# Basic piechart
ggplot(data, aes(x="", y=prop, fill=group)) +
  geom_bar(stat="identity", width=1, color="black") +
  coord_polar("y", start=0) +
  theme_void() +
  theme(legend.position="none") +

  geom_text(aes(y = ypos, label = group), color = "white", size=6) +
  scale_fill_brewer(palette="Set1")
```

group	value	prop	ypos
E	2	3.846154	1.923077
D	21	40.384615	24.038462
C	9	17.307692	52.884615
B	7	13.461538	68.269231
A	13	25.000000	87.500000


A
B
C
D
E

In [17]:
```
library(ggplot2)

# Create test data.
data <- data.frame(
  category=c("A", "B", "C"),
  count=c(10, 60, 30)
)

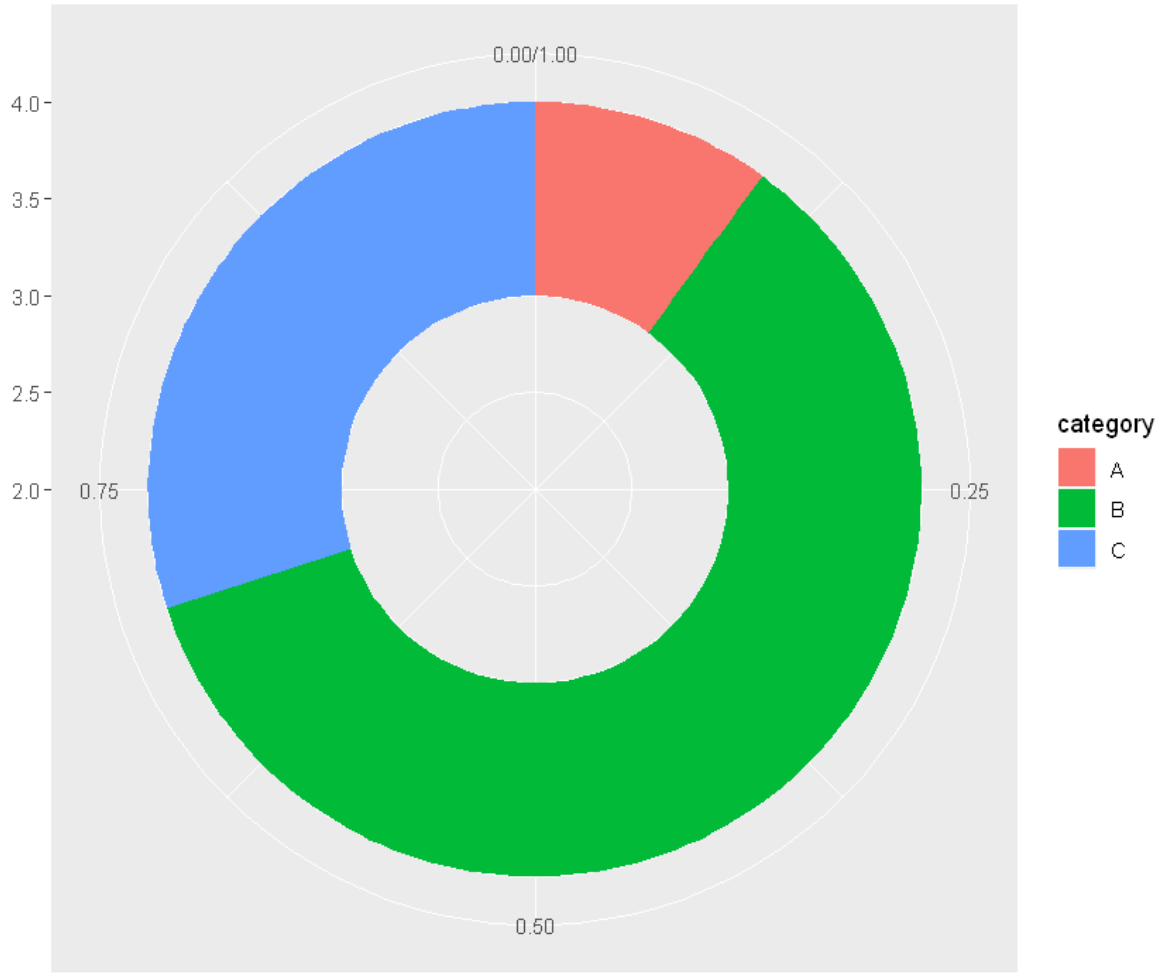
# Compute percentages
data$fraction = data$count / sum(data$count)

# Compute the cumulative percentages (top of each rectangle)
data$ymax = cumsum(data$fraction)

# Compute the bottom of each rectangle
data$ymin = c(0, head(data$ymax, n=-1))

# Make the plot
ggplot(data, aes(ymax=ymax, ymin=ymin, xmax=4, xmin=3, fill=category)) +
     geom_rect() +
     coord_polar(theta="y") + # Try to remove that to understand how the chart is
     xlim(c(2, 4)) # Try to remove that to see how to make a pie chart

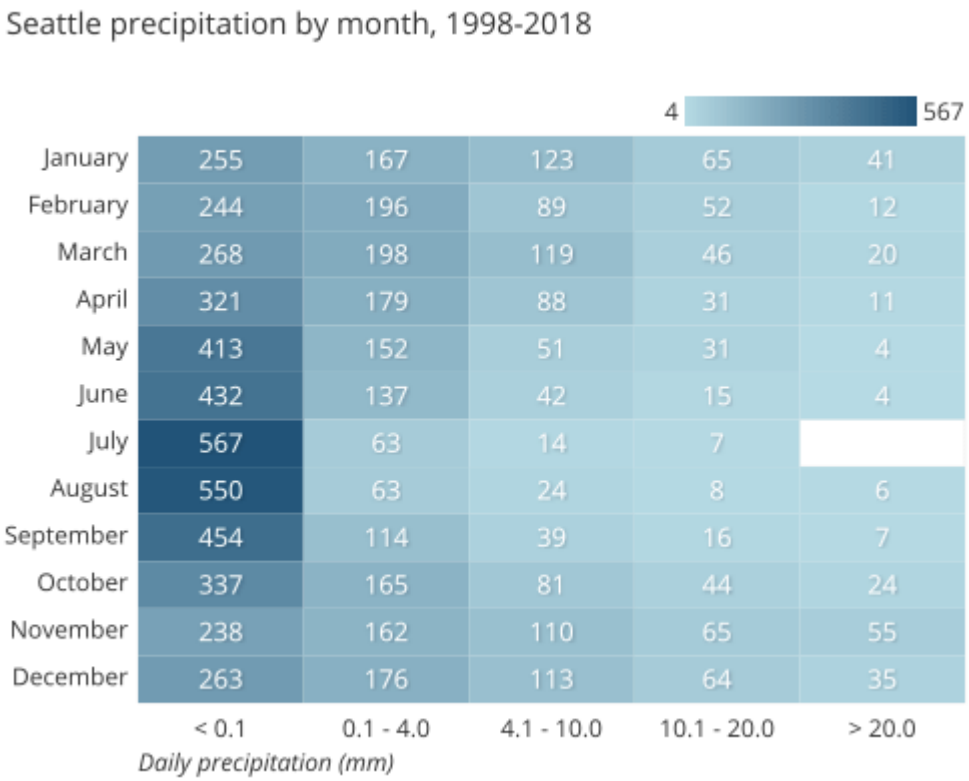
#https://r-graph-gallery.com/128-ring-or-donut-plot.html
```



5.8 Heatmap

5.8.1 What is a heatmap?

A heatmap (aka heat map) depicts values for a main variable of interest across two axis variables as a grid of colored squares. The axis variables are divided into ranges like a bar chart or histogram, and each cell's color indicates the value of the main variable in the corresponding cell range.



The example heatmap above depicts the daily precipitation distribution, grouped by month, and recorded over eleven years in Seattle, Washington. Each cell reports a numeric count, like in a standard data table, but the count is accompanied by a color, with larger counts associated with darker colorings. From the heat map, we can see from the darkest colorings in the left-most column that most days had no precipitation across the entire year. The pattern in cell colors across months also shows that rain is more common in the winter from November to March, and least common in the summer months of July and August.

5.8.2 2-d density plots

The term heatmap is also used in a more general sense, where data is not constrained to a grid. For example, tracking tools for websites can be set up to see how users interact with the site, like studying where a user clicks, or how far down a page readers tend to scroll.

Every click (or other tracking event) is associated with a position, which radiates a small amount of numeric value around its location. These values are totaled together across all events and then plotted with an associated colormap. The visual language of these tools' output, associating value with color, is similar to the type of heatmap defined at the top, just without a grid-based structure. Heatmaps of this type are sometimes also known as 2-d density plots.



5.8.3 When you should use a heatmap

Heatmaps are used to show relationships between two variables, one plotted on each axis. By observing how cell colors change across each axis, you can observe if there are any patterns in value for one or both variables.

The variables plotted on each axis can be of any type, whether they take on categorical labels or numeric values. In the latter case, the numeric value must be binned like in a histogram in order to form the grid cells where colors associated with the main variable of interest will be plotted.

Cell colorings can correspond to all manner of metrics, like a frequency count of points in each bin, or summary statistics like mean or median for a third variable. One way of thinking of the construction of a heatmap is as a table or matrix, with color encoding on top of the cells. In certain applications, it is also possible for cells to be colored based on non-numeric values (e.g. general qualitative levels of low, medium, high).

Example of data structure

MONTH	< 0.01	0.1 - 4.0	4.1 - 10.0	...
January	255	167	123	...
February	244	196	89	...
March	268	198	119	...
April	321	179	88	...
...	...	...	...	...

Different visualization applications can have different ways of accepting data for plotting as a heatmap. In one major form, data can be supplied in the same way it would be naturally displayed as a table. The first column will hold values for one axis of the heatmap, while the names of the remaining columns will correspond with bins for the remaining axis. Values in those columns will be encoded into the heatmap itself.

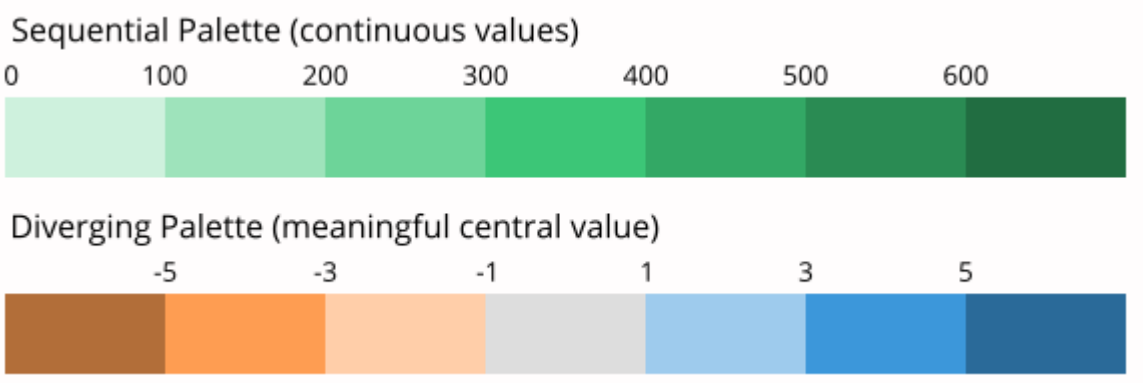
The other common form for heatmap data sets it up in a three-column format. Each cell in the heatmap is associated with one row in the data table. The first two columns specify the ‘coordinates’ of the heat map cell, while the third column indicates the cell’s value.

MONTH	PRCP_BUCKET	COUNT
March	10.1 - 20.0	46
March	> 20.0	20
April	< 0.1	321
April	0.1 - 4.0	179
...	...	...

5.8.4 Best practices for using a heatmap

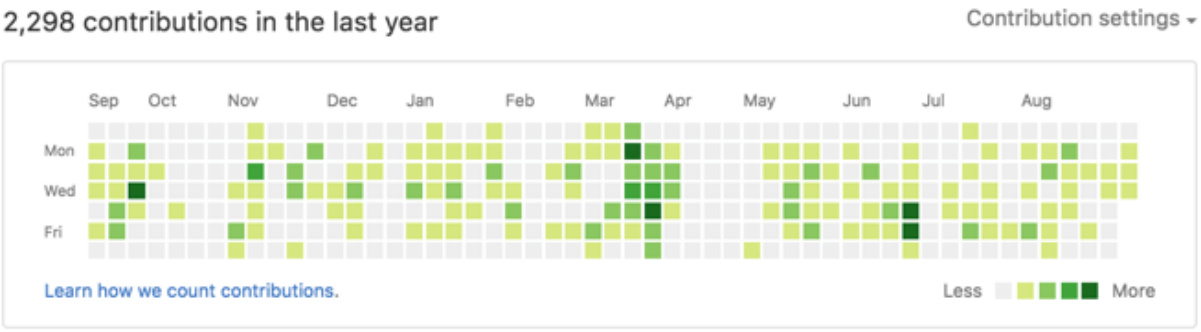
5.8.4.1 Choose an appropriate color palette

Color is a core component of this chart type, so it's worth making sure that you choose an appropriate color palette to match the data. Most frequently, there will be a sequential color ramp between value and color, where lighter colors correspond to smaller values and darker colors to larger values, or vice versa. However, a diverging color palette may be used when values have a meaningful zero point.



5.8.4.2 Include a legend

As an associated note, it is generally required for a heatmap to include a legend for how colors map to numeric values. Since color on its own has no inherent association with value, a key is vital for viewers to grasp the values in a heatmap. An exception for including a legend can come when the absolute association of value to color is not important, only the relative patterns of data plotted.

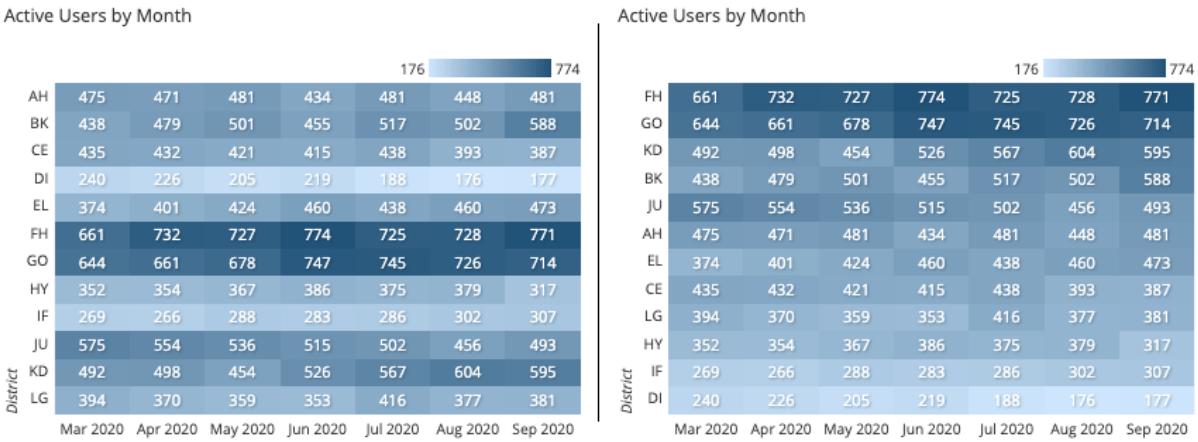


5.8.4.3 Show values in cells

There is a lack of precision for mapping color to value, especially compared to other encodings like position or length. Where possible, it is a good idea to add cell value annotations to the heatmap as a double encoding of value.

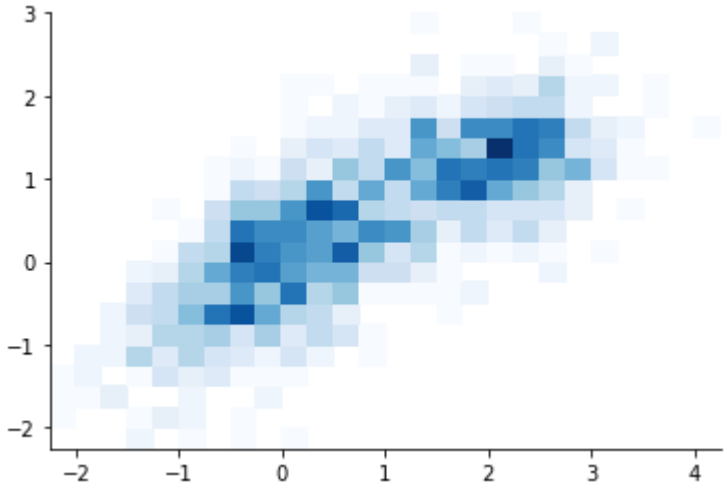
5.8.4.4 Sort levels by similarity or value

When one or both axis variables in a plot are categorical in nature, it can be worth considering changing the order in which those axis variable levels are plotted. If the categories do not have an inherent ordering, we might want to choose an order that best helps the reader grasp patterns in the data. A common option is to sort categories by their average cell value from largest to smallest.



5.8.4.5 Select useful tick marks

For numeric axis variables, choices can be made in how bins are set up and how they are indicated in the chart. If there are few bins, it is fine to keep tick marks on each bin like for a categorical axis variable. However, when there are a lot of bins, a better option is to plot tick marks between sets of bins to avoid overcrowding. The number of bins that you should use and how large they are will depend on the nature of the data, so it can be a good idea to experiment with different settings.

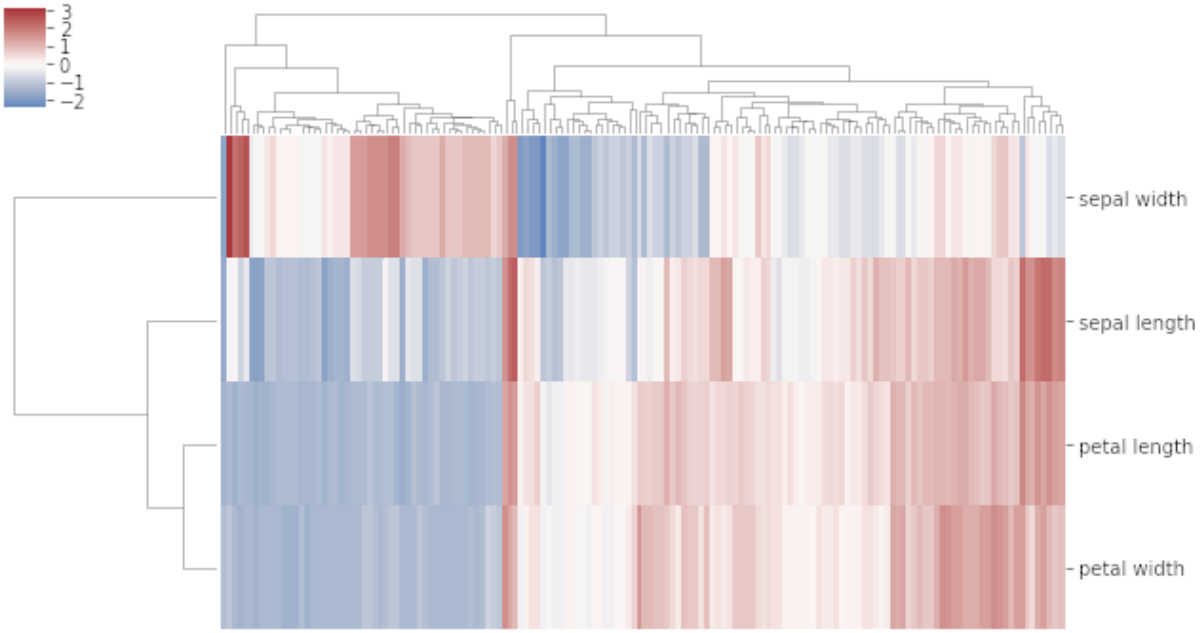


5.8.5 Common heatmap options

5.8.5.1 Clustered heatmap

Instead of having the horizontal axis represent levels or values of a single variable, it is a common variation to have it represent measurements of different variables or metrics. If we set the vertical axis as individual observations, we end up with something resembling a standard data table, where each row is an observation and the columns the entity's value on each measured variable.

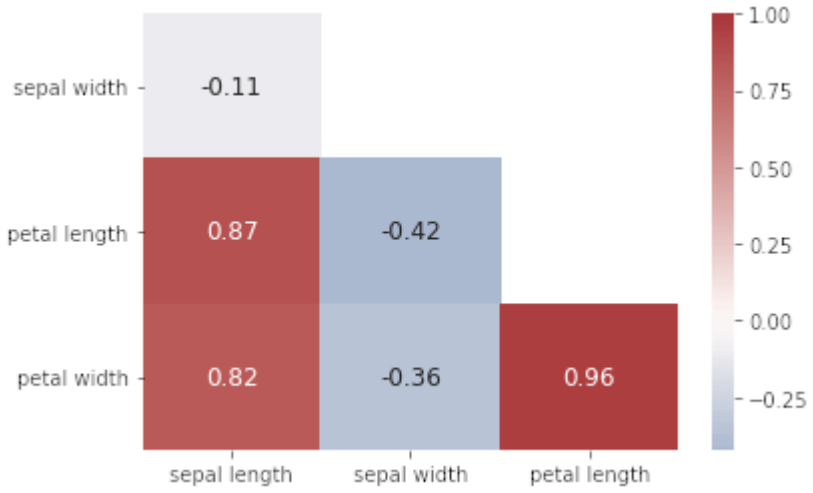
This type of heatmap is sometimes known as a clustered or clustering heatmap, since the goal of this kind of chart is to build associations between both the data points and their features. We want to see which individuals are similar or different from each other, with a similar objective for variables. Analysis tools that construct this type of heatmap will usually implement clustering as part of their process. This use case is found in areas like the biological sciences, such as when studying similarities in gene expression across individuals.



5.8.6 Correlogram

A correlogram is a variant of the heatmap that replaces each of the variables on the two axes with a list of numeric variables in the dataset. Each cell depicts the relationship between the intersecting variables, such as a linear correlation.

Correlograms are often seen in an exploratory role, helping analysts understand relationships between variables in service of building descriptive or predictive statistical models.



Now it is your turn to put into practice the lessons we have been learning in this course. Using the gapminder dataset, select variables to display as a heatmap

a clue:

The heatmap() function is natively provided in R. It produces high quality matrix and offers statistical tools to normalize input data, run clustering algorithm and visualize the result with dendrograms. It is one of the very rare case where I prefer base R to ggplot2.

ggplot2 allows to build heatmaps thanks to geom_tile(). However, I personally prefer the heatmap() function above since only it offers option for normalization, clustering and Dendrogram.

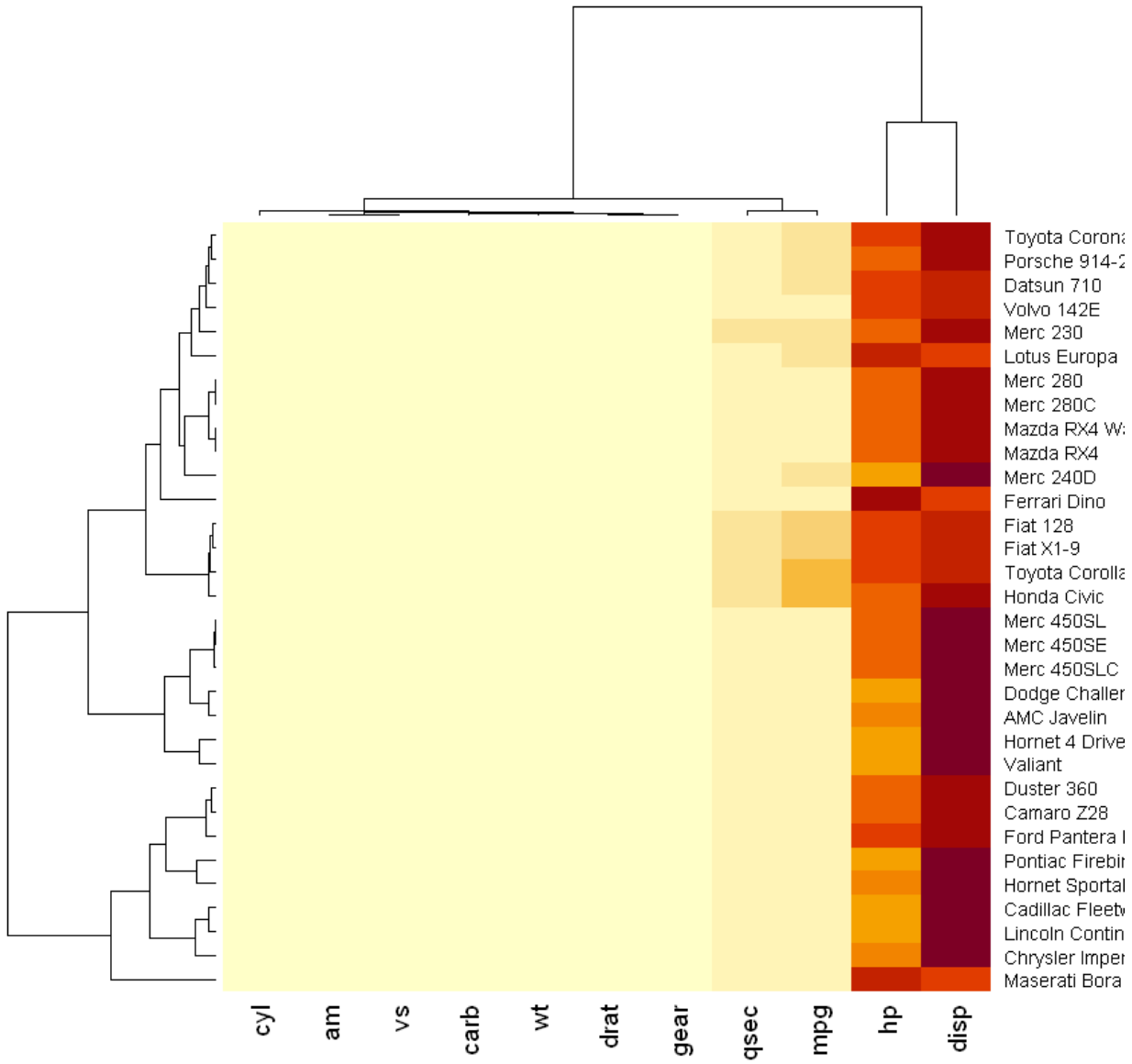
In [22]:
```
# The mtcars dataset:
data <- as.matrix(mtcars)

data

# Default Heatmap
heatmap(data)
```

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4
Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Ford Pantera L	15.8	8	351.0	264	4.22	3.170	14.50	0	1	5	4
Ferrari Dino	19.7	6	145.0	175	3.62	2.770	15.50	0	1	5	6

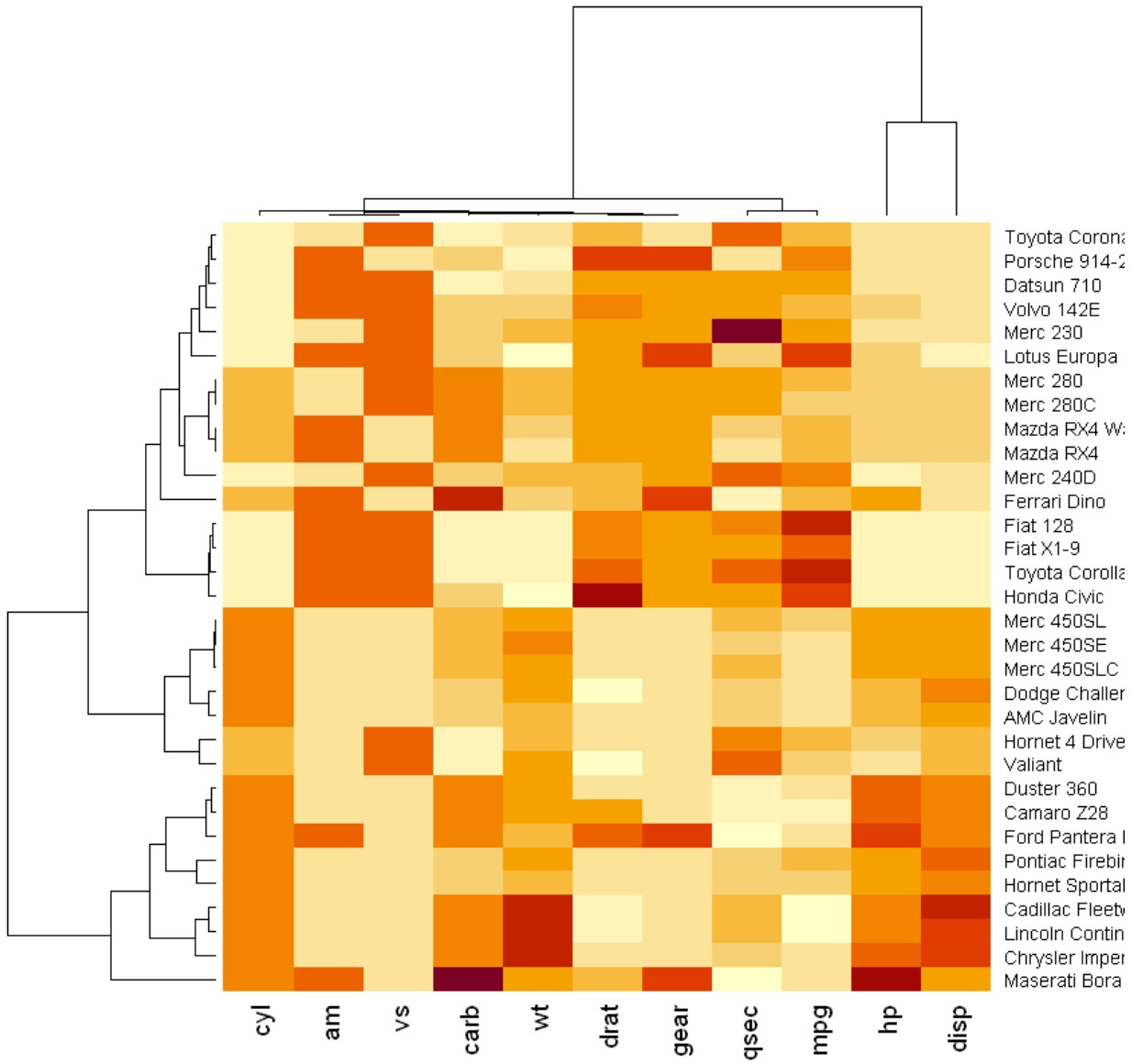
	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Maserati Bora	15.0	8	301.0	335	3.54	3.570	14.60	0	1	5	8
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2



Note that it takes as input a matrix. If you have a data frame, you can convert it to a matrix with as.matrix(), but you need numeric variables only.

Normalizing the matrix is done using the scale argument of the heatmap() function. It can be applied to row or to column. Here the column option is chosen, since we need to absorb the variation between column.

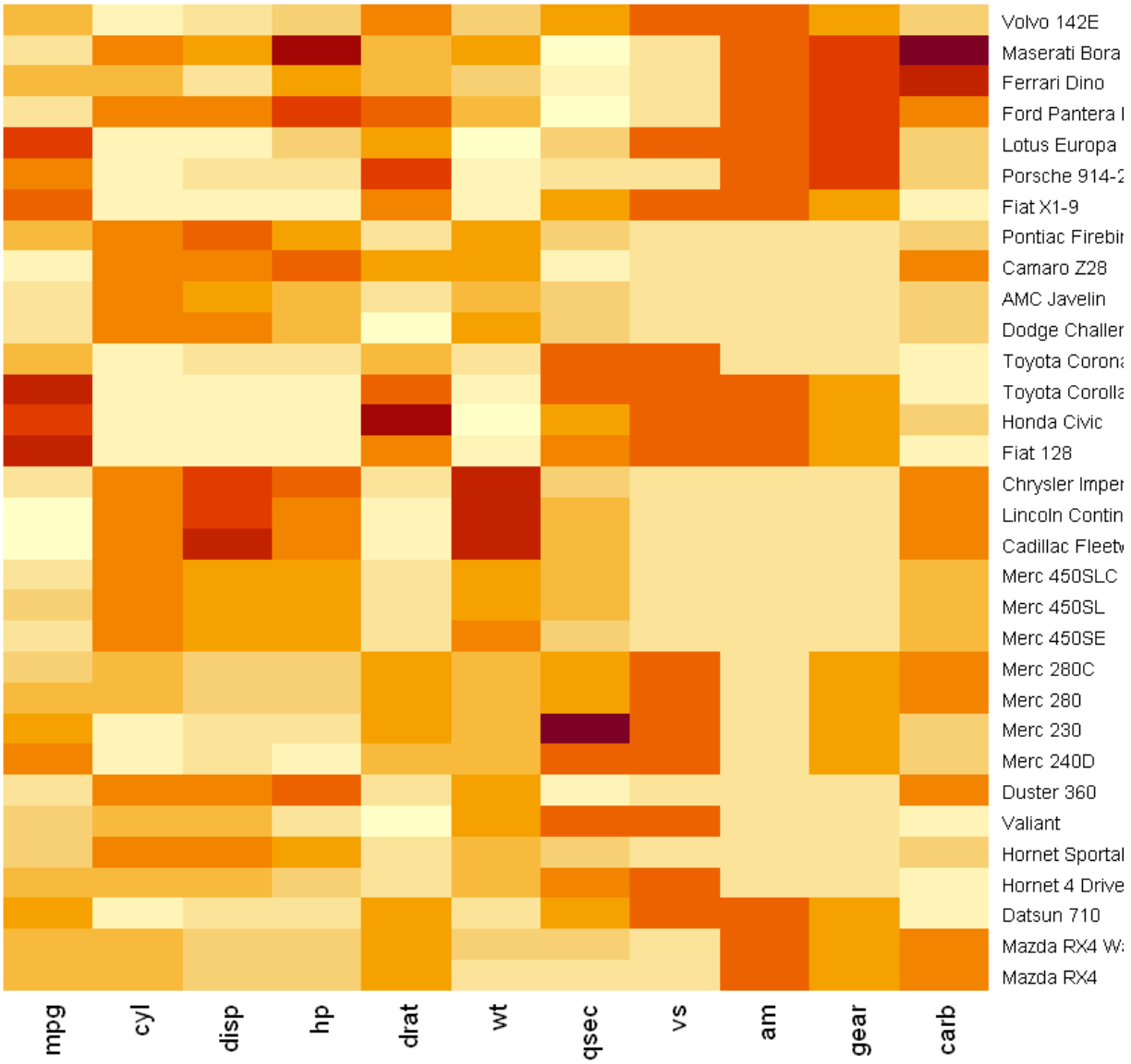
In [23]:
```
# Use 'scale' to normalize
heatmap(data, scale="column")
```



You may have noticed that order of both rows and columns is different compare to the native mtcar matrix. This is because heatmap() reorders both variables and observations using a clustering algorithm: it computes the distance between each pair of rows and columns and try to order them by similarity.

Moreover, the corresponding dendrograms are provided beside the heatmap. We can avoid it and just visualize the raw matrix: use the Rowv and Colv arguments as follow.

In [24]:
```
# No dendrogram nor reordering for neither column or row
heatmap(data, Colv = NA, Rowv = NA, scale="column")
```



Color palette

There are several ways to custom the color palette:

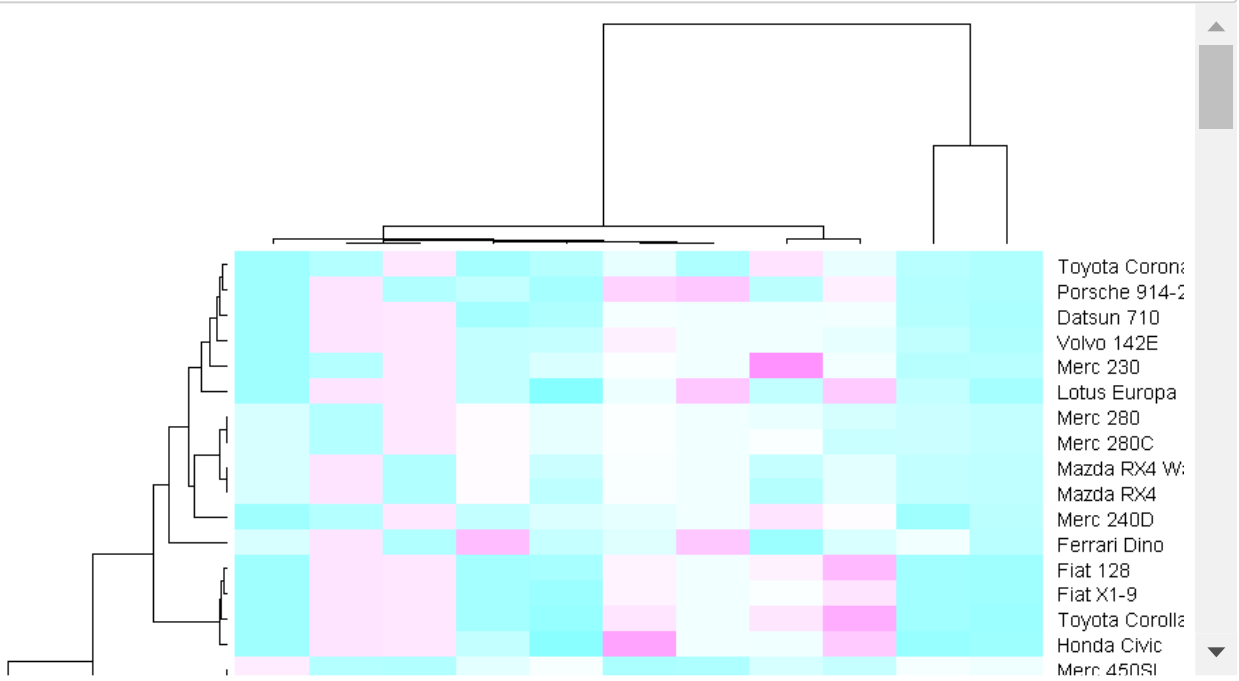
use the native palettes of R:

- terrain.color()
- rainbow()
- heat.colors()

- topo.colors()
- cm.colors()

In [25]:
```
# 1: native palette from R
heatmap(data, scale="column", col = cm.colors(256))
heatmap(data, scale="column", col = terrain.colors(256))

# 2: Rcolorbrewer palette
library(RColorBrewer)
coul <- colorRampPalette(brewer.pal(8, "PiYG"))(25)
heatmap(data, scale="column", col = coul)
```



https://r-graph-gallery.com/215-the-heatmap-function.html (https://r-graph-gallery.com/215-the-heatmap-function.html)

https://r-graph-gallery.com/correlogram.html (https://r-graph-gallery.com/correlogram.html)

In [ ]:

In [ ]:

In [ ]:

In [ ]:

5.9 R graph gallery

5.9.1 https://r-graph-gallery.com/ (https://r-graph-

gallery.com/)

GGPLOT themes:

https://ggplot2.tidyverse.org/reference/ggtheme.html?s=09
(https://ggplot2.tidyverse.org/reference/ggtheme.html?s=09)

In [ ]: